

**RANCANG BANGUN APLIKASI *POINT OF SALE*
DENGAN FITUR PREDIKSI PENJUALAN HARIAN
MENGUNAKAN METODE *LONG SHORT-TERM MEMORY*
(LSTM)**

SKRIPSI

Diajukan Sebagai Salah Satu Syarat Untuk Menyelesaikan
Pendidikan Jenjang Sarjana Terapan
Pada Politeknik Negeri Lhokseumawe



Oleh:

MUHAMMAD KHOLIS

NIM	: 2022573010098
Program Studi	: Teknik Informatika
Jurusan	: Teknologi Informasi dan Komputer

**KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN 2026**

PENGESAHAN PEMBIMBING

Skripsi yang berjudul “*Rancang Bangun Aplikasi Point of Sale dengan Fitur Prediksi Penjualan Harian Menggunakan Metode Long Short-Term Memory (LSTM)*”, disusun oleh Muhammad Kholis, NIM 2022573010098, Program Studi Teknik Informatika, Jurusan Teknologi Informasi dan Komputer Politeknik Negeri Lhokseumawe, telah memenuhi syarat untuk dipertanggungjawabkan di hadapan dewan penguji.

Pembimbing I

Buket Rata, 16 Juli 2026

Pembimbing II

**Zulfan Khairil Simbolon, S.T.,
M.Eng.**

NIP. 196909021993031004

Azhar, S.T., M.T.

NIP. 196408301990031005

Mengetahui,

Ketua Jurusan
Teknologi Informasi dan
Komputer,

Ketua Program Studi
Teknik Informatika,

Salahuddin, S.T., M.Sc.
NIP. 197404242002121001

M. Khadafi, S.T., M.T
NIP. 197507182002121004

PENGESAHAN PENGUJI

Skripsi yang berjudul “*Rancang Bangun Aplikasi Point of Sale dengan Fitur Prediksi Penjualan Harian Menggunakan Metode Long Short-Term Memory (LSTM)*”, disusun oleh Muhammad Kholis, NIM 2022573010098, Program Studi Teknik Informatika, Jurusan Teknologi Informasi dan Komputer Politeknik Negeri Lhokseumawe, telah dipertanggungjawabkan di hadapan dewan penguji dan dinyatakan lulus pada tanggal 27 Juli 2026.

Komisi Sidang,

Zulfan Khairil Simbolon, S.T., M.Eng. (Ketua Sidang)
NIP. 196909021993031004

Azhar, S.T., M.T. (Sekretaris Sidang)
NIP. 196408301990031005

Musta'inul Abdi, S.S.T., M.Kom. (Penguji I)
NIP. 199110302019031015

Dr. Rahmad Hidayat, S.Kom., M.Cs. (Penguji II)
NIP. 198304202012121003

Radhiyattammardhiyyah, S.S.T., M.Sc. (Penguji III)
NIP. 199208262022032011

Mengetahui,

Ketua Jurusan
Teknologi Informasi dan
Komputer,

Ketua Program Studi
Teknik Informatika,

Salahuddin, S.T., M.Sc.
NIP. 197404242002121001

M. Khadafi, S.T., M.T
NIP. 197507182002121004

HALAMAN PERNYATAAN

Dengan ini saya menyatakan bahwa dalam Skripsi ini tidak terdapat karya yang pernah diajukan untuk mendapatkan gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis orang lain, kecuali yang secara tertulis diacu dalam naskah dan disebut dalam daftar pustaka.

Lhokseumawe, 16 Juli 2026

Muhammad Kholis
NIM. 2022573010098

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Allah SWT, karena atas rahmat dan hidayah-Nya, penulis dapat menyelesaikan skripsi ini dengan judul *Rancang Bangun Aplikasi Point of Sale dengan Fitur Prediksi Penjualan Harian Menggunakan Metode Long Short-Term Memory (LSTM)*. Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Terapan pada Program Studi Teknik Informatika Politeknik Negeri Lhokseumawe. Penulis menyadari bahwa skripsi ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang bersifat membangun dari semua pihak.

Dalam penulisan Skripsi ini penulis dapat melaksanakannya dengan baik berkat dukungan, bimbingan, dan bantuan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Allah SWT atas segala rahmat, nikmat, kesehatan, serta kekuatan yang diberikan kepada penulis sehingga dapat menyelesaikan skripsi ini.
2. Keluarga tercinta, terutama Ibunda Jumiati yang selalu memberikan doa, dukungan moral maupun material, serta motivasi kepada penulis selama proses penyelesaian pendidikan.
3. Bapak Dr. Ir. Rizal Syahyadi, S.T., M.Eng.Sc., IPM, ASEAN Eng., APEC Eng., selaku Direktur Politeknik Negeri Lhokseumawe.
4. Bapak Salahuddin, S.T., M.Sc. selaku Ketua Jurusan Teknologi Informasi dan Komputer, Bapak Fachri Yanuar Rudi F, S.S.T., M.T. selaku Sekretaris Jurusan Teknologi Informasi dan Komputer, dan Bapak M. Khadafi, S.T., M.T. selaku Ketua Program Studi Teknik Informatika atas segala bimbingan dan perhatian selama masa perkuliahan.
5. Bapak Zulfan Khairil Simbolon, S.T., M.Eng. selaku Pembimbing Utama dan Bapak Azhar, S.T., M.T. selaku Pembimbing Pendamping yang telah membantu, membimbing, dan mengarahkan penulis dalam proses penyelesaian Skripsi ini.
6. Bapak Musta'inul Abdi, S.S.T., M.Kom., Bapak Dr. Rahmad Hidayat, S.Kom., M.Cs., dan Ibu Radhiyatammardhiyyah, S.S.T., M.Sc. selaku penguji yang telah bersedia memberikan arahan dan masukan dalam pengerjaan Skripsi ini.
7. Seluruh Dosen Pengajar Program Studi Teknik Informatika yang telah memberikan ilmu dan motivasi kepada penulis selama masa perkuliahan.

Penulis menyadari bahwa Skripsi ini masih terdapat kekurangan dan ketidaksempurnaan. Oleh sebab itu, penulis mengharapkan adanya saran dan kritik yang membangun guna dijadikan bahan perbaikan dan pembelajaran untuk ke depannya yang bermanfaat bagi penulis dan rekan-rekan yang membacanya. Semoga bantuan dan kebaikan semua pihak dapat menjadi amal ibadah.

Lhokseumawe, 16 Juli 2026

Muhammad Kholis
NIM. 2022573010098

ABSTRAK

Penelitian ini bertujuan merancang dan membangun aplikasi *Point of Sale* (POS) berbasis *mobile* dengan arsitektur *offline-first* yang dilengkapi fitur prediksi penjualan harian menggunakan metode *Long Short-Term Memory* (LSTM), guna membantu pelaku UMKM, dalam hal ini kantin EatsTEDI Universitas Gadjah Mada sebagai mitra penelitian, dalam pengambilan keputusan operasional berbasis data. Sistem dikembangkan menggunakan *framework* Flutter yang terintegrasi dengan basis data lokal Isar DB dan basis data *cloud* Supabase PostgreSQL, sedangkan modul prediksi dibangun menggunakan Python (TensorFlow dan Flask) yang di-*deploy* sebagai layanan API terpisah. Data yang digunakan berupa 313 hari aktif transaksi penjualan harian yang diolah sebagai data deret waktu (*time series*) dengan pendekatan target *log-return*. Kinerja model LSTM dievaluasi menggunakan metrik MAE, RMSE, sMAPE, dan MASE, serta dibandingkan dengan model pembanding *Naive*, *Mean-7*, dan SARIMA melalui pengujian *multi-seed*. Hasil evaluasi menunjukkan bahwa model LSTM *Hybrid* (MAE rata-rata Rp667.718,00, MASE 1,840) belum mampu mengungguli metode *Naive* (MAE Rp519.841,00, MASE 1,432) akibat keterbatasan volume data historis dan dominasi persistensi jangka pendek pada data penjualan. Eksperimen augmentasi data sintetis menggunakan TimeGAN berhasil menurunkan galat model LSTM multi-langkah secara signifikan, namun tetap belum melampaui *baseline Seasonal-Naive*. Berdasarkan temuan tersebut, layanan API produksi menyajikan metode *Seasonal-Naive* sebagai metode penyaji utama prediksi harian. Pengujian fungsional *Black Box* terhadap 14 skenario dan pengujian *White Box* terhadap fungsi kritis layanan prediksi menunjukkan tingkat keberhasilan 100%, sehingga sistem POS *offline-first* ini terbukti layak dioperasikan secara penuh dalam mendukung operasional UMKM.

Kata Kunci: *Point of Sale* (POS), Prediksi Penjualan Harian, *Long Short-Term Memory* (LSTM), *Offline-First*, TimeGAN, UMKM

ABSTRACT

This study aims to design and develop a mobile-based Point of Sale (POS) application with an offline-first architecture, equipped with a daily sales prediction feature using the Long Short-Term Memory (LSTM) method, to assist MSMEs, in this case the EatsTEDI canteen of Universitas Gadjah Mada as the research partner, in data-driven operational decision-making. The system was developed using the Flutter framework integrated with the local Isar DB database and the Supabase PostgreSQL cloud database, while the prediction module was built using Python (TensorFlow and Flask) deployed as a separate API service. The data used consist of 313 active days of daily sales transactions processed as time-series data using a log-return target approach. The performance of the LSTM model was evaluated using the MAE, RMSE, sMAPE, and MASE metrics, and compared against the Naive, Mean-7, and SARIMA baseline models through multi-seed testing. The evaluation results show that the Hybrid LSTM model (average MAE of IDR 667,718.00 and MASE of 1.840) was unable to outperform the Naive method (MAE of IDR 519,841.00 and MASE of 1.432) due to the limited volume of historical data and the dominance of short-term persistence in the sales data. A synthetic data augmentation experiment using TimeGAN significantly reduced the error of the multi-step LSTM model, yet still did not surpass the Seasonal-Naive baseline. Based on these findings, the production API service presents the Seasonal-Naive method as the primary forecasting method for daily predictions. Black Box functional testing on 14 scenarios and White Box testing on the critical functions of the prediction service showed a 100% success rate, proving that this offline-first POS system is fully viable for supporting MSME operations.

Keywords: *Point of Sale (POS), Daily Sales Prediction, Long Short-Term Memory (LSTM), Offline-First, TimeGAN, MSMEs*

DAFTAR ISI

PENGESAHAN PEMBIMBING	ii
PENGESAHAN PENGUJI	iii
HALAMAN PERNYATAAN.....	iv
KATA PENGANTAR	v
ABSTRAK	vii
ABSTRACT.....	viii
DAFTAR ISI	ix
DAFTAR GAMBAR	xii
DAFTAR TABEL	xiv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	6
1.3 Tujuan Penelitian	7
1.4 Batasan Masalah.....	7
1.5 Manfaat Penelitian	9
1.6 Sistematika Penulisan	10
BAB II TINJAUAN PUSTAKA	13
2.1 <i>State of the Art</i>	13
2.2 Tinjauan Teoritis.....	20
2.2.1 <i>Point of Sale (POS)</i>	20
2.2.2 Peramalan Deret Waktu (<i>Time Series Forecasting</i>)	21
2.2.3 <i>Baseline Seasonal-Naive</i>	22
2.2.4 <i>Recurrent Neural Network (RNN)</i>	23
2.2.5 <i>Long Short-Term Memory (LSTM)</i>	24
2.2.6 <i>Generative Adversarial Network (GAN)</i> dan TimeGAN ...	26
2.2.7 Praproses Data	27
2.2.8 Metrik Evaluasi	28
2.2.9 Flutter.....	30
2.2.10 Pengujian <i>Black Box</i>	30
BAB III METODOLOGI PENELITIAN	32
3.1 Data dan Pengumpulan Data.....	32
3.2 Analisis Kebutuhan Sistem	36
3.2.1 Analisis Kebutuhan Fungsional	36
3.2.2 Analisis Kebutuhan Non-Fungsional	37

3.3	Perancangan Sistem	39
3.3.1	Arsitektur Sistem	39
3.3.2	Pemodelan Sistem <i>Unified Modeling Language</i> (UML)	42
3.3.3	Perancangan Antarmuka Pengguna (<i>User Interface</i>)	62
3.4	Perancangan Metode Penelitian	75
3.4.1	Metode Penelitian.....	75
3.4.1.1	Alur Penelitian	77
3.4.2	Rancangan Algoritma <i>Long Short-Term Memory</i> (LSTM) .	79
3.5	Perancangan Pengujian.....	87
3.5.1	Teknik Pengujian	87
3.5.1.1	Rancangan Pengujian Evaluasi Model Prediksi	87
3.5.1.2	Rancangan Analisis Pengujian <i>Black Box</i>	88
3.5.1.3	Rancangan Analisis Pengujian <i>White Box</i>	91
3.5.2	Indikator Keberhasilan Penelitian	92
BAB IV	HASIL DAN PEMBAHASAN	93
4.1	Hasil Implementasi Sistem.....	94
4.1.1	Implementasi Antarmuka (UI)	94
4.1.2	Implementasi Backend dan Basis Data Cloud.....	110
4.1.3	Implementasi Layanan Prediksi (API)	111
4.2	Preprocessing Data	115
4.2.1	Deskripsi Data dan Eksplorasi Awal	115
4.2.2	Pra-pemrosesan dan Pembentukan Sequence	116
4.3	Implementasi dan Pelatihan Model Prediksi	118
4.3.1	Arsitektur Model dan Skenario Eksperimen.....	119
4.3.2	Hasil Pelatihan Model	120
4.3.3	Implementasi Augmentasi Data Sintetis TimeGAN	122
4.3.4	Implementasi Prediksi Rekursif ke Depan.....	124
4.4	Evaluasi Model	126
4.4.1	Evaluasi Model dan Perbandingan dengan Baseline	128
4.4.2	Evaluasi Multi-Seed dan Analisis Kestabilan	130
4.4.3	Evaluasi Augmentasi Data Sintetis TimeGAN	132
4.5	Hasil Pengujian Sistem.....	134
4.5.1	Skenario Pengujian	134
4.5.2	Hasil Black Box Testing	135
4.5.3	Hasil White Box Testing	137
4.5.4	Evaluasi Kinerja Model.....	140
4.6	Pembahasan.....	140

4.6.1	Pembahasan Pengujian Fungsional dan Arsitektur Offline-First	140
4.6.2	Pembahasan Hasil Prediksi Penjualan LSTM	141
4.7	Keterbatasan Sistem dan Penelitian	144
4.7.1	Keterbatasan Sistem POS.....	144
4.7.2	Keterbatasan Penelitian Model Prediksi	145
BAB V	PENUTUP.....	148
5.1	Kesimpulan	148
5.2	Saran	150
	DAFTAR PUSTAKA.....	153
	LAMPIRAN	156

DAFTAR GAMBAR

3.1	Arsitektur Sistem POS	40
3.2	<i>Use Case Diagram</i> Sistem POS	44
3.3	Activity Diagram Alur Kerja Transaksi Kasir	54
3.4	Activity Diagram Sinkronisasi Latar Belakang.....	55
3.5	Activity Diagram Proses AI Smart Analytics	56
3.6	Activity Diagram Kelola Produk (Tambah/Ubah/Hapus)	57
3.7	Class Diagram Sistem POS	58
3.8	<i>Entity Relationship Diagram</i> (ERD) Sistem POS	60
3.9	<i>Wireframe</i> Halaman Registrasi Toko Baru	63
3.10	<i>Wireframe</i> Halaman <i>Login</i>	64
3.11	<i>Wireframe</i> Halaman Pemilihan Toko.....	65
3.12	<i>Wireframe</i> Halaman Dasbor Owner	66
3.13	<i>Wireframe</i> Halaman POS (Pembayaran)	67
3.14	<i>Wireframe</i> Halaman Riwayat Transaksi	68
3.15	<i>Wireframe</i> Halaman Kelola Produk	69
3.16	<i>Wireframe</i> Halaman Kelola Kategori	70
3.17	<i>Wireframe</i> Halaman Laporan	71
3.18	<i>Wireframe</i> Halaman Pengaturan.....	72
3.19	<i>Wireframe</i> Halaman Informasi Toko	73
3.20	<i>Wireframe</i> Halaman Struk Digital	74
3.21	<i>Wireframe</i> Halaman Katalog Produk.....	75
3.22	Alur Penelitian	77
3.23	Alur Kerja Algoritma <i>Long Short-Term Memory</i> (LSTM).....	80
4.1	Implementasi Halaman Registrasi	95
4.2	Implementasi Halaman <i>Login</i>	96
4.3	Implementasi Halaman Informasi Toko	97
4.4	Implementasi Halaman Pilih Toko	98
4.5	Implementasi Halaman Dasbor Owner	99
4.6	Implementasi Halaman Kelola Produk dan Tambah Produk.....	100
4.7	Implementasi Halaman Kelola Kategori	101
4.8	Implementasi Halaman Kelola Stok dan Ubah Stok Produk.....	102
4.9	Implementasi Halaman Riwayat Stok.....	103
4.10	Implementasi Halaman Kasir (POS) dan Detail Pesanan	104
4.11	Implementasi Halaman Pembayaran	105

4.12 Implementasi Halaman Riwayat Transaksi	106
4.13 Implementasi Halaman Struk Digital	107
4.14 Implementasi Halaman Laporan Analitik.....	108
4.15 Implementasi Halaman <i>Smart Analitik</i>	109
4.16 Implementasi Halaman Riwayat Analisis.....	110
4.17 Kurva Kerugian Pelatihan (<i>Training Loss Curve</i>) Model LSTM	121
4.18 Grafik Pendapatan Aktual vs. Prediksi Model LSTM Hybrid	129
4.19 Boxplot Distribusi MAE Hasil Pengujian <i>Multi-Seed</i>	132

DAFTAR TABEL

2.1	<i>State of the Art</i> Penelitian Terdahulu.....	15
3.1	Struktur Data <i>Dataset</i> Final Penelitian	33
3.2	Spesifikasi Perangkat Keras	37
3.3	Spesifikasi Perangkat Lunak	38
3.4	<i>Use Case</i> Registrasi Toko Baru	45
3.5	<i>Use Case</i> Login Akun	46
3.6	<i>Use Case</i> Mengelola Katalog Produk	46
3.7	<i>Use Case</i> Mengelola Kategori.....	47
3.8	<i>Use Case</i> Melakukan <i>Checkout</i>	48
3.9	<i>Use Case</i> Mengaudit Riwayat Stok	48
3.10	<i>Use Case</i> Melihat <i>Dashboard</i>	49
3.11	<i>Use Case</i> Melihat Prediksi Penjualan Harian.....	49
3.12	<i>Use Case</i> Melihat Riwayat Analisis	50
3.13	<i>Use Case</i> Sinkronisasi Data	51
3.14	<i>Use Case</i> Kirim <i>Broadcast</i> Notifikasi.....	51
3.15	<i>Use Case</i> Mengelola Karyawan	52
3.16	<i>Use Case</i> Mengatur Profil Toko	52
3.17	Daftar Tabel Basis Data POS	61
3.18	Skenario Pengujian <i>Black Box</i>	89
4.1	Statistik Ringkas Dataset Pendapatan Harian	115
4.2	Pembagian Dataset dan Dimensi Sequence.....	117
4.3	Perbandingan Performa pada Data Uji (<i>One-Step-Ahead</i>)	128
4.4	Statistik Performa LSTM Hybrid pada 10 Seed	131
4.5	Perbandingan Performa Eksperimen Augmentasi TimeGAN (<i>Multi-Step</i> , Horizon 7)	133
4.6	Hasil Pengujian Black Box	135
4.7	Hasil Pengujian Mekanisme Sinkronisasi	137
4.8	Hasil Pengujian White Box.....	139

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Transformasi digital pada sektor ritel dan usaha mikro, kecil, dan menengah (UMKM) telah menjadikan aplikasi *Point of Sale* (POS) sebagai infrastruktur transaksi yang tidak lagi bersifat opsional. Aplikasi POS modern tidak hanya mencatat transaksi penjualan, tetapi juga menghasilkan data transaksional harian yang bersifat deret waktu (*time series*) secara terus-menerus. Namun demikian, sebagian besar aplikasi POS yang beredar, khususnya pada skala usaha kecil seperti kantin institusi, masih memposisikan data tersebut sebatas sebagai arsip pelaporan dan belum memanfaatkannya sebagai basis pengambilan keputusan yang bersifat prediktif. Akibatnya, keputusan operasional seperti penentuan volume produksi harian, pengadaan bahan baku, dan penjadwalan tenaga kerja masih banyak ditentukan berdasarkan intuisi pengelola, yang berisiko menimbulkan kelebihan persediaan (*overstock*) maupun kehabisan persediaan (*stockout*).

Ketidakakuratan estimasi permintaan harian berdampak langsung pada efisiensi biaya dan tingkat layanan. Peramalan penjualan yang akurat berperan penting dalam menyeimbangkan permintaan dan pasokan serta meningkatkan profitabilitas pada sektor ritel [1]. Karakteristik data penjualan harian pada usaha kuliner institusional bersifat kompleks: terdapat musiman mingguan berupa perbedaan volume antara hari kerja dan akhir pekan, pengaruh kalender akademik,

hari libur nasional, serta fluktuasi yang tidak beraturan. Metode statistik konvensional seperti *moving average* dan ARIMA memiliki keterbatasan dalam menangkap pola non-linear dan ketergantungan temporal jangka panjang pada data dengan karakteristik semacam ini.

Untuk mengatasi keterbatasan tersebut, pendekatan *deep learning* berbasis *Recurrent Neural Network* (RNN), khususnya *Long Short-Term Memory* (LSTM), telah menjadi salah satu metode mutakhir dalam peramalan deret waktu penjualan. LSTM memanfaatkan mekanisme gerbang (*gating mechanism*) untuk mempertahankan informasi lintas rentang waktu yang panjang sekaligus mengatasi masalah *vanishing gradient* yang melekat pada RNN tradisional [2]. Sejumlah penelitian terkini menunjukkan keunggulan pendekatan berbasis LSTM. Optimasi *hyperparameter* menggunakan Keras Tuner pada LSTM untuk prediksi penjualan harian *dataset* ritel Walmart menghasilkan peningkatan akurasi yang signifikan dibandingkan metode peramalan tradisional [3]. De Castro Moraes dkk. [4] mengembangkan arsitektur hibrida CNN-LSTM bertumpuk (*stacked*) dan paralel untuk menangani pola musiman yang bervariasi serta korelasi antarproduk pada 500 deret waktu ritel. Sementara itu, Mansur dkk. [1] menunjukkan bahwa model hibrida CNN-LSTM yang diperkaya variabel eksogen berupa hari libur, hari gajian, aksi unjuk rasa, dan kondisi cuaca mampu mencapai MAPE sebesar 4,16%, mengungguli model ANN, CNN, dan LSTM tunggal. Perkembangan ini memperkuat posisi LSTM sebagai metode yang relevan untuk diintegrasikan ke dalam sistem POS.

Kendati demikian, terdapat dua kondisi yang membatasi keterbukaan hasil tersebut untuk direplikasi pada usaha berskala kecil. Pertama, model *deep learning* bersifat *data-hungry*, yaitu performanya sangat bergantung pada ukuran *dataset*, sehingga pelatihan dengan data yang tidak memadai secara nyata membatasi kegunaannya [5]. Beberapa penelitian yang melaporkan akurasi tinggi memang menggunakan data berskala besar, seperti [4] yang memanfaatkan riwayat transaksi lima tahun. Kedua, ketika akurasi tinggi berhasil dicapai pada data berukuran kecil, keberhasilan tersebut umumnya bertumpu pada ketersediaan variabel eksogen yang kaya dan bukan pada panjang deret waktunya semata. Penelitian [1] mencapai MAPE 4,16% hanya dengan enam bulan data harian dari satu gerai, namun dengan tambahan tujuh variabel cuaca dan peristiwa, sementara penulisnya sendiri mencatat adanya indikasi *overfitting* pada kurva *validation loss*. Kondisi ini menyisakan persoalan bagi kasus di mana variabel eksogen semacam itu tidak tersedia. Sebagai jalan keluar, augmentasi data sintetis menggunakan *Time-series Generative Adversarial Network* (TimeGAN) [6] diusulkan untuk memperkaya sampel pelatihan dan terbukti menurunkan RMSE serta MAE pada beberapa domain [7]. Namun literatur juga mencatat bahwa manfaat augmentasi semakin tidak signifikan seiring bertambahnya ukuran data awal [8], sehingga efektivitas TimeGAN pada data penjualan harian berskala kecil masih merupakan pertanyaan terbuka yang layak diuji secara empiris.

Celah kedua bersifat metodologis. Sebagian besar penelitian peramalan penjualan berbasis LSTM membandingkan modelnya dengan sesama model

machine learning, tetapi jarang mengujinya secara ketat terhadap *baseline* heuristik sederhana seperti *seasonal-naive*, padahal pada data dengan musiman mingguan yang kuat *baseline* semacam ini kerap sangat kompetitif. Pengalaman kompetisi peramalan berskala besar menunjukkan bahwa metode sederhana masih menjadi pembanding yang sulit dikalahkan dan karenanya wajib disertakan sebagai kontrol metodologis [9]. Celah ketiga bersifat implementatif: kajian yang ada umumnya berhenti pada tahap eksperimen model dan belum sampai pada penyatuan model ke dalam aplikasi POS yang benar-benar digunakan pengguna akhir, sehingga jarak antara riset akademik dan penerapan praktis di ritel masih lebar [10]. Penelitian ini menempatkan diri tepat pada ketiga celah tersebut, yaitu membangun aplikasi POS yang di dalamnya tertanam fitur prediksi penjualan harian berbasis LSTM, mengevaluasi kontribusi augmentasi TimeGAN, serta menguji model secara jujur terhadap *baseline seasonal-naive*.

Studi kasus penelitian ini menggunakan data transaksi platform EatsTEDI pada Departemen Teknik Elektro dan Informatika (DTEDI), Universitas Gadjah Mada. Data mentah terhimpun dalam rentang 24 Agustus 2024 sampai dengan 20 Juni 2026, mencakup 666 hari kalender atau sekitar 22 bulan operasional. Setelah proses penyaringan, hari aktif pertama dengan transaksi tercatat adalah 26 Agustus 2024, sehingga rentang efektif yang digunakan adalah 26 Agustus 2024 sampai dengan 20 Juni 2026 dan menghasilkan 316 hari dengan transaksi tercatat. Rentang efektif tersebut memuat 475 hari kerja secara kalender, sehingga terdapat sejumlah hari tanpa transaksi yang berasal dari masa libur antarsemester, hari libur

nasional, serta penutupan operasional kantin. Ketimpangan ini bukan merupakan kehilangan data yang bersifat acak, melainkan cerminan langsung dari ritme institusional yang justru menjadi objek pemodelan dalam penelitian ini.

Karakteristik demikian menjadikan *dataset* EatsTEDI representatif untuk persoalan yang diangkat. Secara temporal, rentang 22 bulan mencakup empat semester berturut-turut, yakni semester gasal dan genap tahun akademik 2024/2025 serta semester gasal dan genap tahun akademik 2025/2026. Cakupan dua tahun akademik penuh ini memungkinkan pola musiman berbasis kalender akademik, meliputi periode perkuliahan normal, minggu ujian tengah semester, minggu ujian akhir semester, dan masa libur antarsemester, teramati secara berulang dan bukan hanya sekali, sehingga model memiliki peluang mempelajarinya sebagai pola dan bukan sebagai anomali. Secara struktural, data juga memuat musiman mingguan berupa perbedaan volume transaksi antara hari kerja dan akhir pekan. Namun demikian, dengan hanya 316 titik pengamatan, jumlah sampel pelatihan yang tersedia bagi model LSTM tetap tergolong sangat terbatas apabila dibandingkan dengan penelitian sejenis yang menggunakan ribuan hingga jutaan catatan transaksi [3], [4]. Kondisi inilah yang menempatkan *dataset* EatsTEDI sebagai lingkungan uji yang tepat, yaitu cukup kaya secara pola musiman untuk menjadi persoalan peramalan yang bermakna, namun cukup terbatas secara volume untuk menguji secara jujur apakah kompleksitas LSTM dan augmentasi sintetis TimeGAN benar-benar memberikan nilai tambah dibandingkan pendekatan sederhana pada skala usaha kecil.

Berdasarkan uraian tersebut, penelitian ini merancang dan membangun aplikasi *Point of Sale* dengan fitur prediksi penjualan harian menggunakan metode *Long Short-Term Memory*. Kontribusi penelitian mencakup tiga aspek. Pertama, perancangan arsitektur aplikasi POS yang mengintegrasikan modul prediksi ke dalam alur kerja transaksi harian sehingga hasil peramalan dapat langsung dimanfaatkan pengelola. Kedua, evaluasi empiris pengaruh augmentasi data sintesis TimeGAN terhadap performa LSTM pada *dataset* penjualan berskala kecil. Ketiga, perbandingan model terhadap *baseline seasonal-naive* pada beberapa horizon peramalan sebagai kontrol metodologis, guna menghasilkan rekomendasi yang jujur mengenai kondisi di mana pendekatan *deep learning* layak diterapkan pada konteks usaha kuliner berskala kecil di Indonesia.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun aplikasi *Point of Sale* yang mengintegrasikan fitur prediksi penjualan harian ke dalam alur kerja transaksi, sehingga hasil peramalan dapat langsung dimanfaatkan oleh pengelola usaha?
2. Bagaimana menerapkan model *Long Short-Term Memory* untuk memprediksi penjualan harian pada *dataset* EatsTEDI yang berjumlah 316 titik pengamatan, serta bagaimana pengaruh augmentasi data sintesis

Time-series Generative Adversarial Network (TimeGAN) terhadap akurasi model tersebut?

3. Bagaimana tingkat akurasi model *Long Short-Term Memory* dibandingkan terhadap *baseline seasonal-naive* pada beberapa horizon peramalan?

1.3 Tujuan Penelitian

Sejalan dengan rumusan masalah tersebut, penelitian ini bertujuan untuk:

1. Merancang dan membangun aplikasi *Point of Sale* berbasis *mobile* yang mengintegrasikan fitur prediksi penjualan harian ke dalam alur kerja transaksi, serta menguji fungsionalitasnya melalui pengujian sistem.
2. Menerapkan model *Long Short-Term Memory* untuk prediksi penjualan harian pada *dataset* EatsTEDI dan menganalisis pengaruh augmentasi data sintetis TimeGAN terhadap akurasi model pada kondisi data pelatihan yang terbatas.
3. Mengukur tingkat akurasi model *Long Short-Term Memory* terhadap *baseline seasonal-naive* pada beberapa horizon peramalan, guna menghasilkan rekomendasi mengenai kondisi kelayakan penerapan pendekatan *deep learning* pada usaha kuliner berskala kecil.

1.4 Batasan Masalah

Agar penelitian ini terarah dan terukur, ditetapkan batasan sebagai berikut:

1. Sumber data. Data yang digunakan adalah data transaksi platform EatsTEDI

pada Departemen Teknik Elektro dan Informatika (DTEDI), Universitas Gadjah Mada, dengan rentang efektif 26 Agustus 2024 sampai dengan 20 Juni 2026 dan menghasilkan 316 hari dengan transaksi tercatat.

2. Cakupan variabel. Pemodelan dilakukan secara *univariat* dengan variabel target berupa total nilai penjualan harian. Variabel eksogen seperti kondisi cuaca, promosi, dan harga tidak disertakan karena tidak tersedia pada *dataset*.
3. Granularitas prediksi. Prediksi dilakukan pada tingkat agregat penjualan harian, bukan pada tingkat *item* atau kategori produk secara individual.
4. Cakupan metode. Metode peramalan yang diimplementasikan dibatasi pada arsitektur *Long Short-Term Memory* dengan *baseline seasonal-naive* sebagai pembanding. Arsitektur lain seperti *Transformer*, *Temporal Fusion Transformer*, maupun model statistik ARIMA/SARIMA tidak menjadi objek pengujian dalam penelitian ini.
5. Metode augmentasi. Augmentasi data sintetis dibatasi pada metode TimeGAN. Metode augmentasi lain seperti *jittering*, *window warping*, atau augmentasi domain frekuensi tidak diuji.
6. Metrik evaluasi. Evaluasi performa model menggunakan metrik MAE, RMSE, dan MAPE, serta dibandingkan secara relatif terhadap *baseline* melalui rasio kesalahan.
7. Cakupan aplikasi. Aplikasi POS dikembangkan pada platform Android menggunakan *framework* Flutter. Pengembangan untuk platform iOS berada di luar cakupan penelitian.

8. Mekanisme pelatihan model. Model dilatih secara luring (*offline*) dan hasil pelatihan diintegrasikan ke dalam aplikasi. Penelitian ini tidak mencakup mekanisme pelatihan ulang otomatis (*online* atau *incremental learning*) seiring bertambahnya data transaksi.
9. Cakupan pengujian. Pengujian dibatasi pada pengujian fungsionalitas sistem dan pengujian akurasi model. Pengukuran dampak ekonomi penerapan sistem, seperti penurunan biaya persediaan atau tingkat kepuasan pengguna, tidak termasuk dalam ruang lingkup penelitian.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat pada dua tataran. Secara teoritis, penelitian ini menyediakan bukti empiris mengenai performa model *Long Short-Term Memory* pada data penjualan harian berskala kecil dengan karakteristik deret tidak sinambung, suatu kondisi yang selama ini kurang terwakili dalam literatur karena sebagian besar penelitian menggunakan *dataset* berskala besar dan sinambung, sekaligus menguji efektivitas augmentasi data sintetis TimeGAN sebagai upaya mengatasi keterbatasan jumlah sampel pelatihan. Penelitian ini juga menegaskan pentingnya penyertaan *baseline* heuristik sederhana seperti *seasonal-naive* sebagai kontrol metodologis dalam penelitian peramalan berbasis *deep learning*, sehingga dapat menjadi rujukan bagi kajian selanjutnya yang mengintegrasikan model peramalan ke dalam aplikasi transaksional. Secara praktis, penelitian ini menghasilkan aplikasi *Point of Sale* yang tidak hanya

mencatat transaksi tetapi juga menyajikan estimasi penjualan harian sebagai bahan pertimbangan pengelola dalam menentukan volume produksi, pengadaan bahan baku, dan penjadwalan tenaga kerja; memberikan gambaran mengenai pola permintaan kantin institusional yang terkait dengan ritme kalender akademik; serta menyediakan rujukan teknis bagi pengembang perangkat lunak mengenai arsitektur penyatuan model *machine learning* ke dalam aplikasi *mobile*. Lebih lanjut, hasil perbandingan terhadap *baseline* sederhana diharapkan menjadi rekomendasi berbasis bukti bagi pelaku usaha mikro, kecil, dan menengah mengenai kondisi kelayakan penerapan pendekatan *deep learning* pada skala usaha kecil, sehingga sumber daya pengembangan dapat dialokasikan secara proporsional.

1.6 Sistematika Penulisan

Sistematika penulisan ini merupakan kebulatan seluruh materi penyusunan yang diungkapkan secara garis besar dan dibagi menjadi lima bab. Masing-masing bab terdiri dari beberapa sub bab. Adapun susunan sistematikanya adalah sebagai berikut:

BAB I PENDAHULUAN

Bab ini memuat latar belakang masalah yang menjelaskan urgensi prediksi penjualan harian pada aplikasi *Point of Sale* skala usaha kecil, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Bab ini menyajikan *state of the art* penelitian terdahulu mengenai peramalan penjualan dan sistem *Point of Sale*, serta landasan teori yang meliputi konsep peramalan deret waktu, arsitektur *Long Short-Term Memory*, model pembandingan (*baseline*), metrik evaluasi prediksi, *Generative Adversarial Network* dan TimeGAN, teknologi pengembangan aplikasi, serta metode pemodelan dan pengujian perangkat lunak.

BAB III METODOLOGI PENELITIAN

Bab ini membahas metodologi penelitian yang digunakan, mencakup sumber data dan proses pengumpulan *dataset* EatsTEDI, variabel penelitian, *preprocessing* data, rancangan sistem berupa diagram UML dan ERD, perancangan antarmuka, rancangan algoritma *Long Short-Term Memory*, serta teknik pengujian dan indikator keberhasilan penelitian.

BAB IV HASIL DAN PEMBAHASAN

Bab ini membahas hasil implementasi sistem secara menyeluruh, mencakup tampilan antarmuka aplikasi *Point of Sale* dan layanan API prediksi, tahapan *preprocessing* data, implementasi dan pelatihan model *Long Short-Term Memory*, eksperimen augmentasi data sintetis TimeGAN, hasil evaluasi model terhadap *baseline seasonal-naive*, serta hasil pengujian fungsional sistem beserta pembahasan temuan dan keterbatasan penelitian.

BAB V PENUTUP

Bab ini menyajikan simpulan dari seluruh proses penelitian berdasarkan hasil yang diperoleh, serta saran untuk pengembangan sistem dan penelitian lanjutan di bidang peramalan penjualan berbasis *deep learning*.

BAB II

TINJAUAN PUSTAKA

2.1 *State of the Art*

Kajian *state of the art* pada subbab ini disusun untuk memetakan posisi penelitian yang dilakukan terhadap penelitian-penelitian terdahulu yang relevan. Penelusuran literatur difokuskan pada empat kelompok kajian yang saling berkaitan.

Kelompok pertama adalah penelitian yang menerapkan *Long Short-Term Memory* dan variannya untuk peramalan penjualan ritel. Kelompok kajian ini menunjukkan kecenderungan yang konsisten, yaitu bahwa arsitektur berbasis LSTM mampu menangkap ketergantungan temporal dan pola musiman pada data penjualan secara lebih baik dibandingkan metode statistik konvensional. Namun demikian, sebagian besar penelitian pada kelompok ini menggunakan *dataset* berskala besar dengan rentang pengamatan bertahun-tahun, sehingga hasilnya belum tentu dapat direplikasi pada skala usaha kecil.

Kelompok kedua adalah penelitian yang mengkaji augmentasi data sintetis, khususnya menggunakan *Time-series Generative Adversarial Network* (TimeGAN), sebagai upaya mengatasi keterbatasan jumlah sampel pelatihan. Kelompok ini memperlihatkan hasil yang belum sepenuhnya konvergen: sebagian penelitian melaporkan penurunan galat yang signifikan, sementara sebagian lain mencatat bahwa manfaat augmentasi menurun seiring bertambahnya ukuran data awal.

Kelompok ketiga adalah penelitian yang menyoroti pentingnya *baseline* sederhana sebagai kontrol metodologis dalam evaluasi model peramalan. Kelompok ini menjadi penting karena beberapa penelitian menunjukkan bahwa metode sederhana masih dapat mengungguli model *deep learning* pada kondisi data tertentu.

Kelompok keempat adalah penelitian yang mengintegrasikan modul peramalan ke dalam aplikasi *Point of Sale*. Kelompok ini masih relatif terbatas jumlahnya, dan sebagian besar penelitian di Indonesia pada kelompok ini menggunakan metode statistik atau *machine learning* konvensional, bukan *deep learning*.

Ringkasan penelitian terdahulu beserta persamaan dan perbedaannya dengan penelitian ini disajikan pada Tabel 2.1.

Tabel 2.1 *State of the Art* Penelitian Terdahulu

No	Penulis / Tahun	Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
1	T. de Castro Moraes, X.-M. Yuan, E. P. Chew (2024) [4]	<i>Hybrid Convolutional Long Short-Term Memory Models for Sales Forecasting in Retail</i>	Hibrida CNN-LSTM bertumpuk (S-CNN-LSTM) dan paralel (P-CNN-LSTM)	MAPE 14,15% (bertumpuk) dan 14,76% (paralel) pada 500 deret waktu ritel selama 5 tahun	Sama-sama menggunakan LSTM untuk peramalan penjualan ritel harian	Menggunakan data berskala besar (5 tahun, 500 deret); penelitian ini hanya 316 titik pengamatan. Tidak menguji augmentasi data maupun integrasi ke aplikasi
2	S. Mansur dkk. (2025) [1]	<i>Sales Forecasting for Retail Stores Using Hybrid Neural Networks and Sales-Affecting Variables</i>	Hibrida CNN-LSTM dengan variabel eksogen (cuaca, hari libur, hari gajian, unjuk rasa)	MAPE 4,16%, mengungguli ANN (19,74%), CNN (13,90%), dan LSTM tunggal (10,39%)	Menggunakan LSTM pada data penjualan harian berskala kecil (6 bulan) dari satu gerai	Keberhasilan bertumpu pada tujuh variabel eksogen yang tidak tersedia pada <i>dataset</i> penelitian ini; penulis mencatat indikasi <i>overfitting</i> . Tidak menguji <i>baseline</i> sederhana

No	Penulis / Tahun	Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
3	M. Mitra dkk. (2024) [3]	<i>Harnessing LSTM Neural Networks and Hyperparameter Optimization for Precise Sales Forecasting in Retail</i>	LSTM dengan optimasi <i>hyperparameter</i> menggunakan Keras Tuner	Peningkatan akurasi signifikan dibanding metode peramalan tradisional pada <i>dataset</i> Walmart	Sama-sama menerapkan LSTM untuk prediksi penjualan harian	Menggunakan <i>dataset</i> hierarkis berskala besar dengan variabel harga, promosi, dan hari besar; tidak mengkaji keterbatasan data maupun augmentasi sintetis
4	P. Yadav, C. R. Barde (2025) [10]	<i>Retail Real-Time Sales Prediction System Using LSTM and XGBoost</i>	LSTM untuk pemodelan deret waktu dikombinasikan dengan XGBoost, terintegrasi dengan sistem POS	Sistem prediksi penjualan waktu nyata yang terhubung dengan POS untuk penyerapan data dinamis	Sama-sama mengintegrasikan model peramalan ke dalam sistem <i>Point of Sale</i>	Menggunakan pendekatan hibrida LSTM-XGBoost berbasis <i>web</i> ; penelitian ini menggunakan LSTM murni pada aplikasi <i>mobile</i> dan menguji augmentasi TimeGAN
5	J. Yoon, D. Jarrett, M. van der Schaar (2019) [6]	<i>Time-series Generative Adversarial Networks</i>	TimeGAN, yaitu GAN dengan pembelajaran swa-penyeliaan berbasis <i>autoencoder</i> untuk deret waktu	Menghasilkan data sintetis yang mempertahankan dinamika temporal data asli secara lebih baik dibanding GAN konvensional	Menggunakan TimeGAN sebagai metode pembangkitan data sintetis	Berfokus pada kualitas pembangkitan data secara umum, bukan pada dampaknya terhadap akurasi peramalan penjualan pada data berskala kecil

No	Penulis / Tahun	Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
6	P. Tang, Z. Li, X. Wang, X. Liu, P. Mou (2025) [7]	<i>Time Series Data Augmentation for Energy Consumption Data Based on Improved TimeGAN</i>	TimeGAN yang ditingkatkan dengan <i>multi-head self-attention</i> , model prediksi hibrida CNN-GRU	Penurunan RMSE dan MAE serta peningkatan R^2 setelah augmentasi data	Sama-sama menerapkan augmentasi TimeGAN untuk mengatasi keterbatasan jumlah data pelatihan	Diterapkan pada domain konsumsi energi manufaktur dengan model CNN-GRU; penelitian ini pada domain penjualan kantin dengan model LSTM
7	A.-A. Semenoglou, E. Spiliotis, V. Assimakopoulos (2023) [8]	<i>Data Augmentation for Univariate Time Series Forecasting with Neural Networks</i>	Perbandingan sistematis beberapa teknik augmentasi data pada peramalan deret waktu univariat	Manfaat augmentasi lebih signifikan pada jaringan yang lebih dalam, namun menurun seiring bertambahnya ukuran data awal	Sama-sama mengkaji efektivitas augmentasi data pada peramalan deret waktu univariat	Bersifat studi perbandingan lintas <i>dataset</i> tanpa membangun aplikasi; tidak menggunakan TimeGAN sebagai metode augmentasi utama
8	S. Makridakis, E. Spiliotis, V. Assimakopoulos (2022) [9]	<i>The M5 Competition: Background, Organization, and Implementation</i>	Kompetisi peramalan berskala besar dengan pembandingan metode statistik, <i>machine learning</i> , dan <i>deep learning</i>	Metode sederhana dan kombinasi statistik tetap menjadi pembandingan yang kompetitif terhadap model kompleks	Menekankan pentingnya <i>baseline</i> sederhana sebagai kontrol metodologis dalam evaluasi peramalan	Berupa kajian kompetisi lintas ribuan deret waktu; penelitian ini menerapkan prinsip tersebut pada satu studi kasus dengan <i>baseline seasonal-naive</i>

No	Penulis / Tahun	Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
9	R. A. Sembiring & Y. Sary (2025), <i>Jurnal Ilmiah Teknik Mesin, Elektro dan Komputer</i> [11]	<i>Analisa dan Implementasi Long Short-Term Memory (LSTM) dalam Kebutuhan Persediaan Barang di PT. Gunung Sari Indonesia</i>	LSTM dengan <i>window size</i> 14, 30, dan 60 hari, dibandingkan dengan <i>Moving Average</i> ; evaluasi MAD, MSE, MAPE	MAPE LSTM 102–106%, sedangkan <i>Moving Average</i> 85–88%; LSTM lebih adaptif pada pola fluktuatif, <i>Moving Average</i> lebih stabil pada pola konsisten	Sama-sama menerapkan LSTM pada data transaksi penjualan Indonesia dan membandingkannya dengan metode sederhana	Pembanding berupa <i>Moving Average</i> , bukan <i>seasonal-naive</i> ; tidak menguji augmentasi data sintetis maupun membangun aplikasi POS
10	U. M. Jannah dkk. (2025), <i>Jurnal Teknologi Informasi dan Multimedia</i> [12]	<i>Analisis Prediksi Penjualan Isi Ulang Air Galon Menggunakan Metode LSTM dan SARIMA</i>	Perbandingan LSTM dengan SARIMA pada data penjualan usaha kecil	Perbandingan akurasi kedua metode pada data penjualan harian usaha berskala kecil	Sama-sama membandingkan LSTM dengan metode pembanding pada konteks usaha kecil di Indonesia	Pembanding berupa SARIMA, bukan <i>baseline</i> heuristik; tidak mengkaji keterbatasan jumlah data maupun integrasi ke aplikasi POS

No	Penulis / Tahun	Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
11	R. Verdiyanto, D. Hartanti, E. Purwanto (2025), <i>Infotek: Jurnal Informatika dan Teknologi</i> [13]	<i>Pengembangan Aplikasi Point of Sales untuk Prediksi Penjualan Harian Usaha Minuman Menggunakan Algoritma Random Forest Regression</i>	<i>Random Forest Regression</i> yang diintegrasikan ke dalam aplikasi <i>Point of Sales</i>	Aplikasi POS dengan fitur prediksi penjualan harian untuk usaha minuman	Sama-sama membangun aplikasi POS dengan fitur prediksi penjualan harian pada konteks usaha kecil Indonesia	Menggunakan <i>Random Forest Regression</i> yang tidak memodelkan ketergantungan temporal secara eksplisit; penelitian ini menggunakan LSTM dan menguji augmentasi TimeGAN

Berdasarkan pemetaan pada Tabel 2.1, dapat diidentifikasi bahwa belum terdapat penelitian yang secara bersamaan melakukan tiga hal berikut: menerapkan LSTM untuk prediksi penjualan harian pada *dataset* berskala kecil dengan karakteristik deret tidak sinambung, menguji kontribusi augmentasi sintetis TimeGAN pada kondisi tersebut, serta mengintegrasikan model hasil pelatihan ke dalam aplikasi *Point of Sale* yang benar-benar digunakan pengguna akhir dan mengevaluasinya terhadap *baseline seasonal-naive*. Ketiga aspek inilah yang menjadi kebaruan penelitian ini.

2.2 Tinjauan Teoritis

2.2.1 *Point of Sale* (POS)

Point of Sale (POS) adalah sistem yang digunakan untuk mencatat, memproses, dan mendokumentasikan transaksi penjualan pada titik terjadinya transaksi. Dalam perkembangannya, aplikasi POS tidak lagi berfungsi semata sebagai mesin kasir digital, melainkan juga sebagai sistem informasi yang mengelola data produk, persediaan, pelanggan, dan pelaporan keuangan. Penerapan aplikasi POS pada usaha mikro, kecil, dan menengah terbukti membantu proses pencatatan transaksi menjadi lebih cepat, akurat, dan terdokumentasi, meskipun tingkat penerimaan penggunaannya masih dipengaruhi oleh persepsi kemudahan dan kemanfaatan sistem [14]. Secara arsitektural, sistem POS umumnya terdiri atas modul autentikasi pengguna, modul manajemen produk, modul transaksi, modul persediaan, dan modul pelaporan [15].

Data transaksi yang dihasilkan aplikasi POS secara berkelanjutan memiliki sifat deret waktu, yakni terurut berdasarkan waktu dan memiliki ketergantungan antarpengamatan [16]. Sifat inilah yang membuka peluang pemanfaatan data POS untuk keperluan peramalan, sehingga sistem tidak hanya bersifat *descriptive* (mencatat apa yang telah terjadi) tetapi juga *predictive* (memperkirakan apa yang akan terjadi), sebagaimana telah diterapkan pada sistem POS ritel yang menyertakan modul prediksi penjualan [10], [13].

2.2.2 Peramalan Deret Waktu (*Time Series Forecasting*)

Deret waktu adalah rangkaian pengamatan suatu variabel yang dicatat secara berurutan dalam interval waktu yang tetap. Peramalan deret waktu bertujuan memperkirakan nilai pengamatan pada periode mendatang berdasarkan pola yang terkandung pada pengamatan historisnya [16]. Secara umum, sebuah deret waktu dapat diuraikan menjadi empat komponen, yaitu tren (*trend*), musiman (*seasonality*), siklus (*cyclical*), dan komponen tak beraturan (*irregular* atau *noise*) [16], [17].

Suatu deret waktu dapat memuat lebih dari satu periode musiman sekaligus, sehingga pemodelannya perlu memperhitungkan setiap periode yang relevan [16]. Pada data penjualan harian kantin institusional, komponen musiman muncul dalam dua tingkatan. Musiman mingguan (periode $m = 7$) tampak dari perbedaan volume transaksi antara hari kerja dan akhir pekan, sedangkan musiman berbasis kalender akademik muncul dari perbedaan antara masa perkuliahan, minggu ujian, dan masa libur antarsemester. Pola musiman berbasis kalender semacam ini juga dilaporkan

berpengaruh nyata terhadap volume penjualan ritel harian [1].

Pendekatan peramalan secara garis besar terbagi menjadi tiga kelompok. *Pertama*, pendekatan statistik klasik seperti *moving average*, *exponential smoothing*, ARIMA, dan SARIMA, yang mengasumsikan hubungan linear antarpengamatan [17]. *Kedua*, pendekatan *machine learning* seperti *Random Forest*, *Support Vector Regression*, dan *Gradient Boosting*, yang memperlakukan peramalan sebagai persoalan regresi dengan fitur temporal yang direkayasa secara manual [13]. *Ketiga*, pendekatan *deep learning* seperti RNN, LSTM, dan GRU, yang mampu mempelajari representasi temporal secara otomatis tanpa rekayasa fitur eksplisit [2], [4]. Perbandingan lintas ketiga kelompok pendekatan tersebut pada kompetisi peramalan berskala besar menunjukkan bahwa tidak ada satu kelompok yang unggul secara mutlak, sehingga pemilihannya bergantung pada karakteristik dan ukuran data yang tersedia [9].

2.2.3 Baseline Seasonal-Naive

Seasonal-naive adalah metode peramalan paling sederhana untuk data bermusiman, yang memperkirakan nilai suatu periode sama dengan nilai pada periode musiman terakhir yang bersesuaian [16]. Secara matematis, prediksi untuk waktu $t + h$ dirumuskan sebagai:

$$\hat{y}_{t+h} = y_{t+h-m\lfloor(h-1)/m\rfloor-m} \quad (2.1)$$

dengan m adalah panjang periode musiman. Untuk data penjualan harian dengan

musiman mingguan, $m = 7$, sehingga prediksi hari Senin depan adalah nilai penjualan hari Senin sebelumnya.

Meskipun sederhana, metode ini memiliki peran metodologis yang penting. Hyndman dan Koehler menegaskan bahwa metode naif berfungsi sebagai penyebut normalisasi dalam pengukuran akurasi peramalan, sehingga akurasi suatu model hanya bermakna apabila dinyatakan secara relatif terhadap metode naif tersebut [18]. Pengalaman kompetisi peramalan berskala besar juga menunjukkan bahwa metode sederhana kerap sulit dikalahkan oleh model kompleks, terutama pada deret waktu yang pendek atau sangat bergejolak [9]. Karena itu, penyertaan *baseline seasonal-naive* dalam penelitian ini berfungsi sebagai kontrol metodologis untuk memastikan bahwa keunggulan model LSTM, apabila ada, benar-benar berasal dari kemampuan pemodelan dan bukan sekadar dari pola musiman yang telah dapat ditangkap oleh heuristik sederhana.

2.2.4 *Recurrent Neural Network (RNN)*

Recurrent Neural Network adalah arsitektur jaringan saraf tiruan yang dirancang untuk memproses data sekuensial dengan cara mempertahankan keadaan tersembunyi (*hidden state*) yang diperbarui pada setiap langkah waktu [2]. Pada langkah waktu t , keadaan tersembunyi dihitung sebagai [2]:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (2.2)$$

dengan x_t sebagai masukan pada waktu t , h_{t-1} sebagai keadaan tersembunyi

sebelumnya, serta W dan b sebagai bobot dan bias yang dipelajari.

Kelemahan utama RNN konvensional adalah masalah *vanishing gradient*, yaitu mengecilnya nilai gradien secara eksponensial ketika dipropagasikan mundur melintasi banyak langkah waktu. Akibatnya, RNN kesulitan mempelajari ketergantungan jangka panjang [2].

2.2.5 Long Short-Term Memory (LSTM)

Long Short-Term Memory diperkenalkan oleh Hochreiter dan Schmidhuber sebagai pengembangan RNN yang mengatasi masalah *vanishing gradient* melalui penambahan sel memori (*cell state*) dan tiga mekanisme gerbang (*gating mechanism*) [2]. Sel memori berfungsi sebagai jalur informasi yang memungkinkan gradien mengalir tanpa mengalami peluruhan berlebih, sedangkan gerbang mengendalikan informasi mana yang dibuang, disimpan, dan dikeluarkan.

a. Gerbang Lupa (*Forget Gate*)

Gerbang lupa menentukan proporsi informasi dari sel memori sebelumnya yang akan dipertahankan [2]:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.3)$$

dengan σ adalah fungsi aktivasi sigmoid yang menghasilkan nilai pada rentang $[0, 1]$. Nilai mendekati 0 berarti informasi dibuang, sedangkan nilai mendekati 1 berarti informasi dipertahankan.

b. Gerbang Masukan (*Input Gate*)

Gerbang masukan menentukan informasi baru yang akan disimpan ke dalam sel memori. Tahap ini terdiri atas dua bagian, yaitu penentuan proporsi pembaruan dan pembentukan kandidat nilai baru [2]:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.4)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.5)$$

c. Pembaruan Sel Memori (*Cell State Update*)

Sel memori diperbarui dengan menggabungkan informasi lama yang dipertahankan dan informasi baru yang diterima [2]:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (2.6)$$

dengan $\odot$ menyatakan perkalian elemen per elemen (*Hadamard product*).

d. Gerbang Keluaran (*Output Gate*)

Gerbang keluaran menentukan bagian sel memori yang akan diteruskan sebagai keadaan tersembunyi [2]:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.7)$$

$$h_t = o_t \odot \tanh(C_t) \quad (2.8)$$

Melalui mekanisme tersebut, LSTM mampu mempelajari pola berulang

berjangka panjang seperti musiman mingguan pada data penjualan. Sejumlah penelitian membuktikan efektivitasnya pada peramalan penjualan ritel, baik dalam bentuk LSTM tunggal dengan optimasi *hyperparameter* [3] maupun dalam bentuk arsitektur hibrida dengan CNN [1], [4].

2.2.6 *Generative Adversarial Network (GAN) dan TimeGAN*

Generative Adversarial Network (GAN) adalah kerangka pemodelan generatif yang melatih dua jaringan secara bersamaan dalam skema permainan minimaks, yaitu *generator G* yang menghasilkan data sintetis dan *discriminator D* yang membedakan data asli dari data sintetis [19]. Fungsi objektifnya dirumuskan sebagai:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (2.9)$$

GAN konvensional dirancang untuk data yang tidak memiliki urutan, sehingga kurang mampu mempertahankan dinamika temporal ketika diterapkan pada deret waktu. Untuk mengatasi hal tersebut, Yoon dkk. mengusulkan *Time-series Generative Adversarial Network (TimeGAN)*, yang memadukan kerangka *adversarial* dengan pembelajaran swa-penyeliaan (*self-supervised learning*) berbasis *autoencoder* [6]. Arsitektur TimeGAN terdiri atas empat komponen [6]:

1. ***Embedding function***, memetakan data asli dari ruang fitur ke ruang laten.
2. ***Recovery function***, merekonstruksi data dari ruang laten kembali ke ruang

fitur.

3. **Generator**, menghasilkan barisan laten sintetis dari vektor derau acak.
4. **Discriminator**, membedakan barisan laten asli dari barisan laten sintetis.

Keempat komponen dilatih menggunakan kombinasi tiga fungsi galat, yaitu *reconstruction loss*, *unsupervised adversarial loss*, dan *supervised loss* [6]. Komponen *supervised loss* inilah yang secara khusus memaksa *generator* mempelajari transisi antarlangkah waktu, sehingga dinamika temporal data asli dapat dipertahankan pada data sintetis yang dihasilkan.

Dalam konteks penelitian ini, TimeGAN digunakan sebagai metode augmentasi data untuk memperbanyak sampel pelatihan pada *dataset* yang berjumlah terbatas. Penerapan serupa telah dilaporkan berhasil menurunkan RMSE dan MAE pada domain konsumsi energi [7]. Namun demikian, perlu dicatat bahwa manfaat augmentasi tidak bersifat universal; kajian perbandingan sistematis menunjukkan bahwa keuntungan augmentasi cenderung menurun seiring bertambahnya ukuran data awal maupun berkurangnya kedalaman arsitektur jaringan [8].

2.2.7 Praproses Data

a. Normalisasi

Normalisasi diperlukan agar seluruh nilai masukan berada pada rentang yang seragam, sehingga mempercepat konvergensi pelatihan dan mencegah dominasi fitur bernilai besar. Metode yang umum digunakan pada LSTM adalah *Min-Max*

Scaling [3], [4]:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2.10)$$

b. Pembentukan Jendela Geser (*Sliding Window*)

Data deret waktu univariat perlu diubah menjadi format pembelajaran terbimbing melalui teknik *sliding window* [8]. Dengan panjang jendela w , setiap sampel pelatihan terdiri atas masukan $[y_{t-w+1}, \dots, y_t]$ dan target y_{t+1} . Jumlah sampel yang terbentuk adalah $N - w$, dengan N sebagai jumlah pengamatan. Konsekuensinya, semakin panjang jendela yang digunakan, semakin sedikit sampel pelatihan yang tersedia, yaitu pertimbangan yang menjadi kritis pada *dataset* berukuran terbatas seperti pada penelitian ini.

c. Pembagian Data

Berbeda dengan data yang tidak berurutan, pembagian data deret waktu tidak boleh dilakukan secara acak karena akan menimbulkan kebocoran informasi masa depan (*data leakage*). Pembagian dilakukan secara kronologis, dengan data terawal sebagai data latih dan data terakhir sebagai data uji [16].

2.2.8 Metrik Evaluasi

Evaluasi akurasi model peramalan dalam penelitian ini menggunakan metrik berikut, yang mengacu pada tinjauan Hyndman dan Koehler mengenai ukuran akurasi peramalan [18], dengan y_i sebagai nilai aktual, $\hat{y}_i$ sebagai nilai prediksi, dan n sebagai jumlah pengamatan.

a. Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.11)$$

b. Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.12)$$

c. Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (2.13)$$

MAPE memudahkan interpretasi karena dinyatakan dalam persentase, namun tidak terdefinisi ketika terdapat nilai aktual sama dengan nol dan bersifat asimetris terhadap galat berlebih maupun kurang [18]. Keterbatasan ini relevan bagi penelitian ini mengingat *dataset* memuat hari-hari tanpa transaksi.

d. Mean Absolute Scaled Error (MASE)

Untuk mengatasi keterbatasan MAPE sekaligus memungkinkan perbandingan langsung terhadap *baseline*, digunakan metrik MASE yang diusulkan Hyndman dan Koehler [18]:

$$\text{MASE} = \frac{\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|}{\frac{1}{N-m} \sum_{t=m+1}^N |y_t - y_{t-m}|} \quad (2.14)$$

dengan penyebut dihitung khusus dari data latih, yaitu N menyatakan jumlah

pengamatan pada data latih dan m panjang periode musiman, agar nilai MASE sebanding dengan yang dilaporkan pada literatur. Nilai MASE kurang dari 1 menunjukkan bahwa model berkinerja lebih baik daripada *baseline seasonal-naive*, sedangkan nilai lebih dari 1 menunjukkan sebaliknya. Metrik ini menjadikan perbandingan terhadap *baseline* bersifat eksplisit dan terukur.

2.2.9 Flutter

Flutter adalah *framework* pengembangan aplikasi lintas platform yang dikembangkan Google, menggunakan bahasa pemrograman Dart dan menerapkan pendekatan berbasis *widget* dengan mesin perenderan tersendiri [20]. Pemilihan Flutter dalam penelitian ini didasarkan pada kemampuannya menghasilkan aplikasi dengan basis kode tunggal, performa mendekati aplikasi asli (*native*), serta dukungan ekosistem pustaka yang memadai untuk integrasi dengan layanan basis data dan antarmuka pemrograman aplikasi (API).

2.2.10 Pengujian *Black Box*

Pengujian *black box*, atau disebut juga *behavioral testing*, adalah metode pengujian yang berfokus pada kesesuaian antara masukan dan keluaran sistem berdasarkan spesifikasi fungsional, tanpa memperhatikan struktur kode maupun logika internal program [21]. Pengujian ini diarahkan untuk menemukan kesalahan berupa fungsi yang tidak benar atau tidak tersedia, kesalahan antarmuka, kesalahan akses basis data, serta kesalahan perilaku dan kinerja sistem [21], dengan teknik perancangan kasus uji yang lazim digunakan meliputi *equivalence partitioning* dan

boundary value analysis [22], [23]. Dalam penelitian ini, pengujian *black box* diterapkan untuk memverifikasi fungsionalitas aplikasi *Point of Sale* yang dibangun, meliputi modul autentikasi pengguna, manajemen produk, transaksi penjualan, pengelolaan persediaan, pelaporan, serta modul prediksi penjualan harian, dengan setiap kasus uji mencantumkan skenario pengujian, masukan, hasil yang diharapkan, hasil yang diperoleh, dan simpulan berupa “berhasil” atau “gagal”. Metode ini dipilih karena tujuan pengujian adalah memastikan seluruh fungsi aplikasi berperilaku sesuai spesifikasi kebutuhan dari sudut pandang pengguna akhir, sehingga penilaian akurasi model prediksi tidak termasuk dalam cakupan pengujian ini melainkan dilakukan secara terpisah menggunakan metrik yang telah diuraikan pada Subbab 2.2.8.

BAB III

METODOLOGI PENELITIAN

3.1 Data dan Pengumpulan Data

Penelitian ini menggunakan data sekunder yang bersumber dari basis data transaksi platform katalog menu digital **EatsTEDI** pada Departemen Teknik Elektro dan Informatika (DTEDI), Universitas Gadjah Mada. Data diperoleh dalam bentuk *dump* basis data relasional berformat SQL (`eatstedi-20260621-010820.sql`) berukuran 32,15 MB dengan total sekitar 395.090 baris, mencakup rentang transaksi 24 Agustus 2024 sampai dengan 20 Juni 2026 atau 666 hari kalender secara inklusif. Data tersebut merupakan riwayat transaksi riil yang telah terhimpun jauh sebelum penelitian ini dimulai, bukan data yang dihasilkan oleh aplikasi yang dibangun dalam penelitian ini.

Penggunaan data EatsTEDI dipilih karena merepresentasikan karakteristik usaha kuliner berskala kecil pada lingkungan institusional yang menjadi fokus penelitian. Rentang 22 bulan tersebut mencakup empat semester berturut-turut, yakni semester gasal dan genap tahun akademik 2024/2025 serta 2025/2026, sehingga pola musiman berbasis kalender akademik teramati secara berulang. Di sisi lain, jumlah pengamatannya tetap tergolong terbatas, sehingga sesuai untuk menguji kelayakan penerapan model *Long Short-Term Memory* pada kondisi data yang tidak melimpah.

Data tidak digunakan secara langsung dalam bentuk mentah, melainkan melalui tahap penyaringan dan agregasi terlebih dahulu. Transaksi yang diambil

dibatasi pada nota berstatus lunas ($is_paid = 1$) dari tabel *invoices*, kemudian diagregasi menurut tanggal untuk membentuk deret waktu pendapatan harian. Setelah penyaringan, hari aktif pertama dengan transaksi tercatat adalah 26 Agustus 2024, sehingga rentang efektif yang digunakan adalah 26 Agustus 2024 sampai dengan 20 Juni 2026 dan menghasilkan 316 hari dengan transaksi tercatat. Sesuai batasan penelitian, pemodelan dilakukan secara univariat dengan total nilai penjualan harian sebagai satu-satunya variabel target, tanpa menyertakan variabel eksogen. Adapun struktur data yang digunakan dalam penelitian ini ditunjukkan pada Tabel 3.1.

Tabel 3.1 Struktur Data *Dataset* Final Penelitian

No	Nama Data	Sumber	Keterangan
1	<i>invoice_id</i>	Basis data EatsTEDI	Identitas unik setiap nota transaksi
2	<i>created_at</i>	Basis data EatsTEDI	Waktu pencatatan transaksi pada sistem
3	<i>is_paid</i>	Basis data EatsTEDI	Penanda status pelunasan, digunakan untuk menyaring transaksi sah
4	<i>total</i>	Basis data EatsTEDI	Nilai nominal penjualan pada setiap nota transaksi
5	<i>date</i>	Hasil praproses	Tanggal transaksi (format YYYY-MM-DD) hasil normalisasi <i>created_at</i> , berfungsi sebagai penanda urutan deret waktu
6	<i>revenue</i>	Hasil praproses	Total nilai penjualan harian dalam Rupiah hasil agregasi seluruh nota lunas pada tanggal yang sama; menjadi variabel target pemodelan

Berdasarkan Tabel 3.1, kolom *invoice_id*, *created_at*, *is_paid*, dan *total*

merupakan atribut asli yang tersedia pada basis data EatsTEDI dan berperan sebagai bahan mentah penyaringan serta agregasi. Adapun kolom *date* dan *revenue* bukan merupakan bagian dari basis data asli, melainkan dihasilkan pada tahap praproses. Kolom *revenue* inilah yang menjadi satu-satunya variabel yang dimodelkan, sehingga *dataset* final penelitian berbentuk deret waktu univariat dengan 316 titik pengamatan. Pembagian data latih dan data uji dilakukan secara kronologis, yaitu data terawal sebagai data latih dan data terakhir sebagai data uji, bukan secara acak, guna menghindari kebocoran informasi masa depan (*data leakage*) sebagaimana dijelaskan pada Subbab 2.2.7.

Metode pengumpulan data dalam penelitian ini dilakukan melalui tahapan berikut:

a. Studi Literatur

Studi literatur dilakukan untuk mengkaji penelitian terdahulu yang berkaitan dengan peramalan penjualan ritel berbasis *Long Short-Term Memory*, augmentasi data sintetis menggunakan TimeGAN, serta penggunaan *baseline* sederhana sebagai kontrol metodologis. Hasil studi ini menjadi dasar dalam perancangan metode penelitian dan penetapan metrik evaluasi.

b. Observasi dan Wawancara

Observasi dan wawancara dilakukan terhadap pengelola divisi kewirausahaan mahasiswa platform EatsTEDI di DTEDI Universitas Gadjah Mada. Kegiatan ini bertujuan memahami alur operasional kasir, pencatatan menu harian, pengelolaan stok, serta kendala dalam mengantisipasi sisa

produk, sehingga kebutuhan fitur prediksi pada aplikasi dapat dirumuskan sesuai kondisi lapangan.

c. **Akuisisi Data**

Data transaksi diperoleh langsung dari pengelola platform EatsTEDI dalam bentuk berkas *dump* SQL. Berkas tersebut memuat tabel *invoices* yang menjadi sumber utama nilai transaksi, beserta tabel pendukung *product_sold*, *products*, dan *categories*. Data digunakan sebagai data utama penelitian karena merupakan catatan transaksi riil yang berkesinambungan selama dua tahun akademik penuh.

d. **Praproses Data**

Tahap praproses dilakukan untuk menyesuaikan data dengan kebutuhan pemodelan deret waktu, yang meliputi:

- 1) Penyaringan transaksi berdasarkan status pelunasan;
- 2) Agregasi nilai transaksi menurut tanggal menjadi pendapatan harian;
- 3) Normalisasi nilai menggunakan *Min-Max Scaling*;
- 4) Pembentukan sampel pembelajaran terbimbing melalui teknik *sliding window*.

Hasil dari tahap ini adalah *dataset* final berupa deret waktu univariat yang digunakan sebagai masukan dalam proses pelatihan dan evaluasi model.

3.2 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem merupakan tahap identifikasi terhadap fungsi-fungsi yang harus disediakan oleh sistem serta sumber daya pendukung yang diperlukan agar sistem dapat berjalan sebagaimana mestinya. Analisis ini dibagi menjadi kebutuhan fungsional dan kebutuhan non-fungsional.

3.2.1 Analisis Kebutuhan Fungsional

Kebutuhan fungsional menggambarkan fungsi utama yang harus dimiliki oleh aplikasi *Point of Sale* dengan fitur prediksi penjualan harian.

1. Sistem dapat melakukan registrasi toko dan pendaftaran akun pengguna baru.
2. Sistem dapat melakukan autentikasi pengguna menggunakan email dan kata sandi, serta membedakan hak akses berdasarkan peran Pemilik (*Owner*) dan Kasir.
3. Sistem dapat mengelola data produk dan kategori produk, meliputi penambahan, pengubahan, dan penghapusan data.
4. Sistem dapat mencatat transaksi penjualan melalui alur *checkout* kasir.
5. Sistem dapat mencatat dan menampilkan riwayat mutasi stok produk.
6. Sistem dapat menampilkan dasbor ringkasan yang memuat omzet, jumlah transaksi, dan status persediaan produk.
7. Sistem dapat menampilkan hasil prediksi penjualan harian yang dihasilkan model *Long Short-Term Memory* melalui layanan *REST API*.
8. Sistem dapat menyimpan dan menampilkan riwayat analisis prediksi yang

telah dijalankan.

9. Sistem dapat menyinkronkan data transaksi pada perangkat dengan basis data *cloud*.
10. Sistem dapat mengirimkan notifikasi kepada staf toko.
11. Sistem dapat mengelola data staf karyawan dan pengaturan profil toko.
12. Sistem dapat menghitung performa model prediksi menggunakan metrik evaluasi MAE, RMSE, MAPE, dan MASE.

3.2.2 Analisis Kebutuhan Non-Fungsional

Kebutuhan non-fungsional merupakan kebutuhan pendukung yang berkaitan dengan spesifikasi perangkat keras (*hardware*) dan perangkat lunak (*software*) yang digunakan agar sistem dapat berjalan dengan optimal. Kebutuhan ini tidak berhubungan langsung dengan fungsi utama sistem, namun berperan penting dalam mendukung performa, stabilitas, dan keandalan sistem.

a. Perangkat Keras (*Hardware*)

Perangkat keras yang digunakan dalam penelitian ini dapat dilihat pada Tabel 3.2 berikut.

Tabel 3.2 Spesifikasi Perangkat Keras

No	Nama	Spesifikasi	Keterangan
1	Laptop	Acer Nitro 5, Intel Core i5 Gen 12, RAM 16 GB, SSD 512 GB, NVIDIA GeForce RTX 3050	Perangkat pengembangan aplikasi dan pelatihan model prediksi
2	<i>Smartphone</i>	Android 11	Perangkat pengujian aplikasi <i>mobile</i>

b. Perangkat Lunak (*Software*)

Perangkat lunak yang digunakan dalam pengembangan dan pengujian sistem ini dapat dilihat pada Tabel 3.3 berikut.

Tabel 3.3 Spesifikasi Perangkat Lunak

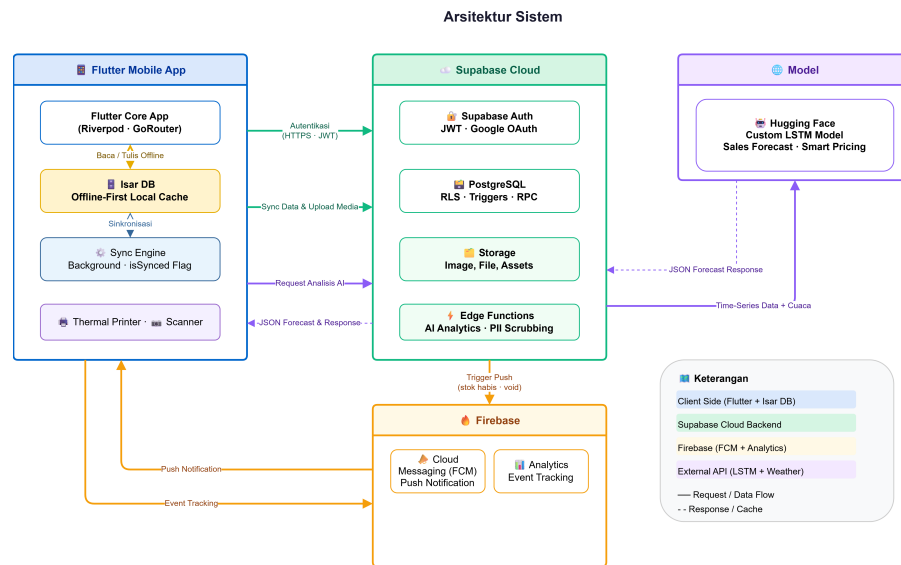
No	Nama	Spesifikasi	Keterangan
1	Sistem Operasi	Windows 11 <i>Home Single Language</i>	Sistem operasi perangkat pengembangan
2	<i>Code Editor</i>	VS Code dan Android Studio	Lingkungan penulisan kode program
3	<i>Framework Mobile</i>	Flutter SDK (bahasa Dart)	Membangun aplikasi <i>Point of Sale</i> lintas platform
4	Bahasa Pemrograman	Python 3.10	Melatih model LSTM, membangun modul augmentasi TimeGAN, dan menyusun layanan <i>REST API</i> prediksi
5	Pustaka <i>Machine Learning</i>	TensorFlow, Keras, PyTorch, Pandas, NumPy, Scikit-Learn	Pemrosesan data dan pembangunan model prediksi
6	<i>Framework API</i>	Flask	Menyajikan hasil prediksi kepada aplikasi <i>mobile</i>
7	Basis Data <i>Cloud</i>	Supabase (PostgreSQL)	Penyimpanan terpusat data transaksi
8	Basis Data Lokal	Isar Database	Penyimpanan data pada perangkat
9	Layanan Notifikasi	Firebase	Pengiriman notifikasi dan pencatatan telemetri aplikasi
10	Lingkungan Analisis	Jupyter Notebook	Eksplorasi data dan pelatihan model prediksi
11	Platform <i>Hosting</i>	Hugging Face	Tempat layanan API prediksi <i>di-deploy</i>
12	Alat Pemodelan	Draw.io dan Mermaid.js	Pembuatan diagram UML, ERD, dan arsitektur sistem

3.3 Perancangan Sistem

Perancangan sistem merupakan tahap yang bertujuan memberikan gambaran mengenai bagaimana aplikasi *Point of Sale* dengan fitur prediksi penjualan harian akan dibangun dan dioperasikan. Tahap perancangan ini mencakup penyusunan arsitektur sistem, pemodelan menggunakan *Unified Modeling Language* (UML), serta perancangan antarmuka pengguna (*user interface*). Adapun rincian perancangan sistem dalam penelitian ini dijelaskan sebagai berikut.

3.3.1 Arsitektur Sistem

Sistem Parzello POS Mobile dirancang menggunakan pendekatan arsitektur *hybrid-cloud* yang membagi pemrosesan antara perangkat lokal dan infrastruktur *cloud*. Secara keseluruhan, sistem terdiri dari empat komponen utama yang saling terintegrasi, yaitu *Flutter Mobile App*, *Supabase Cloud*, *Firebase*, dan *Layanan Inferensi Model LSTM* (Hugging Face). Rancangan arsitektur sistem diilustrasikan pada Gambar 3.1.



Penjelasan mengenai empat komponen utama dalam arsitektur sistem adalah sebagai berikut:

1. **Sisi Klien (*Client-Side*):** Aplikasi dibangun menggunakan *framework* Flutter dengan manajemen *state* Riverpod dan navigasi GoRouter. Komponen inti dari sisi klien adalah Isar DB, sebuah database NoSQL lokal yang berperan sebagai *cache offline-first*, memungkinkan kasir tetap dapat melakukan transaksi penuh meskipun koneksi internet terputus. *Sync Engine* kemudian bekerja di latar belakang untuk menyinkronkan data lokal ke *cloud* secara otomatis ketika koneksi terdeteksi kembali aktif, menggunakan penanda *isSynced* pada setiap *record* untuk menghindari duplikasi data. Selain itu, sisi klien juga mengelola integrasi perangkat keras berupa printer *thermal* Bluetooth/USB untuk mencetak struk dan kamera untuk pemindaian *barcode* produk.

2. **Infrastruktur Cloud (Supabase):** Sebagai tulang punggung infrastruktur *cloud*, Supabase menyediakan empat layanan utama. Supabase Auth menangani autentikasi pengguna melalui protokol JWT dan Google OAuth. Supabase PostgreSQL menjadi basis data relasional terpusat yang dilengkapi kebijakan *Row Level Security* (RLS) untuk mengisolasi data antar *merchant*, pemicu database (*triggers*), serta prosedur tersimpan (*RPC*) untuk memproses transaksi secara aman. Supabase Storage menyimpan aset media seperti gambar produk dan logo toko. Sedangkan *Edge Functions* digunakan untuk memproses *webhook* basis data yang bersifat sensitif, seperti pemicu pengiriman *push notification* peringatan stok menipis kepada pemilik toko.
3. **Telemetri dan Notifikasi (Firebase):** Untuk kebutuhan telemetri dan notifikasi, sistem terintegrasi dengan Firebase. *Firebase Cloud Messaging* (FCM) bertugas menyampaikan *push notification* secara *real-time* kepada pemilik toko, misalnya peringatan stok habis atau pembatalan transaksi oleh kasir. *Firebase Analytics* digunakan untuk mencatat perilaku penggunaan aplikasi, seperti volume transaksi harian, penggunaan fitur *split bill*, dan inisiasi kalibrasi model AI.
4. **Layanan Inferensi Model LSTM (Hugging Face):** Komponen terakhir adalah layanan inferensi untuk mendukung fitur analitik prediktif. Pada penelitian ini, dikembangkan sebuah model *deep learning* berbasis **Long Short-Term Memory (LSTM)** secara mandiri, yang dirancang khusus untuk memproyeksikan data runtun waktu (*time-series*) penjualan harian

pada usaha berskala UMKM. Model prediksi tersebut kemudian dikemas dalam layanan prediksi penjualan berbasis *REST API* (*framework* Flask) yang di-*deploy* pada platform Hugging Face, sehingga dapat diakses oleh sistem secara efisien tanpa memerlukan infrastruktur *server* komputasi tersendiri. Ketika pengguna mengakses fitur *Smart Analytics*, aplikasi Flutter memanggil *endpoint* layanan prediksi tersebut secara langsung melalui protokol HTTPS (dikonfigurasi pada berkas *env.dart* dan dapat dioverride saat *build* melalui *-dart-define*), dengan *payload* berupa data agregat penjualan tanpa informasi identitas pelanggan. Model memproses data tersebut dan mengembalikan hasil prediksi dalam format JSON, mencakup estimasi omzet harian, prediksi produk terlaris, serta rekomendasi promosi berbasis stok, yang selanjutnya divisualisasikan melalui antarmuka grafik interaktif menggunakan pustaka *fl_chart*. Aplikasi juga menangani kondisi *cold start*, yaitu menampilkan peringatan pemuatan ketika *server* inferensi baru aktif kembali dari kondisi *idle*. Setiap hasil analisis disimpan sebagai *snapshot* pada tabel *smart_analytics_snapshots* di Supabase, sehingga riwayat analitik dapat ditinjau kembali tanpa perlu memanggil ulang model.

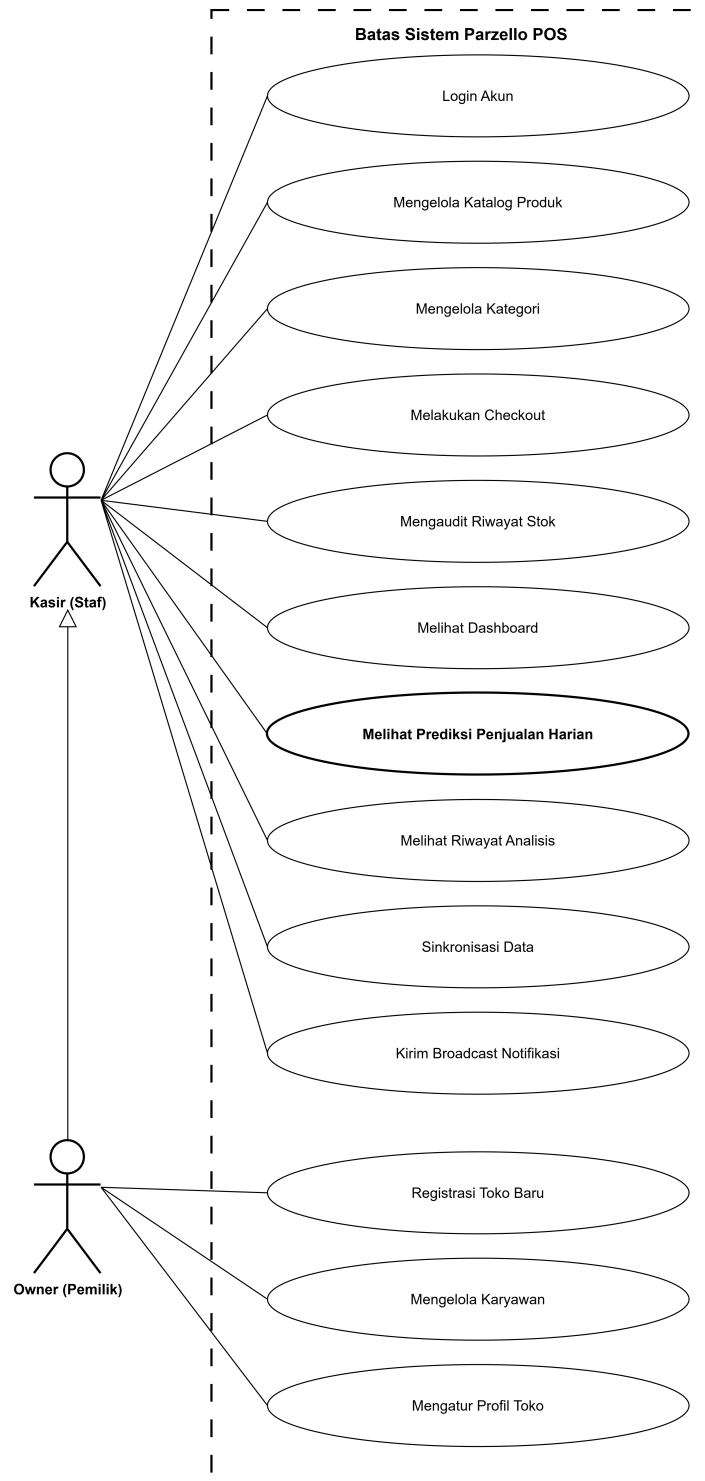
3.3.2 Pemodelan Sistem *Unified Modeling Language* (UML)

Pemodelan sistem merupakan tahapan dalam perancangan yang digunakan untuk menggambarkan interaksi antara pengguna dan sistem serta alur proses yang terjadi di dalam aplikasi *Point of Sale*. Pemodelan dilakukan menggunakan pendekatan *Unified Modeling Language* (UML) agar rancangan sistem dapat

direpresentasikan secara visual dan terstruktur. Pada penelitian ini, pemodelan sistem meliputi *use case diagram*, *activity diagram*, *class diagram*, serta *entity relationship diagram* (ERD).

a. Use Case Diagram

Sistem POS melibatkan dua aktor utama, yaitu **Owner (Pemilik Toko)** dan **Kasir (Karyawan)**. *Use Case Diagram* menggambarkan fungsi-fungsi yang dapat diakses oleh masing-masing aktor. Pembatasan hak akses berbasis peran (*Role-Based Access Control*/RBAC) ditegakkan pada lapisan navigasi aplikasi (*router.dart*), di mana hanya rute Manajemen Karyawan dan Pengaturan Profil Toko yang dibatasi eksklusif untuk Owner, sedangkan fungsi operasional lainnya dapat diakses oleh seluruh staf toko. Hubungan hak akses tersebut digambarkan menggunakan relasi generalisasi aktor, yaitu Owner merupakan spesialisasi dari Kasir sehingga mewarisi seluruh *use case* operasional yang berasosiasi dengan Kasir, ditambah tiga *use case* yang bersifat eksklusif. Adapun ketentuan bahwa pengguna harus terautentikasi lebih dahulu dinyatakan sebagai prasyarat (*precondition*) pada setiap *use case*, bukan sebagai relasi *include*, agar tidak terjadi duplikasi penggambaran. Rancangan *use case* diilustrasikan pada Gambar 3.2.



Gambar 3.2 *Use Case Diagram* Sistem POS

Gambar 3.2 menggambarkan interaksi antara kedua aktor dengan fungsi-fungsi yang tersedia pada sistem. Adapun penjelasan mengenai masing-masing *use case* diuraikan pada bagian berikut.

1) *Use Case* Registrasi Toko Baru

Rancangan *use case* registrasi toko baru digunakan untuk mendaftarkan identitas toko beserta akun pemilik pada saat sistem pertama kali digunakan. Melalui *use case* ini, data toko terbentuk sebagai wadah bagi seluruh data produk, transaksi, dan staf yang dikelola selanjutnya. Rancangan *use case* ini dapat dilihat pada Tabel 3.4.

Tabel 3.4 *Use Case* Registrasi Toko Baru

<i>Use Case</i>	Registrasi Toko Baru
Deskripsi	Proses pendaftaran akun toko baru pada saat inisialisasi awal sistem
<i>Actor</i>	Owner
<i>Precondition</i>	1. Pengguna berada pada halaman registrasi 2. Pengguna mengisi data toko dan data akun pemilik
<i>Postcondition</i>	Akun toko dan akun Owner berhasil terbentuk pada sistem

Tabel 3.4 menunjukkan bahwa sistem menyediakan mekanisme pendaftaran toko baru sehingga aplikasi dapat langsung digunakan tanpa konfigurasi manual pada basis data.

2) *Use Case* Login Akun

Rancangan *use case login* akun digunakan untuk memverifikasi identitas pengguna sebelum mengakses fungsi-fungsi sistem. Proses autentikasi sekaligus menentukan hak akses yang diberikan sesuai peran pengguna. Rancangan *use case*

ini dapat dilihat pada Tabel 3.5.

Tabel 3.5 *Use Case Login Akun*

<i>Use Case</i>	Login Akun
Deskripsi	Proses autentikasi pengguna menggunakan email dan kata sandi
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah memiliki akun terdaftar 2. Pengguna memasukkan email dan kata sandi pada halaman <i>login</i>
<i>Postcondition</i>	Pengguna masuk ke dasbor sesuai hak akses perannya

Tabel 3.5 menunjukkan bahwa sistem membedakan hak akses Owner dan Kasir sejak tahap autentikasi.

3) *Use Case* Mengelola Katalog Produk

Rancangan *use case* mengelola katalog produk digunakan untuk memelihara data produk yang dijual beserta jumlah persediaannya. Rancangan *use case* ini dapat dilihat pada Tabel 3.6.

Tabel 3.6 *Use Case* Mengelola Katalog Produk

<i>Use Case</i>	Mengelola Katalog Produk
Deskripsi	Proses penambahan, penyuntingan, dan penghapusan data produk serta stok
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Pengguna membuka halaman katalog produk
<i>Postcondition</i>	Data produk dan stok tersimpan sesuai perubahan yang dilakukan

Tabel 3.6 menunjukkan bahwa katalog produk dapat dipelihara secara mandiri oleh pengelola toko.

4) *Use Case* Mengelola Kategori

Rancangan *use case* mengelola kategori digunakan untuk mengatur pengelompokan produk agar katalog lebih mudah ditelusuri. Rancangan *use case* ini dapat dilihat pada Tabel 3.7.

Tabel 3.7 *Use Case* Mengelola Kategori

<i>Use Case</i>	Mengelola Kategori
Deskripsi	Proses pengaturan pengelompokan produk pada katalog toko
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Pengguna membuka halaman kategori produk
<i>Postcondition</i>	Data kategori tersimpan dan produk terkelompokkan sesuai kategorinya

Tabel 3.7 menunjukkan bahwa produk dapat dikelompokkan sehingga memudahkan pencarian saat transaksi.

5) *Use Case* Melakukan *Checkout*

Rancangan *use case* melakukan *checkout* merupakan fungsi inti aplikasi kasir yang digunakan untuk memproses transaksi penjualan. Data yang dihasilkan dari *use case* ini menjadi sumber utama bagi pencatatan omzet sekaligus bahan masukan fitur prediksi penjualan. Rancangan *use case* ini dapat dilihat pada Tabel 3.8.

Tabel 3.8 *Use Case* Melakukan *Checkout*

<i>Use Case</i>	Melakukan <i>Checkout</i>
Deskripsi	Proses pemilihan produk ke dalam keranjang hingga penyelesaian pembayaran
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Terdapat produk aktif pada katalog toko
<i>Postcondition</i>	Transaksi tersimpan dan stok produk terbaru secara otomatis

Tabel 3.8 menunjukkan bahwa transaksi penjualan tercatat beserta pembaruan stok produk yang terkait.

6) *Use Case* Mengaudit Riwayat Stok

Rancangan *use case* mengaudit riwayat stok digunakan untuk menelusuri catatan mutasi persediaan sebagai bahan pemeriksaan inventaris. Rancangan *use case* ini dapat dilihat pada Tabel 3.9.

Tabel 3.9 *Use Case* Mengaudit Riwayat Stok

<i>Use Case</i>	Mengaudit Riwayat Stok
Deskripsi	Proses penelusuran <i>log</i> mutasi stok produk
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Terdapat riwayat perubahan stok yang tercatat
<i>Postcondition</i>	Riwayat mutasi stok tersaji beserta waktu dan penyebab perubahannya

Tabel 3.9 menunjukkan bahwa setiap perubahan stok dapat ditelusuri untuk keperluan audit persediaan.

7) *Use Case* Melihat *Dashboard*

Rancangan *use case* melihat *dashboard* digunakan untuk menyajikan

ringkasan performa penjualan toko dalam satu tampilan. Rancangan *use case* ini dapat dilihat pada Tabel 3.10.

Tabel 3.10 *Use Case* Melihat *Dashboard*

<i>Use Case</i>	Melihat <i>Dashboard</i>
Deskripsi	Proses penyajian ringkasan omzet, jumlah transaksi, dan status persediaan
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Terdapat data transaksi yang tercatat
<i>Postcondition</i>	Ringkasan performa penjualan tersaji dalam bentuk kartu dan grafik

Tabel 3.10 menunjukkan bahwa pengelola memperoleh gambaran menyeluruh kondisi toko tanpa menelusuri data satu per satu.

8) *Use Case* Melihat Prediksi Penjualan Harian

Rancangan *use case* melihat prediksi penjualan harian merupakan fungsi yang menjadi inti penelitian ini. Melalui *use case* ini, hasil peramalan yang dihasilkan model *Long Short-Term Memory* disajikan langsung kepada pengelola di dalam alur kerja aplikasi *Point of Sale*. Rancangan *use case* ini dapat dilihat pada Tabel 3.11.

Tabel 3.11 *Use Case* Melihat Prediksi Penjualan Harian

<i>Use Case</i>	Melihat Prediksi Penjualan Harian
Deskripsi	Proses permintaan dan penyajian hasil prediksi penjualan harian dari layanan <i>REST API</i>
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Riwayat transaksi telah memenuhi panjang sekuens minimum yang disyaratkan model
<i>Postcondition</i>	Hasil prediksi penjualan harian tersaji dalam bentuk kartu estimasi dan grafik peramalan

Tabel 3.11 menunjukkan bahwa hasil peramalan dapat langsung dimanfaatkan pengelola sebagai bahan pertimbangan penentuan volume produksi harian.

9) *Use Case* Melihat Riwayat Analisis

Rancangan *use case* melihat riwayat analisis digunakan untuk menelusuri kembali hasil prediksi yang pernah dijalankan beserta sumber komputasinya.

Rancangan *use case* ini dapat dilihat pada Tabel 3.12.

Tabel 3.12 *Use Case* Melihat Riwayat Analisis

<i>Use Case</i>	Melihat Riwayat Analisis
Deskripsi	Proses penyajian daftar analisis prediksi yang telah dijalankan sebelumnya
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Terdapat analisis prediksi yang pernah dijalankan
<i>Postcondition</i>	Riwayat analisis tersaji beserta waktu pelaksanaan dan estimasi yang dihasilkan

Tabel 3.12 menunjukkan bahwa pengelola dapat membandingkan hasil peramalan antarwaktu tanpa menjalankan ulang analisis.

10) *Use Case* Sinkronisasi Data

Rancangan *use case* sinkronisasi data digunakan untuk menyelaraskan data transaksi pada perangkat dengan basis data *cloud*. Rancangan *use case* ini dapat dilihat pada Tabel 3.13.

Tabel 3.13 *Use Case* Sinkronisasi Data

<i>Use Case</i>	Sinkronisasi Data
Deskripsi	Proses penyelarasan data transaksi perangkat dengan basis data <i>cloud</i>
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Terdapat data transaksi yang belum tersinkronkan 2. Perangkat terhubung ke jaringan internet
<i>Postcondition</i>	Data transaksi pada perangkat dan basis data <i>cloud</i> menjadi selaras

Tabel 3.13 menunjukkan bahwa pencatatan transaksi tetap utuh meskipun koneksi sempat terputus.

11) *Use Case* Kirim *Broadcast* Notifikasi

Rancangan *use case* kirim *broadcast* notifikasi digunakan untuk menyampaikan pemberitahuan kepada seluruh staf toko secara serentak. Rancangan *use case* ini dapat dilihat pada Tabel 3.14.

Tabel 3.14 *Use Case* Kirim *Broadcast* Notifikasi

<i>Use Case</i>	Kirim <i>Broadcast</i> Notifikasi
Deskripsi	Proses pengiriman pemberitahuan kepada staf toko
<i>Actor</i>	Owner dan Kasir
<i>Precondition</i>	1. Pengguna telah masuk ke dalam sistem 2. Pengguna menyusun isi pesan yang akan dikirimkan
<i>Postcondition</i>	Notifikasi diterima oleh perangkat staf toko yang dituju

Tabel 3.14 menunjukkan bahwa informasi operasional dapat disebarkan tanpa memerlukan saluran komunikasi terpisah.

12) *Use Case* Mengelola Karyawan

Rancangan *use case* mengelola karyawan digunakan untuk mendaftarkan staf

kasir serta mengatur peran yang diberikan kepada masing-masing staf. Rancangan *use case* ini dapat dilihat pada Tabel 3.15.

Tabel 3.15 *Use Case* Mengelola Karyawan

<i>Use Case</i>	Mengelola Karyawan
Deskripsi	Proses pendaftaran dan pengaturan peran staf toko
<i>Actor</i>	Owner
<i>Precondition</i>	1. Owner telah masuk ke dalam sistem 2. Owner membuka halaman manajemen karyawan
<i>Postcondition</i>	Data staf beserta perannya tersimpan pada sistem

Tabel 3.15 menunjukkan bahwa pengaturan hak akses staf sepenuhnya berada pada kewenangan Owner.

13) *Use Case* Mengatur Profil Toko

Rancangan *use case* mengatur profil toko digunakan untuk memperbarui identitas toko yang ditampilkan pada aplikasi dan bukti transaksi. Rancangan *use case* ini dapat dilihat pada Tabel 3.16.

Tabel 3.16 *Use Case* Mengatur Profil Toko

<i>Use Case</i>	Mengatur Profil Toko
Deskripsi	Proses pengubahan nama, alamat, dan logo toko
<i>Actor</i>	Owner
<i>Precondition</i>	1. Owner telah masuk ke dalam sistem 2. Owner membuka halaman pengaturan profil toko
<i>Postcondition</i>	Identitas toko terbaru pada seluruh tampilan aplikasi

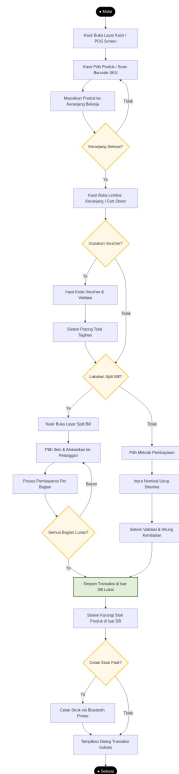
Tabel 3.16 menunjukkan bahwa identitas toko dapat disesuaikan sewaktu-waktu oleh Owner.

b. Activity Diagram

Activity Diagram merupakan gambaran visual dalam bentuk diagram yang digunakan untuk menggambarkan serangkaian aktivitas atau proses yang terjadi di dalam sistem. Diagram ini memudahkan dalam menjelaskan logika prosedural, proses bisnis, serta alur kerja sistem Parzello POS Mobile yang dilengkapi dengan fitur prediksi penjualan harian. Adapun *Activity Diagram* pada sistem ini dijelaskan sebagai berikut.

1. Activity Diagram Alur Kerja Transaksi Kasir (POS Checkout)

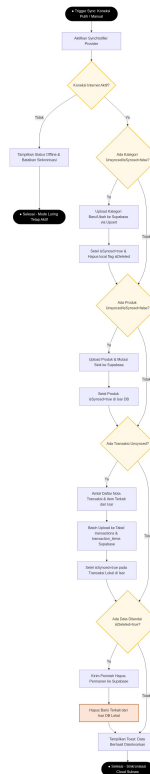
Gambar 3.3 menggambarkan alur kerja kasir atau *owner* dalam memproses transaksi belanja pelanggan di kasir, termasuk opsi penerapan kupon/voucher diskon dan pemisahan tagihan (*split bill*) yang terintegrasi langsung di dalam lembar keranjang belanja dan layar pembayaran, sebelum data disimpan di penyimpanan lokal Isar DB.



Gambar 3.3 Activity Diagram Alur Kerja Transaksi Kasir

2. Activity Diagram Sinkronisasi Latar Belakang (Offline-First Sync Engine)

Gambar 3.4 menggambarkan logika sinkronisasi data dari *local database* (Isar DB) ke cloud database (Supabase) secara otomatis saat status konektivitas terdeteksi *online*. Alur data dilakukan berdasarkan ketergantungan relasi tabel (kategori, produk, transaksi, riwayat stok) untuk menjaga konsistensi, di mana pengunggahan transaksi dilakukan melalui pemanggilan prosedur RPC *create_transaction_v4* pada PostgreSQL agar pencatatan nota dan pemotongan stok di *cloud* berlangsung dalam satu transaksi basis data yang aman.



Gambar 3.4 Activity Diagram Sinkronisasi Latar Belakang

3. Activity Diagram Proses AI Smart Analytics

Gambar 3.5 menggambarkan alur kerja fitur analitik prediktif: pengguna membuka layar *Smart Analytics*, sistem mengagregasi data penjualan historis dari Isar DB lokal, lalu memanggil *endpoint* model LSTM yang di-hosting di Hugging Face secara langsung melalui HTTPS. Selama pemrosesan, sistem menampilkan status pemuatan (termasuk peringatan *cold start* bila *server* inferensi baru aktif dari kondisi *idle*), kemudian merender visualisasi analitik omzet beserta rekomendasi promosi, dan menyimpan hasilnya sebagai *snapshot* riwayat analitik di tabel *smart_analytics_snapshots*.

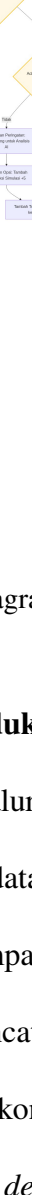
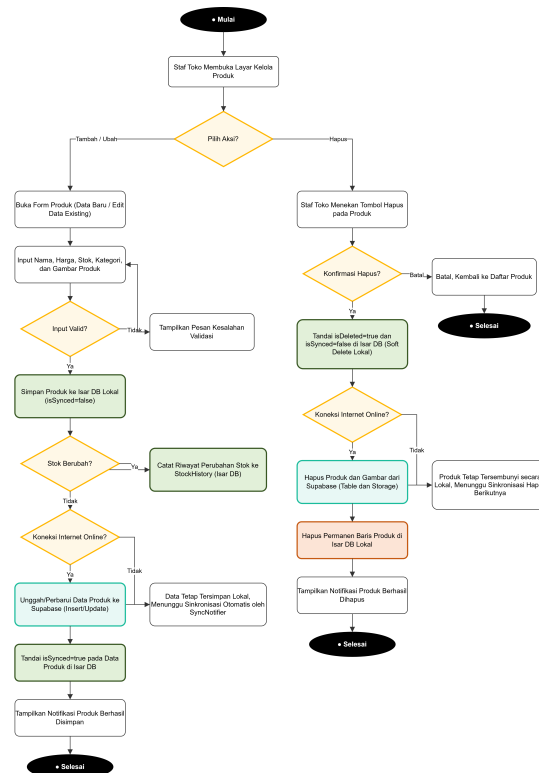


Diagram Kelola Produk (Tambah/Ubah/Hapus)

Gambar 3.6 menggambarkan alur kerja staf toko dalam m

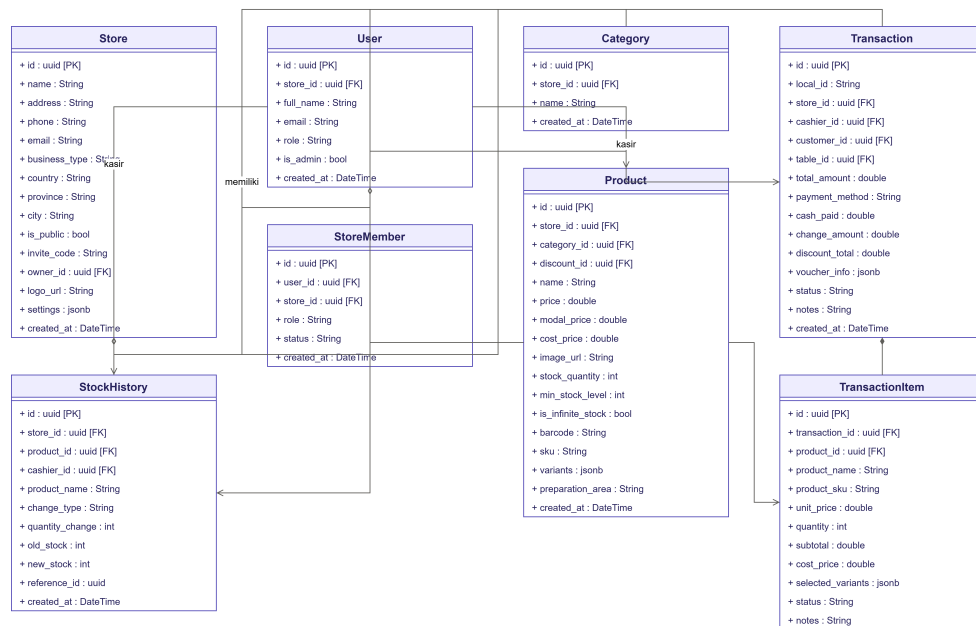
Gambar 3.6 menggambarkan alur kerja staf toko dalam mengelola katalog produk yang menjadi sumber data transaksi bagi model prediksi penjualan. Proses tambah/ubah data disimpan terlebih dahulu ke basis data lokal Isar DB (*offline-first*) beserta pencatatan riwayat perubahan stok, sebelum disinkronkan ke Supabase saat koneksi *online* tersedia. Proses hapus produk diterapkan melalui skema *soft delete* lokal (penanda *isDeleted=true*) yang baru dihapus permanen dari basis data setelah sinkronisasi ke *cloud* berhasil, guna menjaga konsistensi data penjualan yang menjadi masukan pemodelan LSTM.



Gambar 3.6 Activity Diagram Kelola Produk (Tambah/Ubah/Hapus)

c. Class Diagram

Class Diagram menggambarkan struktur sistem dari segi pendefinisian kelas-kelas yang akan dibangun beserta atribut dan metodenya, serta hubungan relasi asosiasi antar-kelas. Pada aplikasi Parzello POS Mobile, rancangan kelas didasarkan pada koleksi basis data lokal Isar DB yang digunakan untuk mendukung penyimpanan *offline-first*. Struktur kelas-kelas ini disajikan pada Gambar 3.7.



Gambar 3.7 Class Diagram Sistem POS

Penjelasan mengenai kelas-kelas utama dalam sistem adalah sebagai berikut:

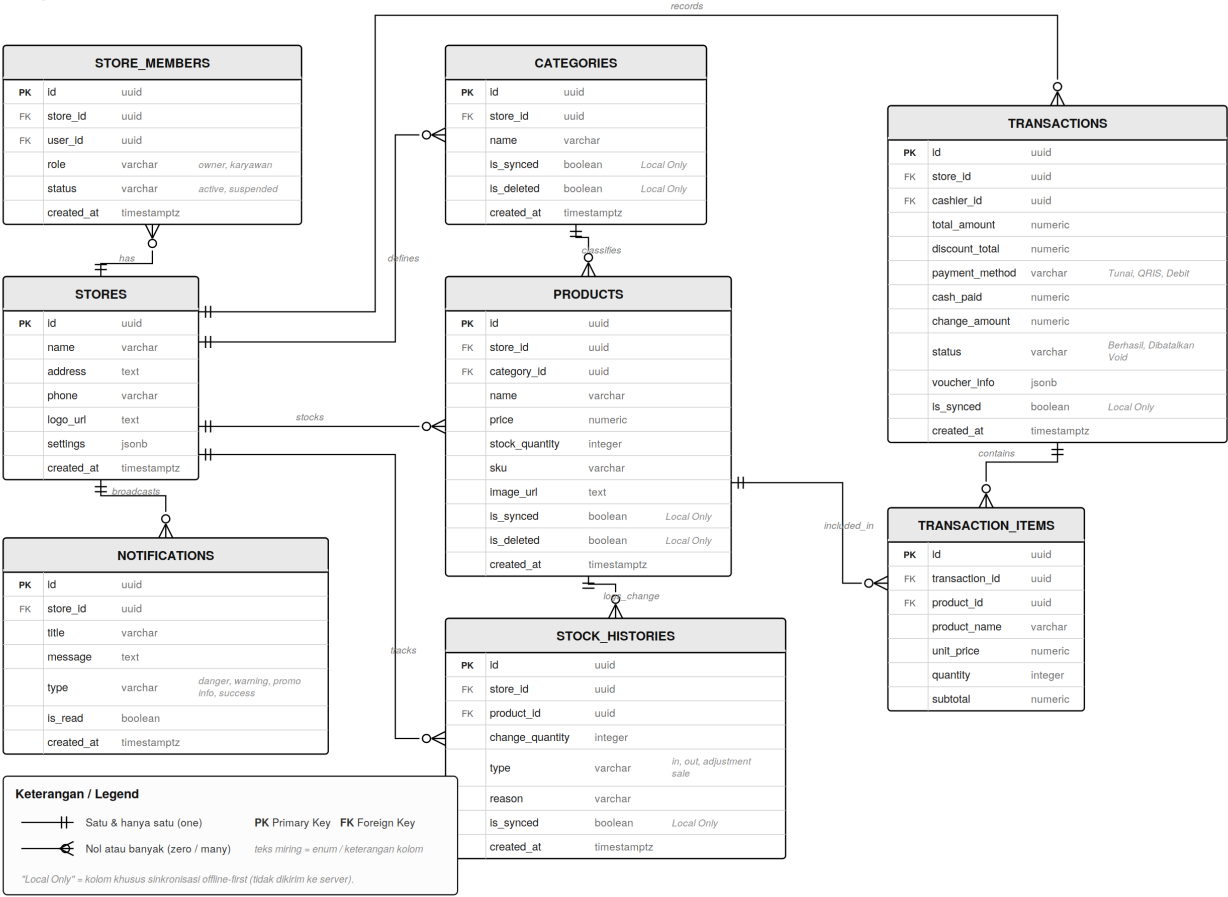
1. **Store:** Merepresentasikan informasi toko/outlet, memiliki relasi satu-ke-banyak dengan Kategori, Produk, dan Transaksi.
2. **Category:** Mengelompokkan produk ke dalam kategori tertentu.
3. **Product:** Menyimpan informasi detail produk termasuk nama, harga jual, harga modal, dan stok.
4. **TransactionLocal:** Menyimpan rangkuman transaksi penjualan kasir di perangkat klien.
5. **TransactionItemLocal:** Menyimpan baris item produk detail dari setiap transaksi belanja.

d. Entity Relationship Diagram (ERD)

Basis data sistem POS dirancang secara hibrida menggunakan Supabase

PostgreSQL untuk penyimpanan *cloud* dan Isar DB untuk penyimpanan lokal di perangkat *mobile*. Skema basis data *cloud* terdiri dari sembilan entitas utama yang saling berelasi sebagaimana diilustrasikan pada Gambar 3.8.

Entity Relationship Diagram — Parzello POS



Gambar 3.8 Entity Relationship Diagram (ERD) Sistem POS

Entitas *stores* menjadi entitas induk yang berelasi dengan seluruh entitas lainnya. Entitas *store_members* mengelola hak akses berbasis peran (*Role-Based Access Control*/RBAC) untuk Owner dan Kasir. Katalog produk dikelola melalui entitas *products* dan *categories*, sedangkan data transaksi disimpan pada entitas *transactions* dan *transaction_items*. Riwayat perubahan stok dicatat pada entitas *stock_history*, sementara entitas *notifications* mendukung sistem notifikasi. Adapun hasil analisis fitur *AI Smart Analytics* disimpan sebagai riwayat *snapshot* pada entitas *smart_analytics_snapshots*. Daftar seluruh tabel basis data yang digunakan dirangkum pada Tabel 3.17.

Tabel 3.17 Daftar Tabel Basis Data POS

No.	Nama Tabel	Deskripsi
1	<i>stores</i>	Menyimpan data identitas dan konfigurasi outlet toko <i>merchant</i> .
2	<i>store_members</i>	Mengelola hak akses dan peran staf (Owner/Kasir) pada setiap toko.
3	<i>categories</i>	Menyimpan kategori pengelompokan produk atau menu dagangan.
4	<i>products</i>	Menyimpan katalog produk beserta harga, stok, dan data SKU/ <i>barcode</i> .
5	<i>transactions</i>	Menyimpan data nota transaksi penjualan kasir sebagai tabel induk.
6	<i>transaction_items</i>	Menyimpan rincian produk yang dibeli dalam setiap transaksi.
7	<i>stock_history</i>	Mencatat <i>log</i> audit perubahan stok masuk dan keluar produk.
8	<i>notifications</i>	Menyimpan pesan notifikasi sistem dan <i>broadcast</i> antar staf toko.
9	<i>smart_analytics_snapshots</i>	Menyimpan riwayat <i>snapshot</i> hasil analitik prediksi LSTM setiap kali fitur <i>Smart Analytics</i> dijalankan.

3.3.3 Perancangan Antarmuka Pengguna (*User Interface*)

Perancangan *wireframe* dalam penelitian ini mencakup rancangan antarmuka pengguna untuk semua layar utama yang menjadi bagian dari alur operasional aplikasi POS. *Wireframe* dirancang dengan mempertimbangkan kebutuhan pengguna, aksesibilitas, dan alur kerja yang efisien. Terdapat 13 rancangan layar yang melayani kebutuhan berbeda sesuai dengan peran pengguna, sebagaimana diuraikan berikut ini.

1. Halaman Registrasi Toko Baru

Halaman pendaftaran akun dan toko baru yang digunakan saat inisialisasi awal sistem oleh Owner, mencakup pengisian data identitas pengguna dan informasi outlet toko.

 ID ▾

Buat Akun

Lengkapi data di bawah ini untuk mendaftar.

Nama Lengkap

 Masukkan nama lengkap

Email

 Masukkan email

Password

 Masukkan password 

Konfirmasi Password

 Ulangi password 

Daftar

ATAU

 Lanjutkan dengan Google

Sudah punya akun? **Masuk**

Gambar 3.9 *Wireframe* Halaman Registrasi Toko Baru

2. Halaman *Login*

Halaman masuk akun untuk Owner maupun Kasir dengan autentikasi berbasis email dan kata sandi melalui Supabase Auth.

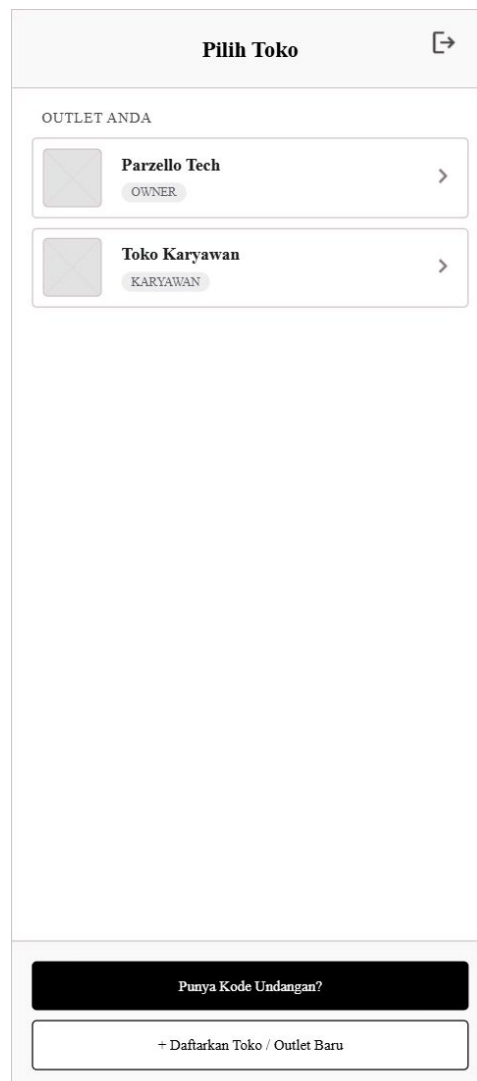
The wireframe shows a login interface with the following elements:

- Title:** Selamat Datang Kembali
- Instruction:** Masukkan email dan password untuk masuk ke dashboard POS.
- Email Field:** A text input field with the placeholder text "Masukkan email Anda".
- Password Field:** A text input field with the placeholder text "Masukkan kata sandi Anda" and a toggle icon (an eye with a slash) on the right.
- Remember Me:** A checkbox labeled "Ingat saya".
- Forgot Password:** A link labeled "Lupa password?".
- Login Button:** A solid black button with the text "Masuk".
- Separator:** A horizontal line with the word "ATAU" in the center.
- Google Login:** A button with a Google icon and the text "Lanjutkan dengan Google".

Gambar 3.10 *Wireframe* Halaman *Login*

3. Halaman *Pemilihan Toko*

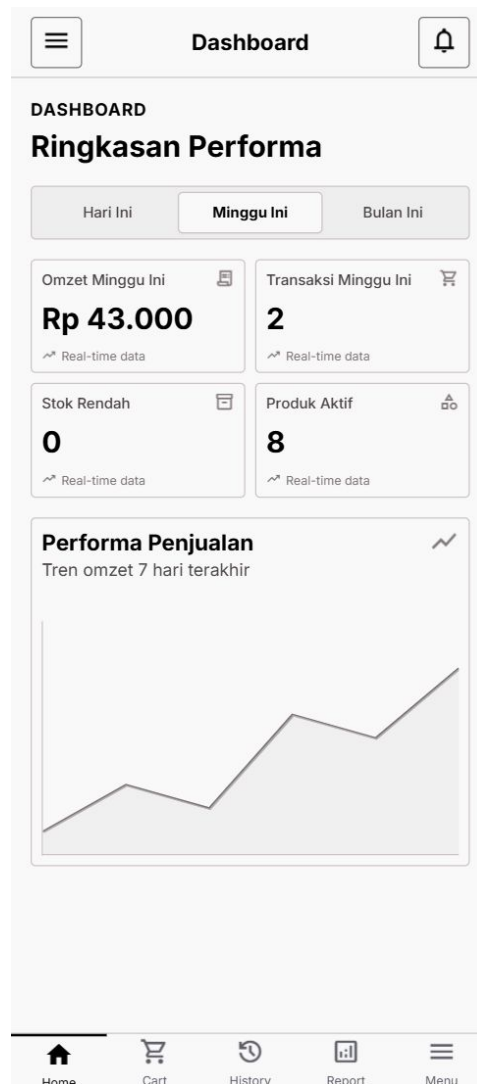
Memungkinkan pengguna yang telah *login* untuk memilih toko aktif yang akan dioperasikan, terutama bagi pengguna yang terdaftar di lebih dari satu outlet.



Gambar 3.11 *Wireframe* Halaman Pemilihan Toko

4. Halaman Dasbor Owner

Ringkasan performa toko yang menampilkan statistik penjualan harian, grafik tren omzet, pintasan fitur penting, dan akses cepat ke modul utama khusus Owner.



Gambar 3.12 Wireframe Halaman Dasbor Owner

5. Halaman POS (Pembayaran)

Layar utama kasir untuk mencari produk, menyusun keranjang pesanan, menerapkan *voucher* diskon, memilih metode pembayaran, dan memproses transaksi.

< **Pembayaran**

TOTAL TAGIHAN
Rp 25.000

Metode Pembayaran


Tunai


QRIS

Uang Tunai Diterima

Rp 25000

Rp 25 rb Rp 50 rb Rp 100 rb

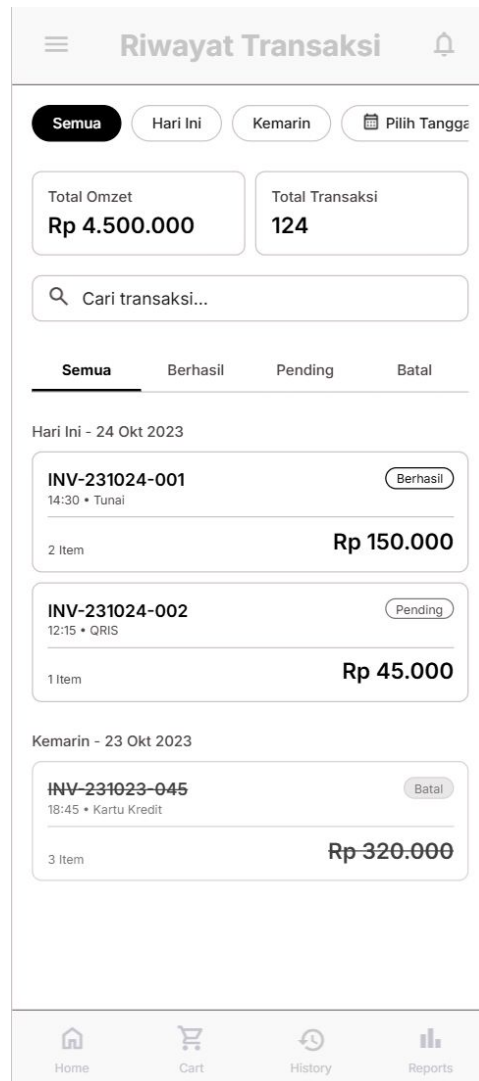
KEMBALIAN
Rp 0 

Konfirmasi & Simpan Transaksi

Gambar 3.13 *Wireframe* Halaman POS (Pembayaran)

6. Halaman Riwayat Transaksi

Menampilkan daftar transaksi yang telah dilakukan lengkap dengan fitur pencarian dan *filter* berdasarkan tanggal, berguna untuk keperluan verifikasi dan audit oleh semua staf toko.



Gambar 3.14 *Wireframe* Halaman Riwayat Transaksi

7. Halaman Kelola Produk

Memungkinkan staf toko untuk menambah, menyunting, dan menghapus produk yang dijual, lengkap dengan harga, stok, SKU/*barcode*, gambar produk, dan kategori.

←

Tambah Produk

Tambah Foto

Maksimal ukuran foto 5MB. Format JPG, PNG.

Nama Produk

Masukkan nama produk

Kategori

Pilih kategori

SKU / Barcode

Contoh: PRD-001

Harga Modal

Rp 0

Harga Jual

Rp 0

Stok Awal

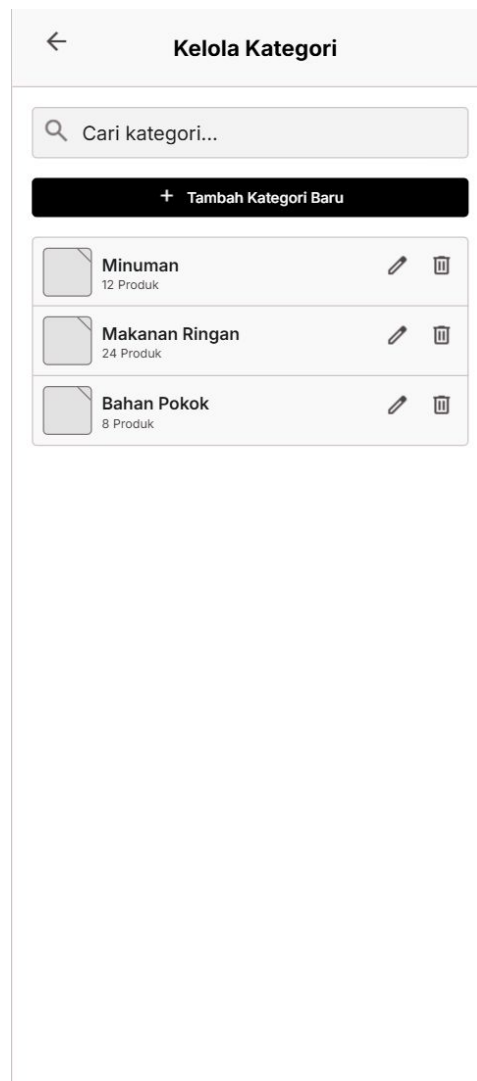
0

Simpan Produk

Gambar 3.15 Wireframe Halaman Kelola Produk

8. Halaman Kelola Kategori

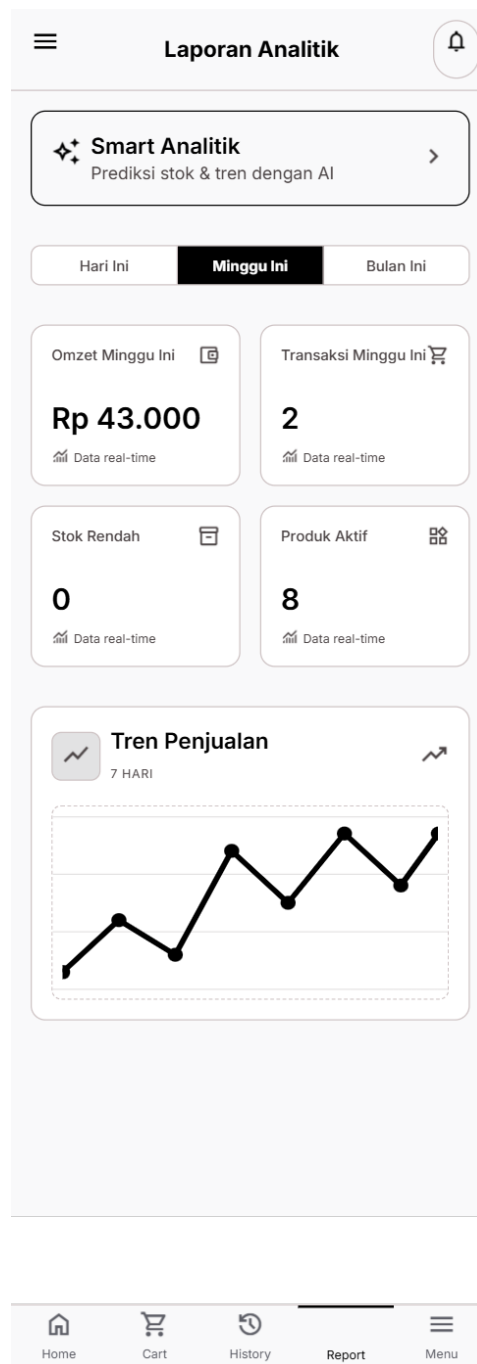
Memungkinkan staf toko untuk membuat, mengedit, dan menghapus kategori produk guna mengorganisasi katalog dagangan toko dengan lebih terstruktur.



Gambar 3.16 *Wireframe* Halaman Kelola Kategori

9. Halaman Laporan

Layar yang dapat diakses seluruh staf toko untuk menampilkan laporan penjualan, tren bisnis, proyeksi penjualan harian berbasis model LSTM, dan ringkasan performa operasional toko.

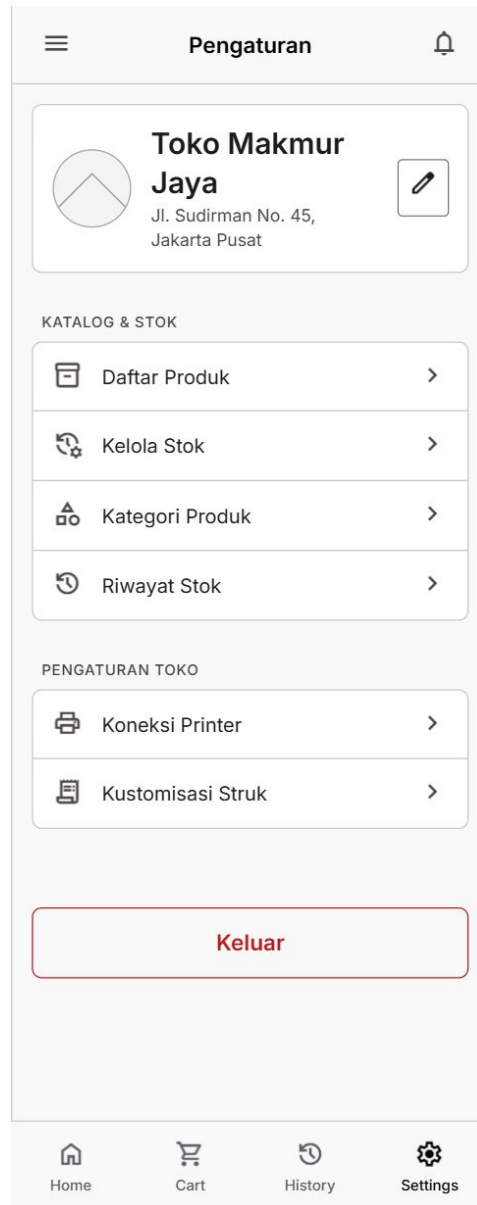


Gambar 3.17 Wireframe Halaman Laporan

10. Halaman Pengaturan

Pusat konfigurasi aplikasi dan toko yang dapat diakses oleh semua staf, dengan sejumlah menu yang hanya tersedia bagi Owner seperti pengaturan

profil toko dan integrasi pembayaran.



Gambar 3.18 *Wireframe* Halaman Pengaturan

11. Halaman Informasi Toko

Memungkinkan Owner untuk mengubah nama toko, alamat, logo outlet, dan informasi operasional lainnya yang ditampilkan kepada pelanggan.

Gambar 3.19 *Wireframe* Halaman Informasi Toko

12. Halaman Struk Digital

Menampilkan pratinjau struk transaksi dalam format digital yang dapat dibagikan kepada pelanggan, mencakup rincian produk, total pembayaran, dan informasi toko.

<
Struk Digital

Parzello Tech
Telp: 085161787501
Terima Kasih Telah Berbelanja

No. Transaksi	#2E17EDFD
Tanggal	10 Jun 2026, 03:00
Metode	Tunai
Burger Daging + Telur 1 x Rp 15.000	Rp 15.000
Burger Sosis + Telur 1 x Rp 10.000	Rp 10.000
Total Belanja	Rp 25.000
Bayar	Rp 25.000
KEMBALIAN	Rp 0

--- SELESAI ---
Powered by Parzello POS

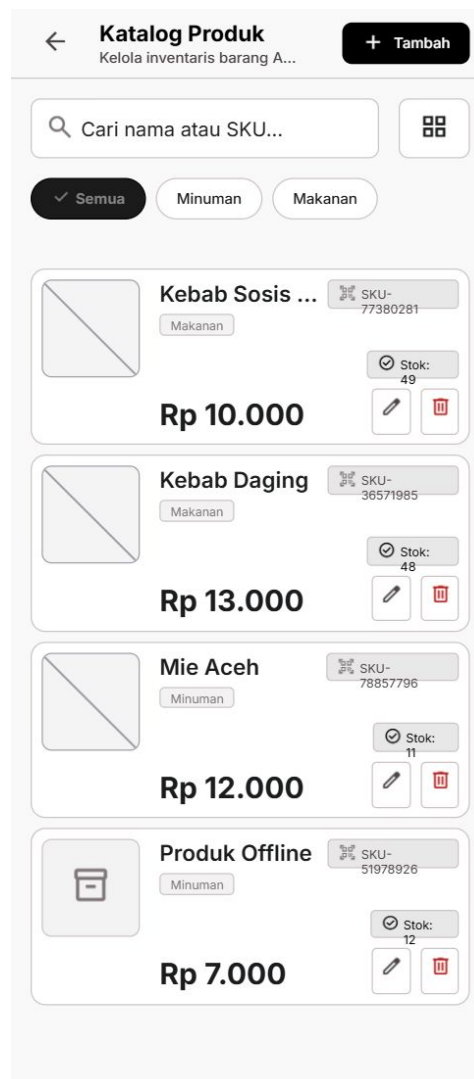
Bagikan

Set Printer

Gambar 3.20 Wireframe Halaman Struk Digital

13. Halaman Katalog Produk

Beranda mode pelanggan yang menampilkan katalog produk toko beserta kategori menu, informasi harga, dan fitur pencarian produk secara mandiri.



Gambar 3.21 Wireframe Halaman Katalog Produk

3.4 Perancangan Metode Penelitian

3.4.1 Metode Penelitian

Penelitian ini merupakan jenis penelitian pengembangan (*Research and Development*) dengan pendekatan kuantitatif untuk pengujian model prediksi dan kualitatif untuk analisis kebutuhan pengguna. Model pengembangan yang digunakan adalah *Agile* (**Tangkas**) dengan pendekatan kombinasi antara *Iterative*

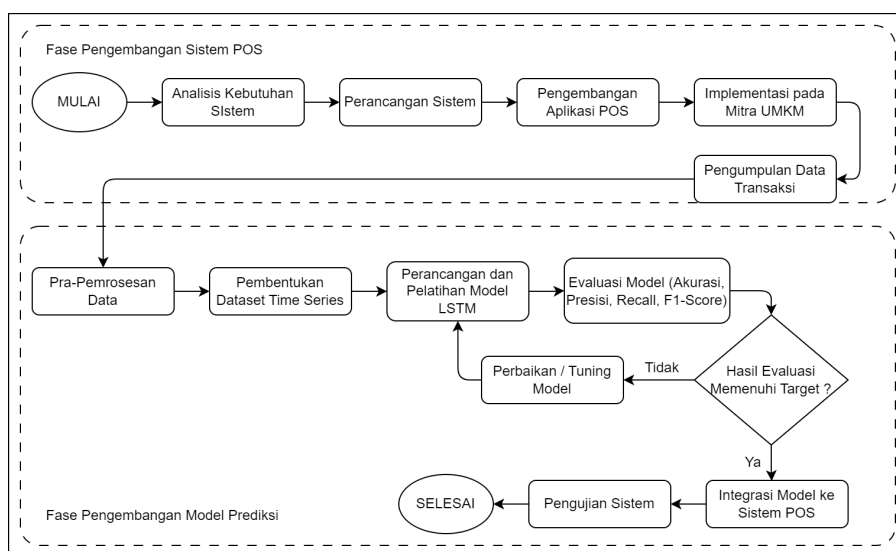
(Berulang) dan *Incremental* (Bertahap), serta mengadopsi prinsip *Lean Software Development*. Metode ini diterapkan melalui fase-fase berikut:

1. **Fokus pada *Minimum Viable Product* (MVP):** Pengembangan difokuskan pada penyelesaian fitur inti sistem kasir terlebih dahulu (*Core POS*, autentikasi pengguna, manajemen produk, dan integrasi *hardware* printer) agar produk minimal dapat segera berfungsi untuk mempercepat waktu rilis ke pasar (*time-to-market*). Fitur lanjutan seperti analisis cerdas (*Smart Analytics*) dan notifikasi *push* direncanakan pada fase pasca-MVP.
2. **Pengembangan Berbasis Sesi/Iterasi Pendek (*sprints*):** Proses pengembangan dipecah menjadi siklus iterasi pendek dalam bentuk sesi-sesi harian terfokus. Setiap sesi ditujukan untuk menambahkan fungsionalitas baru (*increment*) secara bertahap atau melakukan optimasi spesifik, seperti perbaikan antarmuka dasbor, optimasi navigasi *sidebar*, dan penataan ulang modul *database*.
3. **Refaktorisasi Kode dan Adaptasi Perubahan:** Pengembangan bersifat adaptif dan responsif terhadap kendala teknis atau perubahan kebutuhan yang ditemui di tengah proses pengembangan. Siklus mencakup proses refaktorisasi kode mengikuti pembaruan standar *framework* Flutter, penyelarasan skema *database* (*schema alignment*), optimasi sinkronisasi data lokal-*cloud*, serta mitigasi kebocoran memori (*memory leaks*).
4. **Uji Coba Berulang (*Iterative Testing*):** Melakukan pengujian fungsionalitas menggunakan metode *Black Box Testing* secara berkala untuk setiap

penambahan fitur baru untuk memastikan integritas dan stabilitas aplikasi kasir serta modul prediksi penjualan berbasis LSTM tetap terjaga.

3.4.1.1 Alur Penelitian

Alur penelitian ini dirancang secara sistematis untuk membangun sistem *Point of Sale* (POS) berbasis *mobile* yang dilengkapi fitur prediksi penjualan harian menggunakan metode LSTM. Sebagaimana diilustrasikan pada Gambar 3.22, tahapan penelitian dibagi menjadi dua fase utama yang saling berkaitan.



Gambar 3.22 Alur Penelitian

Pada **Fase Pengembangan dan Implementasi Sistem POS**, penelitian diawali dengan analisis kebutuhan melalui observasi langsung dan identifikasi permasalahan pada mitra EatsTEDi. Selanjutnya dilakukan perancangan sistem meliputi arsitektur *offline-first*, desain basis data relasional (Supabase PostgreSQL dan Isar DB), serta perancangan *Use Case Diagram*, *Activity Diagram*, ERD, dan antarmuka pengguna. Setelah tahap perancangan, sistem POS dikembangkan

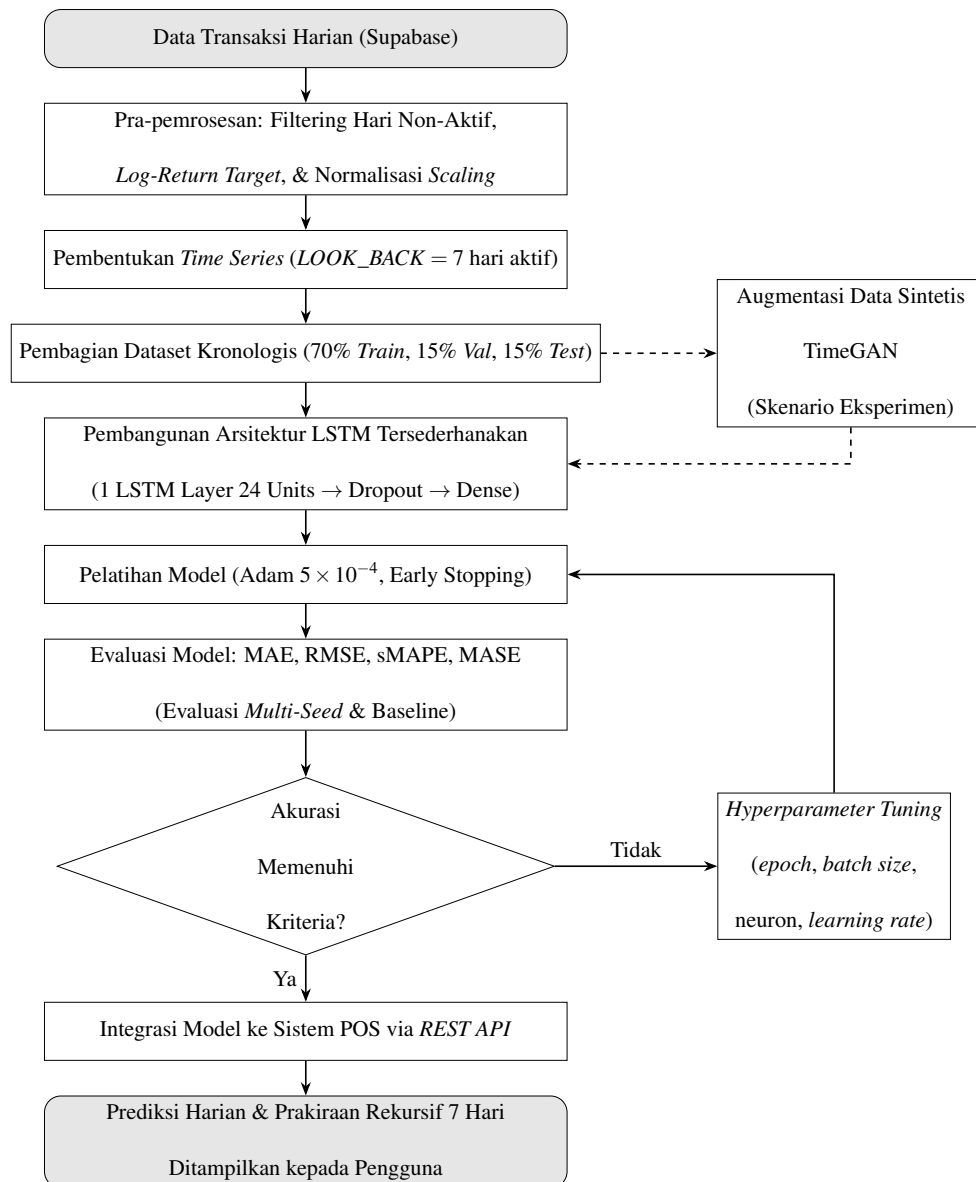
menggunakan Flutter sebagai *frontend*, Supabase sebagai basis data *cloud*, Isar DB sebagai basis data lokal, dan layanan prediksi penjualan Python berbasis Flask yang di-*deploy* pada platform Hugging Face untuk pengolahan modul AI. Aplikasi yang telah dibangun kemudian diimplementasikan dan diujicobakan secara fungsional pada mitra EatsTEDI untuk menguji kesesuaian alur transaksi kasir dan penyajian hasil prediksi kepada pengguna. Perlu ditegaskan bahwa data yang digunakan untuk melatih dan mengevaluasi model prediksi pada penelitian ini **bukan** berasal dari aplikasi Parzello POS yang baru dibangun tersebut, melainkan berupa data sekunder historis yang bersumber dari platform EatsTEDI yang telah beroperasi sebelumnya (*dump SQL eatstedi-20260621-010820.sql*), mencakup rentang waktu 24 Agustus 2024 s.d. 20 Juni 2026. Dengan demikian, aplikasi yang dibangun dalam penelitian ini berperan sebagai wadah penyajian hasil prediksi dan objek pengujian fungsional, sedangkan data latih model prediksi diperoleh dari riwayat transaksi mitra yang sudah ada jauh sebelum penelitian ini dimulai.

Pada **Fase Pengembangan Model Prediksi Penjualan**, data transaksi historis sekunder tersebut selanjutnya melalui tahap pra-pemrosesan meliputi pembersihan data, agregasi harian, normalisasi, dan pembentukan data deret waktu (*time series*). *Dataset* kemudian dibagi menjadi data latih dan data uji. Model LSTM dilatih menggunakan data latih untuk mempelajari pola penjualan jangka pendek maupun jangka panjang. Setelah pelatihan, model dievaluasi menggunakan metrik regresi deret waktu, yaitu MAE, RMSE, sMAPE, dan MASE, kemudian

layanan prediksi produksi diintegrasikan ke dalam sistem POS sehingga aplikasi mampu menampilkan hasil prediksi penjualan harian kepada pengguna melalui *REST API*.

3.4.2 Rancangan Algoritma *Long Short-Term Memory* (LSTM)

Desain metode LSTM memberikan gambaran mengenai alur kerja algoritma yang digunakan dalam proses prediksi penjualan harian pada sistem POS. Alur kerja algoritma diilustrasikan pada Gambar 3.23.



Gambar 3.23 Alur Kerja Algoritma *Long Short-Term Memory* (LSTM)

Gambar 3.23 menggambarkan alur kerja model LSTM dari *input* data transaksi harian hingga proses evaluasi model. Adapun penjelasan rinci mengenai tahapan algoritma LSTM yang dirancang dalam penelitian ini adalah sebagai berikut:

1. **Pra-pemrosesan Data dan Transformasi Target:** Data transaksi harian

ditarik dari database cloud Supabase. Langkah-langkah pra-pemrosesan yang dilakukan meliputi:

- *Filtering hari non-aktif (Zero Days)*: Menghapus hari Sabtu, Minggu, serta bulan Januari dan Juli (libur semester kampus) untuk menghindari distorsi gap yang panjang pada deret waktu. Model dilatih murni menggunakan indeks hari kerja aktif (*business days*).
- *Transformasi Logaritma*: Data nominal pendapatan Y_t ditransformasikan ke skala logaritma menggunakan $Y'_t = \log(Y_t + 1)$ untuk menstabilkan variansi.
- *Transformasi Target (Log-Return)*: Model tidak memprediksi pendapatan logaritmik secara langsung, melainkan memprediksi perubahan pendapatan relatif (*log-return*) r_t menggunakan pendekatan *hybrid residual* sebagaimana dirumuskan pada Subbab 2.4.1, agar model mewarisi sifat autokorelasi jangka pendek yang dominan pada data penjualan.
- *Pengodean Siklik*: Fitur waktu (*day of week, week of year, month*) diubah menjadi nilai sinus dan kosinus untuk merepresentasikan sifat periodik kalender.
- *Normalisasi*: Fitur-fitur masukan dinormalisasi menggunakan *Min-Max Scaling* ke rentang $[0, 1]$, di mana proses pemasangan (*fitting*) parameter *scaler* hanya dilakukan pada data latih (*training set*) untuk mencegah kebocoran informasi (*data leakage*).

2. **Pembentukan Sequence (Sliding Window):** Dataset deret waktu diubah menjadi bentuk urutan berpasangan (*sequence pairs*) menggunakan metode *sliding window*. Ukuran *window* masukan (*look-back*) ditetapkan sebesar 7 hari aktif terakhir ($LOOK_BACK = 7$) untuk memprediksi target 1 hari aktif berikutnya ($HORIZON = 1$). Dengan demikian, model dilatih dengan format prediksi satu langkah ke depan (*one-step-ahead prediction*).

3. **Pembagian Dataset Kronologis:** Dataset dibagi secara kronologis (*time-based split*) untuk menjaga integritas temporal deret waktu, dengan proporsi pembagian sebagai berikut:

- *Data Latih (Training Set)*: Sebesar 70% dari total data aktif (periode Agustus 2024 s.d. Agustus 2025).
- *Data Validasi (Validation Set)*: Sebesar 15% (periode September s.d. Desember 2025) digunakan untuk pemantauan proses latih dan *early stopping*.
- *Data Uji (Testing Set)*: Sebesar 15% (efektif periode Februari s.d. Juni 2026, karena Januari tersaring sebagai libur semester) digunakan untuk pengujian akhir model.

4. **Pembangunan Arsitektur LSTM Tersederhanakan:** Untuk menghindari masalah *overfitting* dan *model collapse* pada dataset yang relatif kecil, arsitektur LSTM dirancang secara minimalis (*shallow network*):

- *Input Layer*: Menerima matriks berukuran 7×8 (panjang *sequence* 7 dan total 8 fitur masukan).

- *LSTM Layer*: Terdiri atas 1 *layer* LSTM dengan 24 *hidden units*.
- *Dropout Layer*: Dengan laju 0,1 untuk mencegah ketergantungan berlebih antar-node.
- *Dense Layer*: Terdiri atas 16 neuron dengan fungsi aktivasi *Rectified Linear Unit* (ReLU).
- *Output Layer*: Terdiri atas 1 neuron (*Dense* dengan aktivasi linear) untuk memprediksi nilai *log-return* $\hat{r}_t$.

Model ini memiliki total parameter terlatih sebanyak 3.585 parameter, yang sebanding dengan ukuran data latih.

5. **Pelatihan Model:** Model dilatih menggunakan algoritma optimasi Adam dengan *learning rate* awal sebesar 5×10^{-4} dan fungsi kerugian (*loss function*) *Mean Squared Error* (MSE). Untuk mengoptimalkan konvergensi, diterapkan dua mekanisme *callback*:

- *Early Stopping*: Menghentikan pelatihan jika loss validasi tidak membaik selama 20 *epoch* berturut-turut untuk mencegah *overfitting*, serta mengembalikan bobot terbaik (*restore best weights*).
- *Reduce Learning Rate on Plateau*: Mengurangi *learning rate* sebesar faktor 0,5 jika loss validasi mendatar selama 8 *epoch*.

Ukuran *batch* pelatihan diatur sebesar 16 dengan jumlah maksimum pelatihan 200 *epoch*.

6. **Evaluasi Komparatif dan Multi-Seed:** Untuk membuktikan keandalan model LSTM, kinerjanya dibandingkan dengan empat model pembanding

(*baseline*):

- *Naive (Kemarin)*: Prediksi hari ini dianggap sama dengan nilai penjualan riil hari aktif sebelumnya.
- *Mean-7*: Prediksi berupa rata-rata bergerak dari 7 hari aktif terakhir.
- *SARIMA*: Model statistik klasik Seasonal Autoregressive Integrated Moving Average untuk mendeteksi pola autokorelasi deret waktu.
- *Seasonal-Naive*: Prediksi berupa rata-rata beberapa kemunculan terakhir dari hari yang sama dalam minggu (Senin dengan Senin, Selasa dengan Selasa, dan seterusnya), untuk menangkap pola musiman mingguan.

Dari keempat *baseline* tersebut, *Naive*, *Mean-7*, dan *SARIMA* dievaluasi pada skema satu langkah ke depan (*one-step-ahead*), sedangkan *Seasonal-Naive* dievaluasi pada skema prediksi multi-langkah dan menjadi kandidat metode penyaji pada layanan API produksi. Notasi parameter *Seasonal-Naive* dilambangkan dengan k , yaitu jumlah kemunculan terakhir hari sejenis yang dirata-ratakan. Selain prediksi harian sebagai fokus evaluasi ilmiah, layanan API produksi juga menyajikan angka proyeksi mingguan dan bulanan menggunakan metode rata-rata sederhana (*Mean-4* untuk empat minggu terakhir dan *Mean-3* untuk tiga bulan terakhir). Keduanya merupakan fitur pelengkap visualisasi aplikasi di luar lingkup evaluasi ilmiah penelitian ini, sejalan dengan Batasan Masalah yang memfokuskan prediksi pada rentang harian.

Mengingat inisialisasi bobot awal jaringan saraf bersifat acak dan dapat memengaruhi stabilitas pada data kecil, evaluasi dilakukan menggunakan metode ***Multi-Seed Evaluation***. Model dilatih sebanyak 10 kali menggunakan *seed* acak yang berbeda. Hasil evaluasi dilaporkan dalam bentuk rata-rata $\pm$ standar deviasi dari metrik MAE, RMSE, SMAPE, dan MASE guna memperoleh estimasi performa yang kokoh dan dapat dipertanggungjawabkan.

7. **Eksperimen Augmentasi Data Sintetis (TimeGAN):** Sebagai upaya mitigasi terhadap keterbatasan jumlah data latih, dirancang skenario eksperimen augmentasi data sintetis menggunakan **TimeGAN** (*Time-series Generative Adversarial Networks*), yaitu arsitektur GAN khusus deret waktu yang diusulkan oleh Yoon et al. (2019). TimeGAN diimplementasikan menggunakan *framework* PyTorch dan terdiri atas lima jaringan saraf (*embedder*, *recovery*, *generator*, *supervisor*, dan *discriminator*) yang dilatih dalam tiga fase: (1) pelatihan *autoencoder* (*embedder*–*recovery*), (2) pelatihan *supervised* pada ruang laten, dan (3) pelatihan gabungan (*joint training*) secara *adversarial*. Model dilatih pada jendela deret waktu sepanjang 24 hari aktif dengan tiga fitur numerik (omzet, jumlah transaksi, dan kuantitas terjual), kemudian menghasilkan 298 sekuens data harian sintetis dari 199 sekuens data riil (rasio augmentasi 1,5). Data sintetis tersebut digabungkan dengan data riil untuk melatih ulang model LSTM (skenario *augmented*), lalu performanya dibandingkan terhadap model

LSTM tanpa augmentasi dan *baseline Seasonal-Naive* guna mengukur seberapa besar augmentasi data mampu memperbaiki akurasi prediksi pada data terbatas.

8. **Rekonstruksi dan Prakiraan Rekursif Multi-Langkah:** Output model berupa *log-return* prediksi ($\hat{r}_t$) direkonstruksi kembali ke skala nominal Rupiah ($\hat{Y}_t$) menggunakan nilai penjualan hari sebelumnya (Y_{t-1}) sesuai formula rekonstruksi pada Subbab 2.4.1. Untuk menyajikan proyeksi 7 hari aktif ke depan pada aplikasi POS, diterapkan metode ***Recursive Forecasting*** (memasukkan hasil prediksi langkah sebelumnya sebagai input untuk langkah berikutnya). Untuk mencegah masalah akumulasi bias (*error propagation*) yang dapat menyebabkan nilai prediksi meluruh atau meledak, diimplementasikan tiga teknik pengaman:

- *Clipping log-return*: Membatasi nilai prediksi *log-return* pada rentang persentil 10–90 dari data pelatihan untuk mencegah nilai ekstrem.
- *Damping factor*: Meredam prediksi perubahan *log-return* pada langkah ke- h menuju nilai 0 dengan faktor peredam ϕ^h (di mana $\phi = 0.7$). Hal ini membuat ramalan jangka panjang stabil dan kembali ke nilai rata-rata historis (sifat *mean-reverting*).
- *Safety rail*: Membatasi hasil akhir nominal pendapatan berada dalam rentang pendapatan terkecil dan terbesar yang pernah tercatat secara historis pada hari aktif.

3.5 Perancangan Pengujian

3.5.1 Teknik Pengujian

Pengujian sistem pada penelitian ini mencakup dua pendekatan utama untuk memastikan kualitas fungsional aplikasi dan keandalan model prediksi yang diintegrasikan. Pendekatan pengujian tersebut dibagi menjadi pengujian evaluasi model prediksi penjualan dan pengujian fungsionalitas sistem secara keseluruhan menggunakan metode *black box testing*.

3.5.1.1 Rancangan Pengujian Evaluasi Model Prediksi

Pengujian model prediksi dilakukan untuk mengukur kinerja dan tingkat akurasi dari algoritma *Long Short-Term Memory* (LSTM) yang dikembangkan. Pengujian ini menggunakan data pengujian (*testing data*) yang diambil dari data histori transaksi penjualan harian mitra UMKM. Evaluasi akurasi model diukur berdasarkan tingkat kesalahan (*error*) prediksi menggunakan empat metrik utama, yaitu MAE, RMSE, sMAPE, dan MASE, yang formula lengkapnya telah dijabarkan pada Subbab 2.6. Sebagai konvensi implementasi metrik sMAPE, apabila penyebut bernilai nol (nilai aktual dan prediksi sama-sama nol), penyebut diganti dengan nilai 1 agar tidak terjadi galat pembagian dengan nol (*division by zero*). Konvensi inilah yang diverifikasi pada pengujian *white box*.

Sebagai kriteria keberhasilan model pada diagram alur (Gambar 3.23), ditetapkan kriteria kelayakan yang bersifat **relatif terhadap baseline**, yaitu model dinyatakan layak apabila **MASE model** < **MASE baseline Naive** pada data uji

yang sama. Kriteria relatif ini dipilih karena nilai MASE dinormalisasi terhadap galat naif satu langkah pada data latih (*in-sample*), sehingga nilai $MASE > 1$ pada data uji hanya menandakan bahwa periode uji lebih bergejolak dibandingkan periode latih, bukan berarti model secara otomatis lebih buruk daripada metode naif; perbandingan yang bermakna secara metodologis adalah antar metode pada data uji yang identik. Apabila kriteria tersebut belum terpenuhi, dilakukan iterasi penyetelan konfigurasi (*hyperparameter tuning*) meliputi jumlah neuron, *learning rate*, *batch size*, dan bentuk target prediksi. Pengujian evaluasi ini dilakukan dengan mengukur kinerja model menggunakan pendekatan *multi-seed evaluation* (10 *seeds* acak) untuk memastikan stabilitas performa model LSTM terhadap variasi inisialisasi bobot awal jaringan saraf. Hasil pengujian akhir tersebut selanjutnya dibandingkan dengan model pembanding (*baselines*) untuk menentukan kelayakan model LSTM. Selain itu, diuji pula skenario perbandingan performa model LSTM yang dilatih dengan dan tanpa tambahan data sintetis hasil augmentasi TimeGAN terhadap *baseline Seasonal-Naive*, guna mengukur efektivitas augmentasi data dalam memitigasi keterbatasan ukuran dataset.

3.5.1.2 Rancangan Analisis Pengujian *Black Box*

Pengujian fungsional sistem pada penelitian ini menggunakan metode *black box testing*, yaitu metode pengujian perangkat lunak yang berfokus pada validasi fungsional sistem dengan mengamati kesesuaian antara masukan (*input*) dan keluaran (*output*) yang dihasilkan, tanpa memperhatikan struktur internal kode program. Pengujian diterapkan pada fitur-fitur utama sistem untuk memastikan

aplikasi berjalan sesuai dengan kebutuhan operasional pengguna.

Rancangan skenario pengujian *black box* untuk fitur-fitur utama sistem POS ini dijabarkan pada Tabel 3.18 berikut:

Tabel 3.18 Skenario Pengujian *Black Box*

No.	Fitur / Fungsi	Skenario Pengujian	Hasil yang Diharapkan
1	Registrasi Toko Baru	Mengisi <i>form</i> registrasi dengan email baru, kata sandi, nama toko, alamat, dan nomor HP.	Akun Owner berhasil dibuat, data toko tersimpan di <i>database</i> Supabase <i>cloud</i> , dan beralih ke halaman utama.
2	Login Akun	Memasukkan email dan kata sandi yang <i>valid/invalid</i> pada layar <i>login</i> .	Jika <i>valid</i> , pengguna masuk ke dasbor sesuai hak akses peran. Jika <i>invalid</i> , sistem menampilkan pesan <i>error</i> autentikasi.
3	Checkout Transaksi	Memasukkan produk ke keranjang, memilih metode bayar, dan mengonfirmasi nominal bayar.	Transaksi berhasil disimpan di <i>database</i> lokal Isar DB, jumlah stok produk berkurang otomatis, dan dialihkan ke struk sukses.
4	Terapkan Diskon	Memasukkan kode <i>voucher</i> /nilai diskon pada keranjang belanja sebelum pembayaran.	Total belanja terpotong sesuai nilai nominal diskon/ <i>voucher</i> dan tercatat pada rincian nota.
5	Mencetak Struk	Menghubungkan printer Bluetooth <i>thermal</i> dan menekan tombol cetak struk dari riwayat transaksi.	Printer <i>thermal</i> mencetak struk fisik transaksi secara lengkap (nama produk, total, tanggal, metode bayar).
6	Kelola Produk	Menambah produk baru dengan nama, harga, stok, foto, dan kategori serta mengubah/menghapus produk.	Data produk baru tersimpan di <i>database</i> lokal Isar DB, terdaftar pada <i>list</i> produk, dan dapat diubah/dihapus.

No.	Fitur / Fungsi	Skenario Pengujian	Hasil yang Diharapkan
7	Kelola Kategori	Menambahkan kategori baru, menyunting nama kategori, atau menghapus kategori produk.	Kategori berhasil dibuat, muncul sebagai opsi <i>filter</i> menu, dan diperbarui secara <i>real-time</i> .
8	Audit Mutasi Stok	Melakukan transaksi/penyesuaian stok, lalu membuka halaman riwayat mutasi stok.	Riwayat keluar-masuk barang tercatat pada <i>log stock_history</i> dengan jenis mutasi yang sesuai.
9	Dasbor Ringkasan	Membuka dasbor laporan penjualan harian, mingguan, dan grafik visualisasi omzet.	Menampilkan statistik ringkasan omzet, total transaksi harian, dan grafik visualisasi garis yang akurat.
10	<i>Smart Analytics</i>	Membuka layar <i>Smart Analytics</i> dengan koneksi internet aktif.	Grafik prediksi penjualan 7 hari ke depan dari layanan API prediksi berhasil ditampilkan dan tersimpan sebagai <i>snapshot</i> .
11	<i>Broadcast Notifikasi</i>	Owner mengirim pesan <i>broadcast</i> kepada staf toko melalui menu notifikasi.	Notifikasi <i>broadcast</i> diterima dan muncul di perangkat kasir secara <i>real-time</i> .
12	Sinkronisasi <i>Offline</i>	Mematikan koneksi internet (luring), membuat transaksi lokal, lalu mengaktifkan internet kembali (daring).	Transaksi disimpan secara lokal di Isar DB (<i>offline-first</i>), lalu terunggah otomatis ke Supabase <i>cloud</i> saat internet aktif kembali.
13	Kelola Staf Karyawan	Owner menambahkan akun staf kasir baru dan mengubah peran staf.	Akun staf berhasil terdaftar pada toko dengan peran yang sesuai. Menu ini tidak dapat diakses oleh Kasir.
14	Atur Profil Toko	Owner mengubah nama, alamat, dan logo toko pada menu pengaturan.	Informasi profil toko berhasil diperbarui dan tampil pada struk serta katalog. Menu ini eksklusif untuk Owner.

3.5.1.3 Rancangan Analisis Pengujian *White Box*

Selain pengujian fungsional *black box*, penelitian ini turut menerapkan pengujian *white box* secara terbatas (*lightweight*) yang berfokus pada verifikasi kebenaran logika internal dan jalur eksekusi kode (*code path*) pada fungsi-fungsi kritis di layanan API prediksi, tanpa memandang antarmuka luarnya. Pengujian ini dilakukan dengan menelusuri (*trace*) jalur logika kode secara manual menggunakan data masukan yang telah diketahui nilai keluarannya, kemudian membandingkan hasil eksekusi aktual terhadap hasil kalkulasi manual sebagai acuan kebenaran (*ground truth*). Tiga fungsi inti yang menjadi target pengujian *white box* adalah:

1. Fungsi perhitungan *Seasonal-Naive (by_dow)*, untuk memverifikasi jalur percabangan logika ketika jumlah data historis per hari kerja mencukupi ($\geq k$), tidak mencukupi ($< k$), maupun tidak tersedia sama sekali (*fallback*).
2. Fungsi metrik evaluasi sMAPE dan MASE, untuk memverifikasi jalur penanganan kasus pembagian dengan nilai penyebut nol atau mendekati nol agar tidak menghasilkan galat pembagian (*division by zero*).
3. Fungsi prediksi rekursif (*forecast_hybrid_stable*), untuk memverifikasi jalur eksekusi tiga mekanisme pengaman secara berurutan, yaitu *clipping*, *log-return*, *damping factor*, dan *safety rail* nominal omzet.

3.5.2 Indikator Keberhasilan Penelitian

Keberhasilan penelitian ini diukur melalui indikator-indikator berikut, yang pencapaiannya dijawab pada Bab IV dan Bab V:

1. Aplikasi *Point of Sale* (POS) *mobile* berfungsi membantu pelaku UMKM dalam mengelola transaksi penjualan, inventaris produk, dan laporan keuangan secara efisien dengan kemampuan operasional *offline-first*, yang divalidasi melalui pengujian fungsional *Black Box*.
2. Metode LSTM terimplementasi pada sistem dan kinerjanya terukur secara objektif menggunakan metrik evaluasi kesalahan prediksi terhadap model pembandingan (*baseline*).
3. Sistem mampu menyajikan informasi prediksi dan laporan penjualan dalam bentuk visualisasi grafik yang mudah dipahami oleh pengguna, khususnya pelaku UMKM.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil rancang bangun dan pembahasan sistem Point of Sale (POS) dengan fitur prediksi penjualan harian menggunakan metode Long Short-Term Memory (LSTM). Pembahasan dibagi menjadi tujuh bagian utama, yaitu hasil implementasi sistem, *preprocessing* data, implementasi dan pelatihan model prediksi, evaluasi model, hasil pengujian sistem, pembahasan hasil penelitian, serta keterbatasan sistem dan penelitian.

Perlu ditegaskan di awal bahwa bab ini memaparkan dua hal yang berbeda namun saling terkait: (1) hasil eksperimen dan evaluasi metode LSTM sebagai objek kajian utama penelitian (Subbab 4.2 s.d. 4.4), dan (2) metode yang benar-benar disajikan oleh layanan API produksi (*endpoint* `/api/predict/daily`) yang dipanggil aplikasi *mobile* (Subbab 4.1.3 dan 4.5). Berdasarkan hasil evaluasi komparatif pada Subbab 4.4, LSTM belum terbukti unggul dibandingkan *baseline* statistik pada volume data penelitian ini, sehingga layanan produksi menerapkan *fallback* berbasis bukti berupa metode *Seasonal-Naive* ($k = 4$), sedangkan model LSTM (`.keras`) tetap didokumentasikan sebagai artefak riset. Arsitektur layanan prediksi bersifat *model-agnostic* sehingga dapat langsung menyajikan model LSTM begitu data mencukupi untuk mengungguli *baseline*. Selain itu, granularitas mingguan dan bulanan yang disebutkan pada Subbab 4.1.3 merupakan fitur pelengkap visualisasi pada antarmuka *Smart Analytics* dan berada di luar lingkup evaluasi ilmiah pada

bab ini, yang berfokus pada granularitas harian sesuai Batasan Masalah.

4.1 Hasil Implementasi Sistem

Bagian ini memaparkan realisasi fisik dari rancangan sistem yang telah disusun pada Bab III, yang mencakup implementasi antarmuka aplikasi mobile (kasir dan owner), implementasi backend dan basis data cloud (Supabase), serta implementasi layanan prediksi (API) di sisi server.

4.1.1 Implementasi Antarmuka (UI)

Aplikasi *mobile* dikembangkan menggunakan *framework* Flutter dengan bahasa pemrograman Dart, mengacu pada rancangan *wireframe* pada Subbab 3.2.8. Cuplikan hasil implementasi antarmuka diambil langsung dari aplikasi yang berjalan pada data toko mitra EatsTEDI, disusun berurutan mengikuti alur penggunaan aplikasi mulai dari registrasi akun, manajemen toko dan produk, transaksi kasir, hingga laporan analitik.

1. Halaman Registrasi

Halaman *Buat Akun* merupakan titik masuk awal bagi pengguna baru. Form registrasi memuat kolom Nama Lengkap, Email, *Password*, dan Konfirmasi *Password*, dilengkapi opsi pendaftaran cepat *Lanjutkan dengan Google* serta tautan menuju halaman Login bagi pengguna yang telah memiliki akun.

03.05

ZA ID

Buat Akun

Mulai kelola bisnis Anda dengan sistem POS modern.

Nama Lengkap

Masukkan nama lengkap Anda

Email

Masukkan email Anda

Password

Masukkan kata sandi Anda

Konfirmasi Password

Konfirmasi kata sandi Anda

Daftar

ATAU

Lanjutkan dengan Google

Sudah punya akun? **Masuk**

Gambar 4.1 Implementasi Halaman Registrasi

2. Halaman *Login*

Halaman *Selamat Datang Kembali* menyediakan autentikasi menggunakan Email dan *Password*, opsi *Ingat saya* dan *Lupa password*, serta tombol masuk melalui akun Google. Tombol *Masuk sebagai Pelanggan* disediakan sebagai jalur akses terpisah bagi pelanggan yang hendak memindai QR katalog mandiri tanpa memerlukan akun staf toko.

03.05

zA ID

Selamat Datang Kembali

Masukkan email dan password untuk masuk ke dashboard POS.

Email

Masukkan email Anda

Password

Masukkan kata sandi Anda

☒ Ingat saya [Lupa password?](#)

Masuk

ATAU

Lanjutkan dengan Google

Masuk sebagai Pelanggan

Belum punya akun? [Daftar](#)

Gambar 4.2 Implementasi Halaman *Login*

3. Halaman Informasi Toko

Setelah registrasi, pengguna diarahkan ke halaman *Informasi Toko* untuk membuat outlet pertamanya, memuat kolom Nama Toko/*Outlet*, Provinsi dan Kota/Kabupaten (*dropdown* berjenjang), serta Alamat Lengkap, yang kemudian dikonfirmasi melalui tombol *Konfirmasi & Mulai*.

04.00

< **Selamat Datang!**

Satu langkah lagi untuk memulai bisnis Anda.
Mari buat toko pertama Anda.

< **Informasi Toko**

NAMA TOKO / OUTLET

Contoh: Kopi Kenangan Jaya

PROVINSI

Pilih Provinsi

KOTA / KABUPATEN

Pilih provinsi dulu

ALAMAT LENGKAP

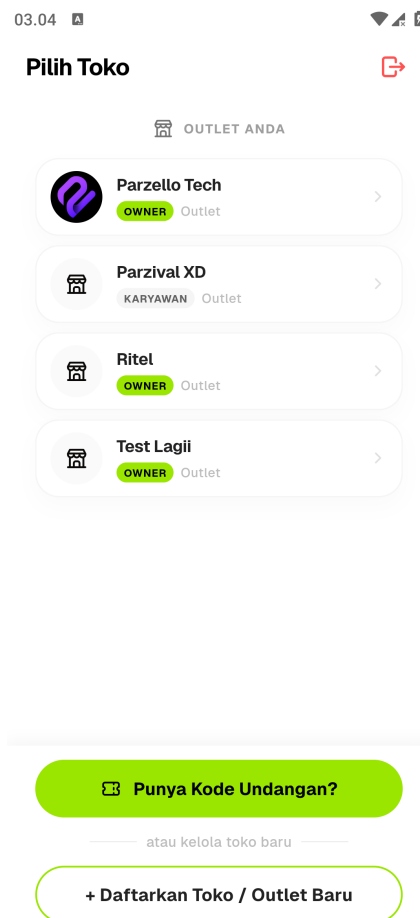
Jl. Merdeka No. 123

Konfirmasi & Mulai

Gambar 4.3 Implementasi Halaman Informasi Toko

4. Halaman Pilih Toko

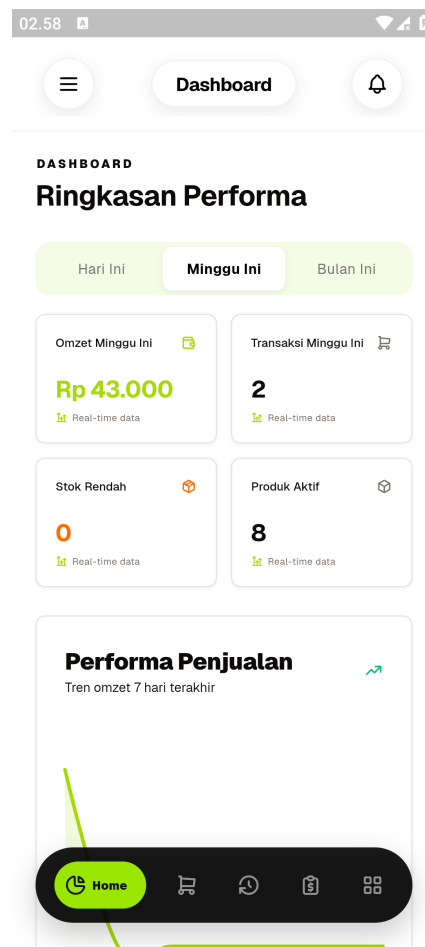
Menampilkan daftar seluruh *outlet* yang terasosiasi dengan akun pengguna beserta peran pada masing-masing *outlet* (Owner atau Karyawan). Pengguna dapat bergabung ke *outlet* lain melalui *Kode Undangan* atau mendaftarkan *outlet* baru sebelum memilih toko aktif yang ingin dioperasikan.



Gambar 4.4 Implementasi Halaman Pilih Toko

5. Halaman Dasbor Owner

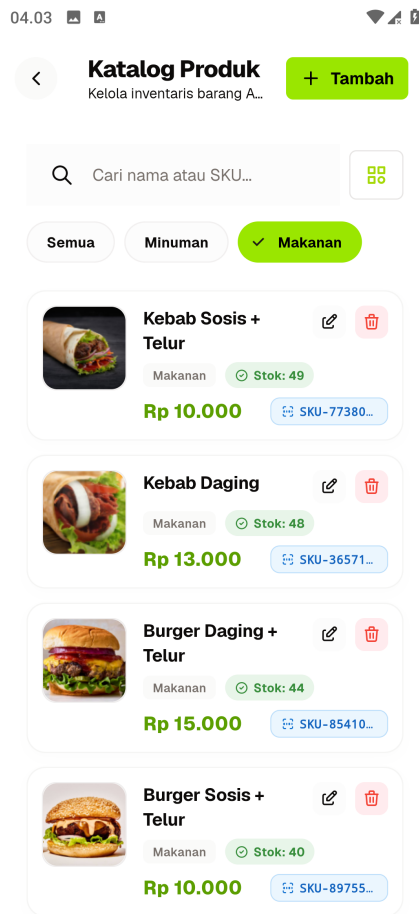
Menyajikan *Ringkasan Performa* toko dengan *filter* rentang waktu (Hari Ini, Minggu Ini, Bulan Ini), empat kartu statistik *real-time* (Omzet, Transaksi, Stok Rendah, dan Produk Aktif), serta grafik *Performa Penjualan* yang memvisualisasikan tren omzet tujuh hari terakhir. Navigasi bawah aplikasi (Dasbor, Kasir, Riwayat, Laporan, dan menu lainnya) juga tampak pada halaman ini.



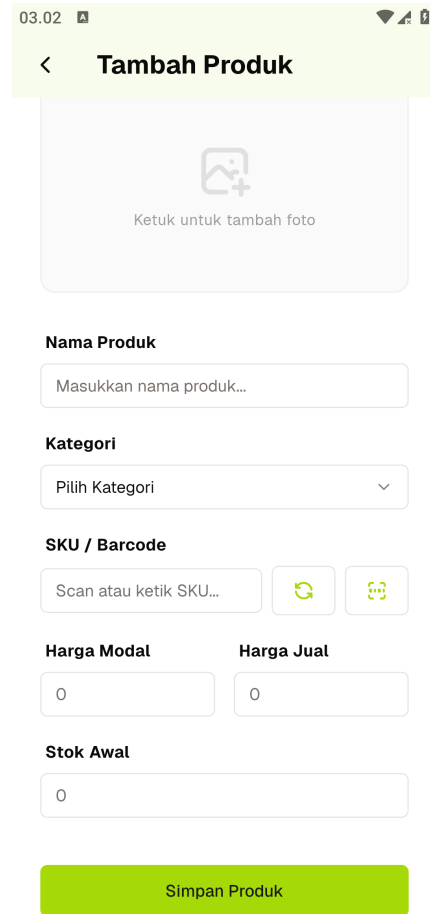
Gambar 4.5 Implementasi Halaman Dasbor Owner

6. Halaman Kelola Produk dan Tambah Produk

Halaman *Katalog Produk* menampilkan daftar inventaris beserta foto, kategori, status stok, harga, dan kode SKU, dilengkapi kolom pencarian dan *filter* kategori (Semua, Minuman, Makanan). Formulir *Tambah Produk* memuat unggah foto produk, nama, kategori, SKU/*Barcode* (dapat dipindai atau digenerasi otomatis), harga modal, harga jual, dan stok awal.



(a) Katalog Produk

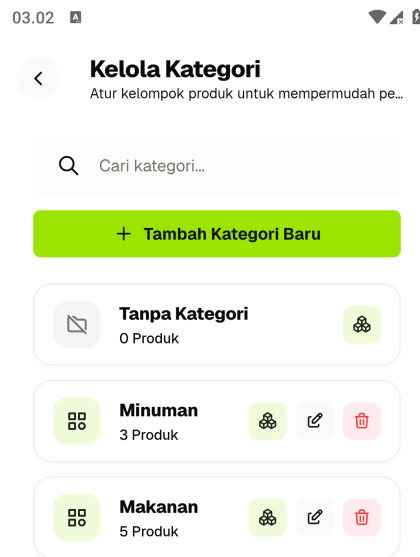


(b) Tambah Produk

Gambar 4.6 Implementasi Halaman Kelola Produk dan Tambah Produk

7. Halaman Kelola Kategori

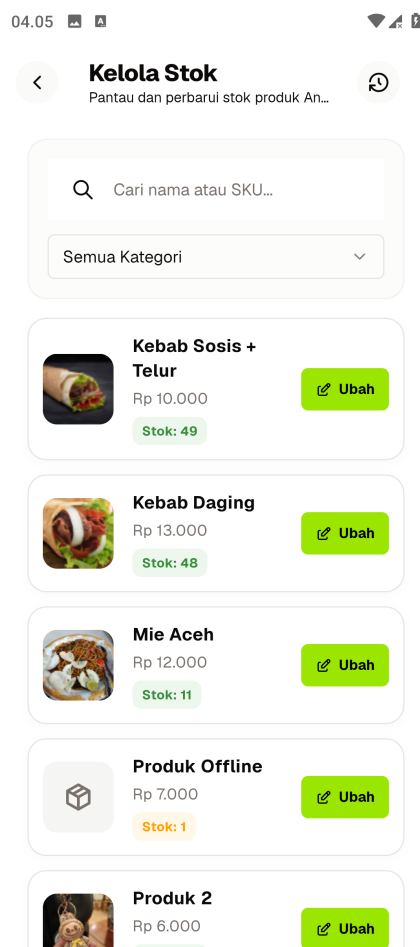
Menyediakan pengelompokan produk melalui kategori yang dapat ditambah, disunting, dan dihapus, dilengkapi pencarian kategori dan penghitung jumlah produk pada setiap kategori (misalnya Minuman dan Makanan).



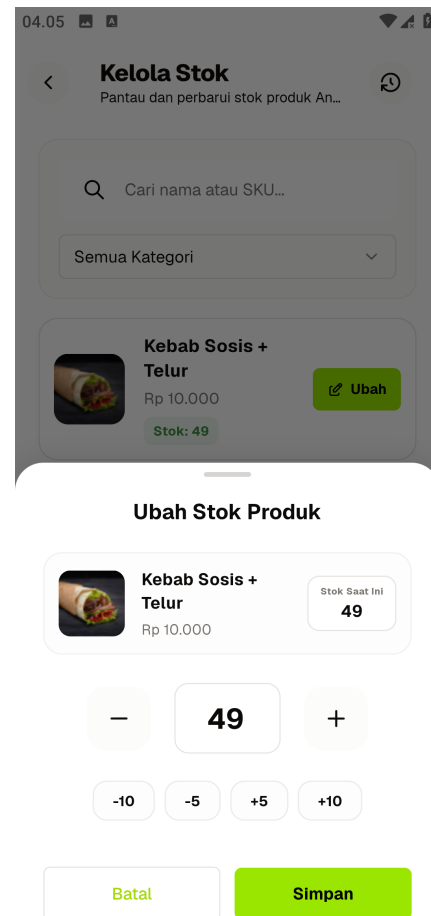
Gambar 4.7 Implementasi Halaman Kelola Kategori

8. Halaman Kelola Stok dan Ubah Stok Produk

Halaman *Kelola Stok* menampilkan daftar produk beserta harga dan jumlah stok terkini, dengan tombol *Ubah* pada setiap baris. Menekan tombol tersebut memunculkan modal *Ubah Stok Produk* berisi *stepper* penambahan/pengurangan stok manual serta pilihan pintasan (-10, -5, +5, +10) sebelum disimpan.



(a) Kelola Stok

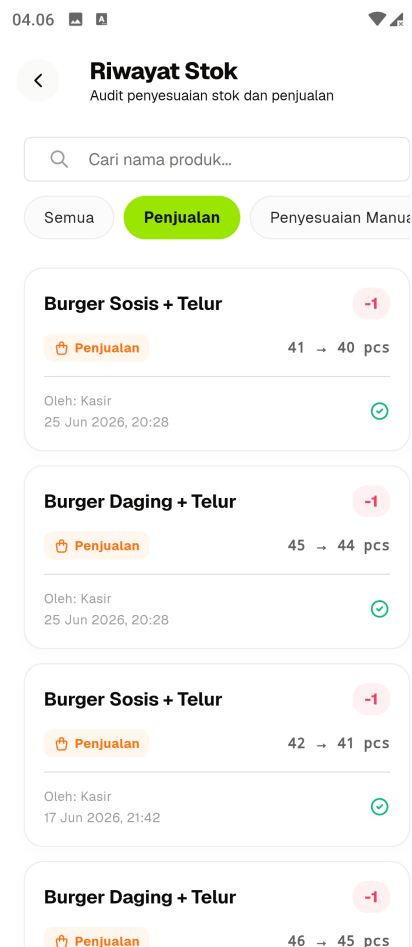


(b) Modal Ubah Stok Produk

Gambar 4.8 Implementasi Halaman Kelola Stok dan Ubah Stok Produk

9. Halaman Riwayat Stok

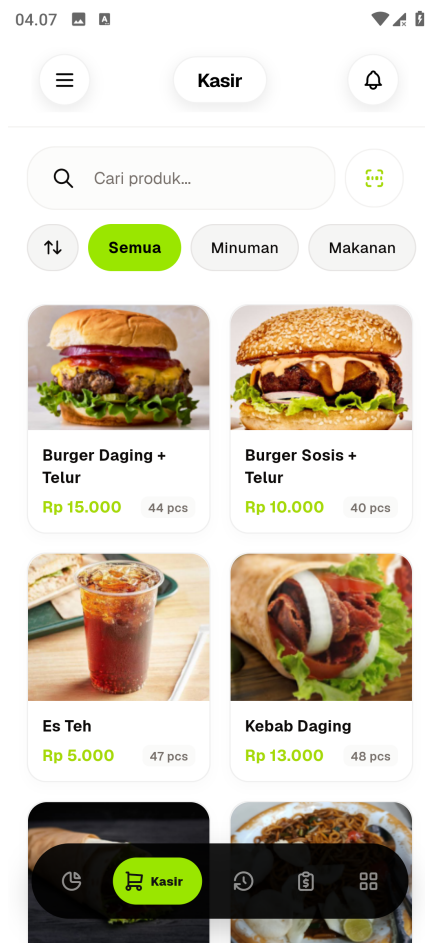
Berfungsi sebagai log audit mutasi stok, menampilkan jenis mutasi (Penjualan atau Penyesuaian Manual), perubahan jumlah stok sebelum dan sesudah (misalnya dari 41 menjadi 40 *pcs*), staf yang melakukan perubahan, serta waktu kejadian, dilengkapi *filter* berdasarkan jenis mutasi.



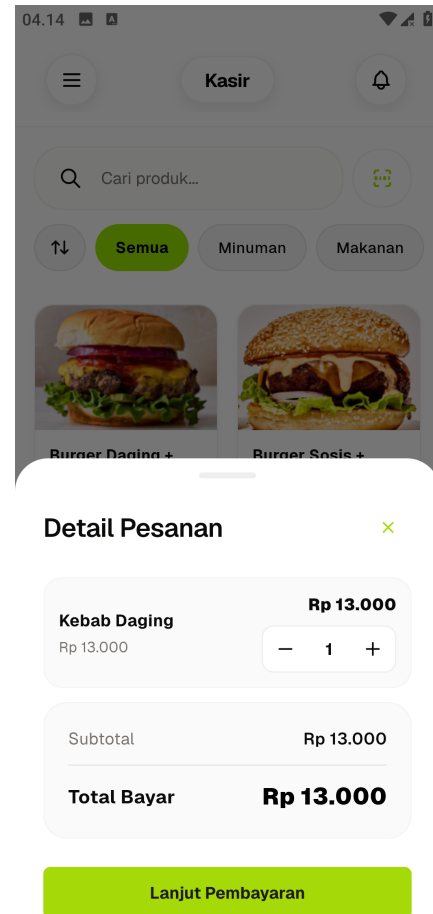
Gambar 4.9 Implementasi Halaman Riwayat Stok

10. Halaman Kasir (POS) dan Detail Pesanan

Merupakan layar operasional utama kasir, menampilkan katalog produk dalam bentuk *grid* dengan pencarian dan pemindaian *barcode*. Memilih produk memunculkan modal *Detail Pesanan* yang merinci nama item, harga satuan, kuantitas melalui *stepper*, subtotal, dan total bayar sebelum dilanjutkan ke pembayaran.



(a) Kasir (POS)

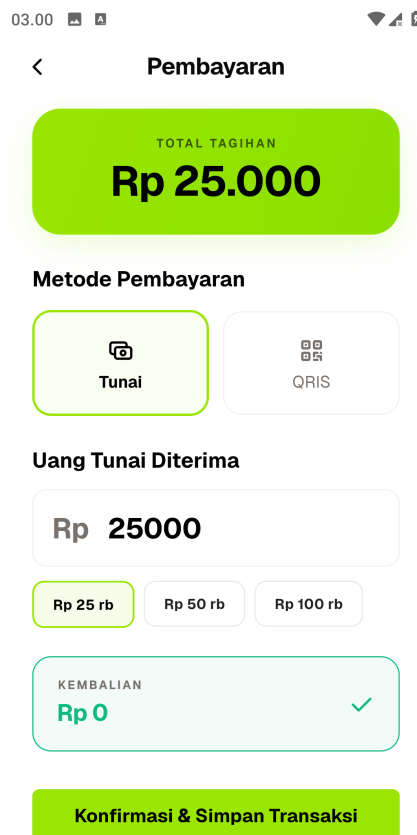


(b) Detail Pesanan

Gambar 4.10 Implementasi Halaman Kasir (POS) dan Detail Pesanan

11. Halaman Pembayaran

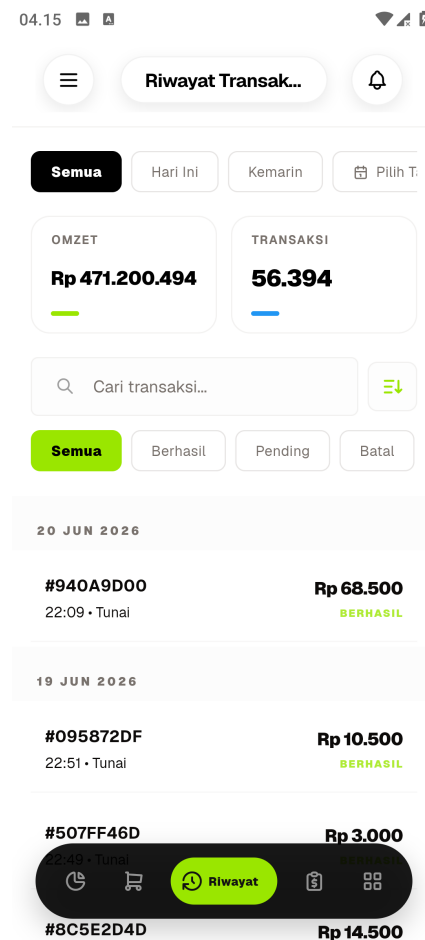
Menampilkan total tagihan, pilihan metode pembayaran (Tunai atau QRIS), kolom *Uang Tunai Diterima* beserta pintasan nominal pecahan umum (Rp25rb, Rp50rb, Rp100rb), dan kalkulasi kembalian otomatis sebelum transaksi dikonfirmasi dan disimpan.



Gambar 4.11 Implementasi Halaman Pembayaran

12. Halaman Riwayat Transaksi

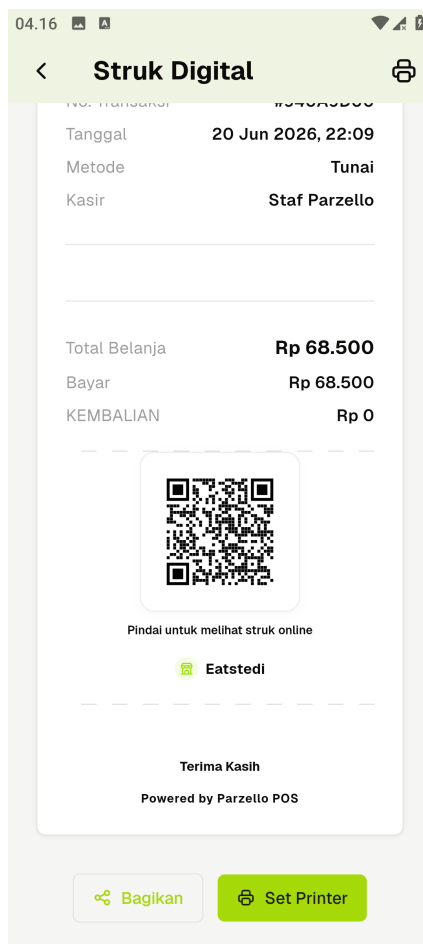
Menampilkan ringkasan kumulatif Omzet dan jumlah Transaksi, *filter* periode (Semua, Hari Ini, Kemarin, Pilih Tanggal) serta status (Berhasil, *Pending*, Batal), dan daftar transaksi yang dikelompokkan per tanggal lengkap dengan nomor invoice, jam, metode bayar, dan nominal.



Gambar 4.12 Implementasi Halaman Riwayat Transaksi

13. Halaman Struk Digital

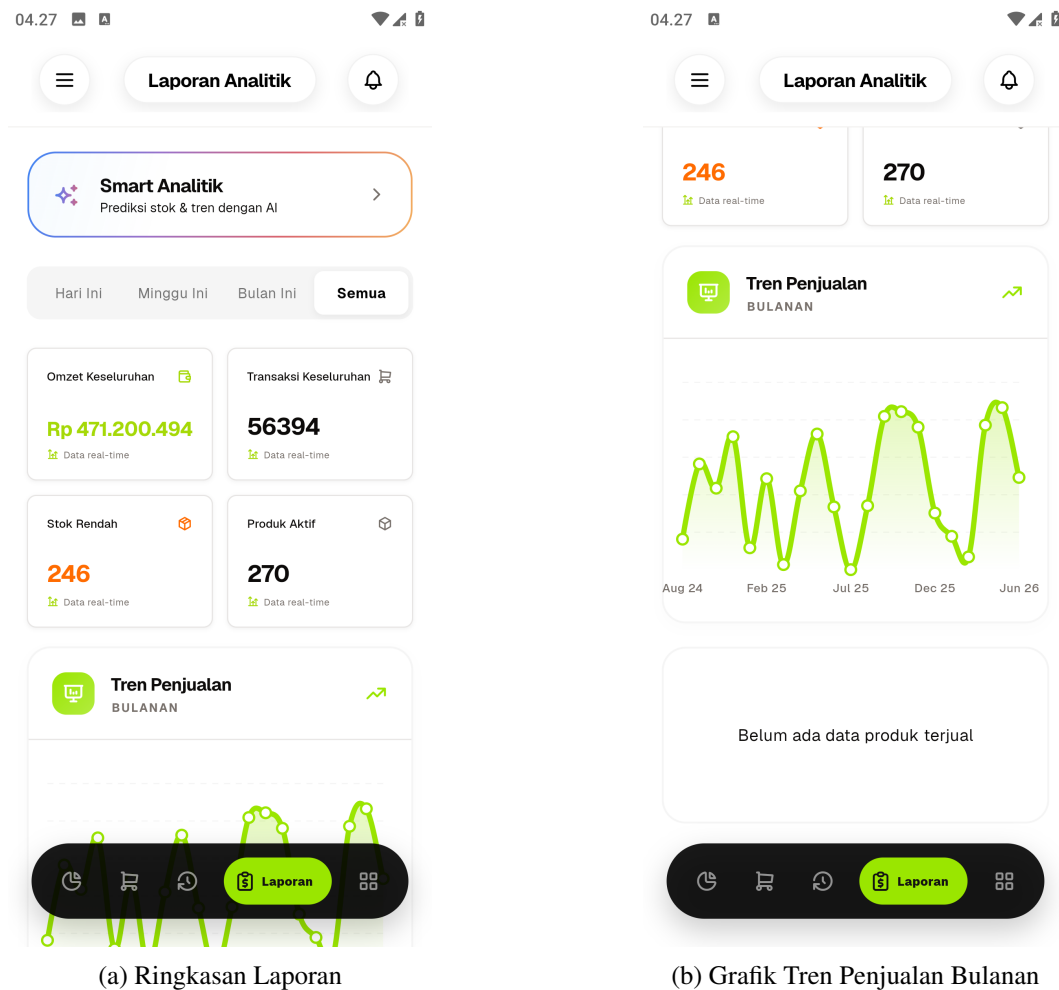
Menampilkan rincian nomor transaksi, tanggal, metode pembayaran, nama kasir, total belanja, dan kembalian, dilengkapi kode QR yang dapat dipindai pelanggan untuk melihat struk secara *online*, serta tombol *Bagikan* dan *Set Printer* untuk mencetak struk fisik.



Gambar 4.13 Implementasi Halaman Struk Digital

14. Halaman Laporan Analitik

Menyajikan ringkasan Omzet dan Transaksi Keseluruhan, jumlah Stok Rendah dan Produk Aktif, serta grafik *Tren Penjualan* bulanan yang memvisualisasikan riwayat omzet toko EatsTEDI sejak Agustus 2024 hingga Juni 2026, sejalan dengan rentang data penelitian pada Subbab 3.1.5. Kartu *Smart Analitik* pada bagian atas halaman menjadi pintu masuk menuju fitur prediksi berbasis AI.

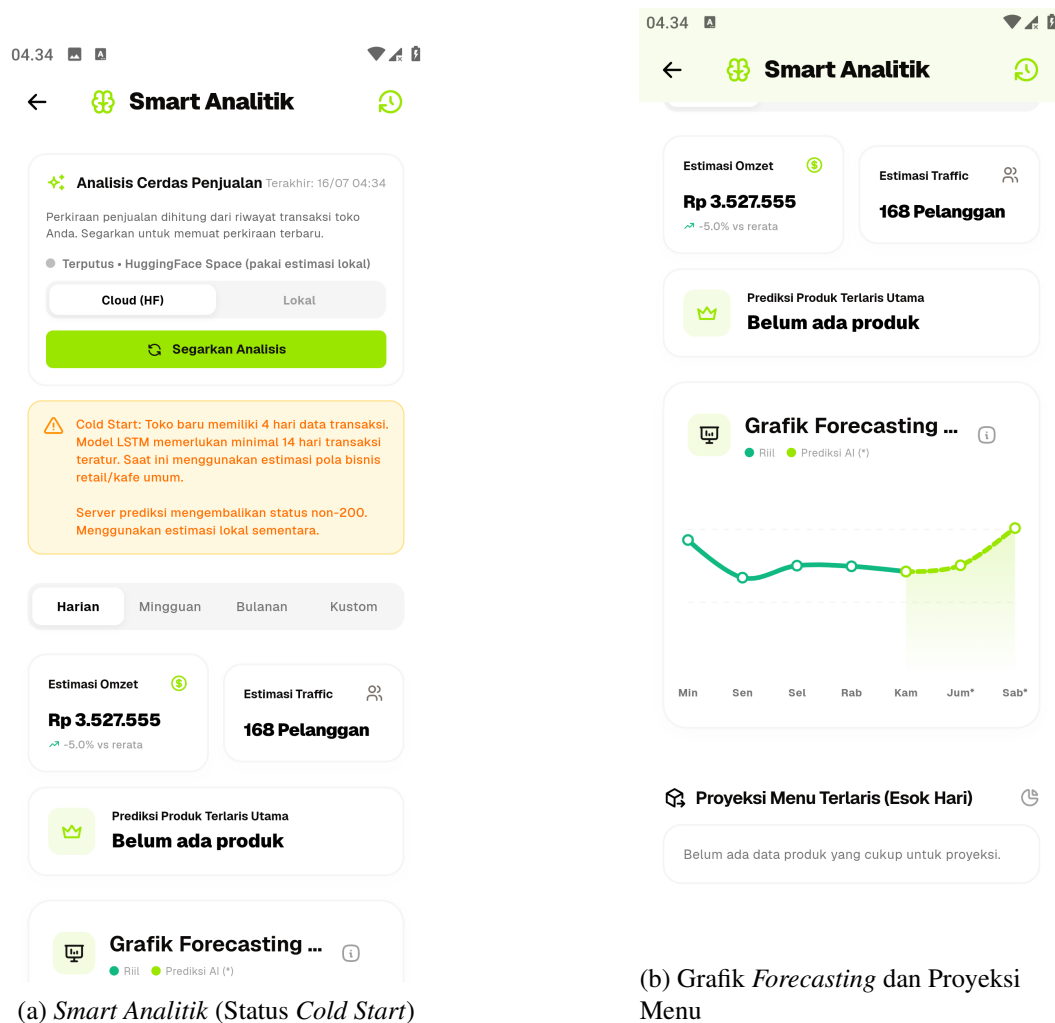


Gambar 4.14 Implementasi Halaman Laporan Analitik

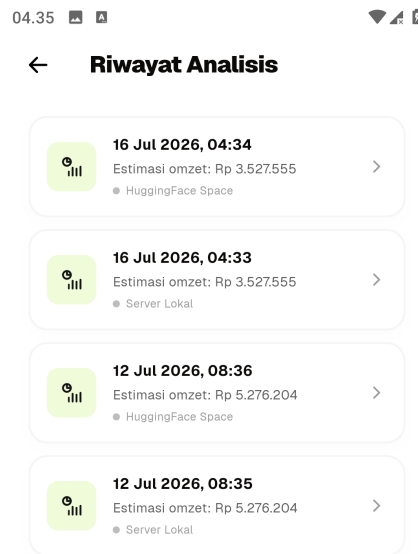
15. Halaman *Smart Analitik* dan Riwayat Analisis

Halaman *Smart Analitik* memanggil layanan prediksi yang di-*deploy* pada *HuggingFace Space (Cloud)* dengan estimasi lokal *on-device* sebagai jalur cadangan apabila layanan *cloud* tidak dapat dijangkau, konsisten dengan strategi *offline-first* yang dibahas pada Subbab 3.2.1. Ketika riwayat transaksi toko masih kurang dari batas minimum sekuens LOOK_BACK yang disyaratkan model (lihat Subbab 3.2.7), sistem menampilkan peringatan *cold start* dan menyajikan estimasi berbasis pola bisnis retail/kafe

umum sebagai gantinya. Halaman ini menampilkan kartu Estimasi Omzet dan Estimasi *Traffic*, grafik *Forecasting* yang membedakan data riil dengan hasil prediksi AI, serta proyeksi menu terlaris untuk hari berikutnya. Setiap analisis yang dijalankan turut dicatat pada halaman *Riwayat Analisis* beserta sumber komputasi (*Cloud* atau Lokal) dan estimasi omzet yang dihasilkan.



Gambar 4.15 Implementasi Halaman *Smart Analitik*



Gambar 4.16 Implementasi Halaman Riwayat Analisis

4.1.2 Implementasi Backend dan Basis Data Cloud

Infrastruktur backend cloud diimplementasikan menggunakan platform Supabase BaaS dengan database PostgreSQL. Struktur relasional database diwujudkan melalui sembilan tabel utama yang saling terhubung:

1. *stores*: Menyimpan data identitas outlet merchant (id, nama, alamat, nomor HP, URL logo, created_at).
2. *store_members*: Mengatur relasi hak akses pengguna dengan peran Owner atau Karyawan di masing-masing outlet.

3. *categories*: Mengelola daftar kategori pengelompokan produk.
4. *products*: Menyimpan katalog inventaris barang (id, nama, harga, stok, barcode, URL foto, is_synced, is_deleted).
5. *transactions*: Menyimpan data nota induk transaksi penjualan (id, total belanja, metode pembayaran, kasir_id, created_at).
6. *transaction_items*: Menyimpan rincian item produk yang terjual pada setiap transaksi penjualan (id, transaksi_id, produk_id, jumlah, harga).
7. *stock_history*: Mencatat log audit mutasi perubahan stok produk (stok masuk, keluar, penjualan, audit manual).
8. *notifications*: Mengelola pesan notifikasi sistem dan info siaran antar staf toko secara real-time.
9. *smart_analytics_snapshots*: Menyimpan riwayat *snapshot* hasil analitik prediksi LSTM setiap kali fitur *Smart Analytics* dijalankan, sehingga hasil analisis sebelumnya dapat ditinjau kembali tanpa memanggil ulang model.

Keamanan akses data antar-outlet dijamin melalui penerapan kebijakan Row Level Security (RLS) di database PostgreSQL. Kebijakan ini memastikan bahwa pengguna (kasir/owner) hanya dapat membaca dan menulis data yang memiliki *store_id* yang sesuai dengan toko tempat mereka terdaftar.

4.1.3 Implementasi Layanan Prediksi (API)

Modul kecerdasan buatan untuk kalkulasi prediksi penjualan harian dikembangkan menggunakan Python dan terdiri atas dua bagian yang saling melengkapi. Bagian pertama adalah *pipeline* riset berbasis Jupyter Notebook yang

melatih model prediksi LSTM (TensorFlow/Keras) pada tiga granularitas waktu (harian, mingguan, dan bulanan) beserta eksperimen augmentasi data sintetis TimeGAN (PyTorch). Bagian kedua adalah layanan API produksi berbasis *framework* Flask yang di-*deploy* pada platform Hugging Face dan diintegrasikan langsung dengan aplikasi POS, sehingga beban komputasi prediksi terpisah dari beban kerja transaksional kasir.

Layanan API produksi menyediakan tujuh *endpoint* REST dengan fungsi sebagai berikut:

1. **Endpoint Prediksi Penjualan** (*/api/predict/daily*, */api/predict/weekly*, */api/predict/monthly*): Meramalkan omzet ke depan dengan tanggal prediksi yang otomatis melewati hari tutup (akhir pekan serta libur semester Januari/Juli). Berdasarkan hasil evaluasi komparatif pada penelitian ini (Subbab 4.4), metode penyaji yang dipilih pada *endpoint* produksi adalah metode statistik paling stabil, yaitu *Seasonal-Naive* ($k = 4$, rata-rata empat kemunculan terakhir hari yang sama, kompromi antara responsivitas terhadap tren terbaru dan peredaman fluktuasi acak, setara dengan rentang sekitar satu bulan hari sejenis) untuk harian, *Mean-4* untuk mingguan, dan *Mean-3* untuk bulanan, dengan prediksi *Naive* disertakan sebagai pembanding, sedangkan model LSTM hasil pelatihan didokumentasikan sebagai artefak riset (*.keras*).
2. **Endpoint Rekomendasi Operasional** (*/api/recommendations/stock* dan */api/recommendations/target*): Menghasilkan rekomendasi alokasi anggaran

belanja per kategori produk dan kuantitas stok produk teratas berdasarkan proyeksi omzet (dilengkapi *safety buffer* 15% dan peringatan khusus produk mudah basi), serta tiga tingkatan target omzet harian (konservatif, moderat, dan agresif) berbasis statistik 15 hari aktif terakhir.

3. **Endpoint Pencatatan dan Status** (*/api/sales/record* dan */api/status*):

Menerima rekap penjualan harian dari aplikasi POS dan memperbaruinya ke dataset historis (*upsert* per tanggal) agar dasar prediksi selalu mengikuti data terkini, serta memeriksa kesehatan layanan dan ketersediaan berkas data.

Aplikasi Flutter memanggil *endpoint* tersebut secara langsung melalui HTTPS dengan *payload* agregat penjualan tanpa informasi identitas pelanggan, kemudian menyimpan hasil analisis sebagai *snapshot* pada tabel *smart_analytics_snapshots* di Supabase.

Inti implementasi *endpoint* prediksi harian pada berkas *app.py* disajikan pada Kode Program IV.1 (dipersingkat). Metode *Seasonal-Naive* dihitung dengan merata-ratakan *k* kemunculan terakhir dari setiap hari kerja yang sama, dan tanggal prediksi dihasilkan dengan melewati hari tutup.

```

1 @app.route('/api/predict/daily', methods=['GET', 'POST'])
2 def predict_daily():
3     data = request.get_json(silent=True) or {}
4     n_days = int(data.get("n_days", 7))
5     k = int(data.get("k", 4))
6     dfa = get_daily_dataframe(data.get("history", None))
7     last_date = dfa['date'].max()

```

```

8
9     # Seasonal-Naive: rata-rata k kemunculan terakhir
10    # dari hari kerja yang sama (0=Senin ... 4=Jumat)
11    by_dow = {}
12    for dow in range(5):
13        dow_data = dfa[dfa['day_of_week'] == dow]['revenue']
14        if len(dow_data) >= k:
15            by_dow[dow] = dow_data.tail(k).mean()
16        elif len(dow_data) > 0:
17            by_dow[dow] = dow_data.mean()
18        else:
19            by_dow[dow] = dfa['revenue'].tail(10).mean()
20
21    # Naive: prediksi = omzet hari kerja terakhir
22    naive_value = float(dfa['revenue'].iloc[-1])
23
24    # Tanggal prediksi otomatis melewati hari tutup
25    future_dates = generate_future_business_days(last_date,
26                                                    n_days)
27    predictions = []
28    for dt in future_dates:
29        pred_sn = int(round(by_dow.get(dt.weekday(),
30                                     naive_value)))
31        predictions.append({
32            "date": dt.strftime('%Y-%m-%d'),
33            "predicted_revenue_seasonal_naive": pred_sn,
34            "predicted_revenue_naive":

```

```

        int(round(naive_value)))
33     return jsonify({"predictions": predictions}), 200

```

Kode Program IV.1: Implementasi Endpoint Prediksi Harian Seasonal-Naive (*app.py*)

4.2 Preprocessing Data

Bagian ini menjelaskan tahapan pengolahan data sebelum digunakan untuk melatih model prediksi, mencakup deskripsi dan eksplorasi awal dataset, serta pra-pemrosesan dan pembentukan sequence data latih.

4.2.1 Deskripsi Data dan Eksplorasi Awal

Data yang digunakan dalam pengujian model prediksi ini merupakan rekapitulasi transaksi penjualan harian kantin EatsTEDi UGM pada periode 26 Agustus 2024 hingga 20 Juni 2026. Setelah dilakukan agregasi nilai transaksi per hari dan penyaringan terhadap hari-hari non-aktif (di mana kantin tutup), diperoleh total sebanyak 313 hari aktif bisnis. Statistik ringkas mengenai dataset pendapatan harian ini disajikan pada Tabel 4.1.

Tabel 4.1 Statistik Ringkas Dataset Pendapatan Harian

Karakteristik	Nilai
Jumlah hari aktif	313 hari
Rentang waktu	26 Agu 2024 – 20 Jun 2026
Total revenue (omzet)	Rp 471.200.494,00
Rata-rata revenue per hari aktif	Rp 1.505.433,00
Revenue tertinggi	Rp 3.738.500,00 (6 Mei 2026)

Hasil eksplorasi awal menunjukkan bahwa deret data pendapatan harian

kantin EatsTEDI bersifat sangat fluktuatif dan dipengaruhi secara kuat oleh kalender akademis universitas. Terdapat pola penurunan tajam hingga bernilai Rp 0 pada masa libur semester mahasiswa (sepanjang bulan Januari dan Juli) serta pola musiman mingguan di mana penjualan menurun secara bertahap menjelang akhir pekan. Pola musiman tahunan tidak dapat teramati secara penuh karena rentang waktu data yang tersedia kurang dari dua tahun, sehingga deret waktu ini tergolong pendek (*short time-series*) untuk standar pemodelan berbasis *deep learning* yang membutuhkan volume data historis yang besar.

4.2.2 Pra-pemrosesan dan Pembentukan Sequence

Pemodelan prediksi pada penelitian ini difokuskan pada granularitas harian dengan strategi prediksi satu langkah ke depan (*one-step-ahead prediction*). Untuk mengatasi variansi data yang ekstrem akibat fluktuasi harian yang sangat tajam, target prediksi tidak diprediksi menggunakan bentuk nilai nominal mentah, melainkan menggunakan bentuk *log-return* (selisih logaritmik) pendapatan antar hari aktif yang berurutan. Pendekatan residual ini dirancang agar model cukup mempelajari koreksi atau deviasi nilai pendapatan dari hari aktif sebelumnya, sehingga model secara tidak langsung mewarisi sifat autokorelasi jangka pendek yang kuat pada deret waktu transaksi.

Variabel masukan (*features*) terdiri atas nilai *log-return* pendapatan target dan enam fitur kalender yang dikodekan secara siklik (*cyclical encoding* sinus-kosinus untuk nomor minggu dalam setahun, bulan, dan hari kerja aktif) serta variabel biner Ramadan. Variabel jumlah transaksi harian dan kuantitas

produk terjual secara sengaja dikeluarkan dari variabel masukan model prediksi untuk menghindari kebocoran data (*data leakage*), karena kedua nilai tersebut belum tersedia saat modul prediksi dijalankan untuk memprediksi hari berikutnya. Proses penskalaan data dilakukan menggunakan metode *Min-Max Scaling* yang dipasang (*fit*) hanya pada data latih untuk mencegah terjadinya kebocoran informasi dari data uji.

Pembagian dataset dilakukan secara kronologis (*chronological split*) dengan rasio 70% untuk data latih (*train set*), 15% untuk data validasi (*validation set*), dan 15% untuk data uji (*test set*). Pembentukan urutan temporal (*sequence*) dilakukan menggunakan panjang *look-back* sebesar 7 hari aktif terakhir. Rincian pembagian data serta dimensi *sequence* disajikan pada Tabel 4.2.

Tabel 4.2 Pembagian Dataset dan Dimensi Sequence

Subset Data	Jumlah Hari	Jumlah Sequence	Dimensi Masukan
Latih (<i>train</i>)	219	212	(212, 7, 8)
Validasi (<i>val</i>)	46	39	(39, 7, 8)
Uji (<i>test</i>)	48	41	(41, 7, 8)

Implementasi pra-pemrosesan data, meliputi penyaringan hari aktif, transformasi target *log-return*, dan pengodean siklik fitur kalender, disajikan pada Kode Program IV.2.

```

1 df = pd.read_csv(DAILY_CSV, parse_dates=['date'])
2 # Hanya hari aktif (revenue > 0) yang dilatih ke model
3 dfa = df[df['revenue'] > 0].copy() \
4     .sort_values('date').reset_index(drop=True)
5

```

```

6 dfa['rev_log'] = np.log1p(dfa['revenue'])
7 # TARGET: log-return (perubahan pendapatan harian)
8 dfa['logret'] = dfa['rev_log'].diff().fillna(0.0)
9 # Pengodean siklik sinus-kosinus fitur kalender
10 dfa['week_sin'] = np.sin(2*np.pi*dfa['week_of_year']/52)
11 dfa['week_cos'] = np.cos(2*np.pi*dfa['week_of_year']/52)
12 dfa['month_sin'] = np.sin(2*np.pi*dfa['month']/12)
13 dfa['month_cos'] = np.cos(2*np.pi*dfa['month']/12)
14 dfa['dow_sin'] = np.sin(2*np.pi*dfa['day_of_week']/5)
15 dfa['dow_cos'] = np.cos(2*np.pi*dfa['day_of_week']/5)
16
17 # 'logret' wajib kolom pertama (kolom target)
18 FEATURES = ['logret', 'week_sin', 'week_cos', 'month_sin',
19             'month_cos', 'dow_sin', 'dow_cos', 'is_ramadan']
20
21 # Pembagian kronologis 70/15/15 (tanpa pengacakan)
22 n = len(dfa); n_tr = int(n*0.70); n_va = int(n*0.15)
23 df_tr = dfa.iloc[:n_tr]
24 df_va = dfa.iloc[n_tr:n_tr+n_va]
25 df_te = dfa.iloc[n_tr+n_va:]

```

Kode Program IV.2: Pra-pemrosesan Data: Filter Hari Aktif, Log-Return, dan Fitur Siklik

4.3 Implementasi dan Pelatihan Model Prediksi

Bagian ini memaparkan implementasi arsitektur model LSTM beserta skenario eksperimen, hasil pelatihan model, implementasi augmentasi data sintesis

TimeGAN, dan implementasi mekanisme prediksi rekursif untuk kebutuhan visualisasi jangka pendek pada aplikasi.

4.3.1 Arsitektur Model dan Skenario Eksperimen

Arsitektur model prediksi yang diimplementasikan adalah LSTM satu lapis (*single-layer LSTM*) dengan 24 *hidden units*, diikuti oleh lapisan *Dropout* dengan laju 0,1 untuk regularisasi, satu lapisan *Dense* dengan 16 neuron beraktivasi *Rectified Linear Unit* (ReLU), dan diakhiri oleh satu lapisan keluaran (*output layer*) tunggal beraktivasi linear. Model ini dirancang secara minimalis dengan total parameter terlatih hanya sebanyak 3.585 parameter. Pemilihan model berkapasitas rendah ini didasarkan pada jumlah data latih yang sangat terbatas (212 sampel sequence), sehingga penggunaan model LSTM berkapasitas besar (*stacked layers*) akan berisiko tinggi memicu kegagalan konvergensi pelatihan atau bias *overfitting*. Proses optimasi menggunakan metode Adam dengan *learning rate* awal sebesar 5×10^{-4} , fungsi kerugian (*loss function*) MSE, serta didukung *callbacks EarlyStopping* dan *ReduceLROnPlateau*. Konstruksi arsitektur model beserta konfigurasi pelatihannya disajikan pada Kode Program IV.3.

```

1 LOOK_BACK = 7    # jendela masukan: 7 hari aktif
2 HORIZON    = 1    # prediksi 1 langkah ke depan
3 UNITS      = 24
4 DROPOUT    = 0.1
5
6 def build_lstm():
7     m = Sequential([

```

```

8         LSTM(UNITS, input_shape=(LOOK_BACK, len(FEATURES))),
9         Dropout(DROPOUT),
10        Dense(16, activation='relu'),
11        Dense(HORIZON)
12    ], name='lstm_hybrid_residual')
13    m.compile(optimizer=Adam(5e-4), loss='mse',
14              metrics=['mae'])
15
16    return m
17
18    cb = [EarlyStopping('val_loss', patience=20,
19                      restore_best_weights=True),
20          ReduceLROnPlateau('val_loss', factor=0.5,
21                            patience=8, min_lr=1e-6)]
22
23    model = build_lstm()
24    hist = model.fit(X_tr, y_tr, validation_data=(X_va, y_va),
25                  epochs=200, batch_size=16, callbacks=cb)

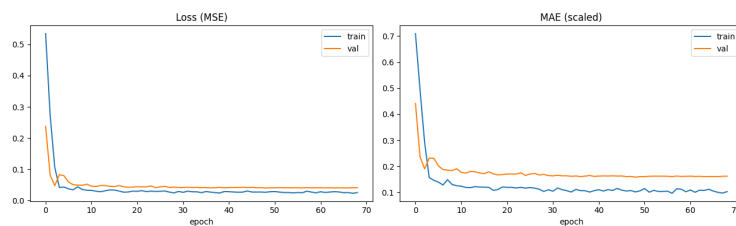
```

Kode Program IV.3: Konstruksi Arsitektur LSTM Tersederhanakan dan Konfigurasi Pelatihan

4.3.2 Hasil Pelatihan Model

Pada konfigurasi akhir model tersederhanakan, proses pelatihan berjalan secara stabil di mana galat validasi menunjukkan tren penurunan yang konsisten melampaui *epoch* pertama. Mekanisme *EarlyStopping* berhasil menghentikan proses pelatihan dan mengembalikan bobot terbaik model sebelum terjadi gejala

overfitting. Hal ini berbanding terbalik dengan uji coba pada konfigurasi awal penelitian yang menggunakan arsitektur jaringan saraf berskala besar dan target berupa nilai nominal mentah, di mana model mengalami kolaps (*model collapse*) dengan galat validasi terbaik yang dicapai langsung pada *epoch* pertama (model tidak melakukan pembelajaran) dan menghasilkan prediksi bernilai rendah yang konstan secara sistematis. Penyederhanaan arsitektur, pemilihan target *log-return*, serta pembatasan horizon menjadi satu langkah ke depan terbukti mampu memperbaiki performa pelatihan secara signifikan, di mana nilai MAE uji terbaik model LSTM mengalami penurunan drastis dari sekitar Rp 1.805.196,00 pada konfigurasi awal menjadi berkisar antara Rp 500.000,00 hingga Rp 600.000,00 pada konfigurasi akhir. Iterasi dari konfigurasi awal (arsitektur besar, target nominal) menuju konfigurasi akhir tersederhanakan inilah yang merupakan realisasi dari *loop hyperparameter tuning* dengan kriteria akurasi ($MASE < 1$) pada diagram alur Gambar 3.23. Meskipun kriteria $MASE < 1$ pada akhirnya tidak tercapai oleh konfigurasi manapun, iterasi dihentikan pada konfigurasi terbaik dan keputusan produksi dialihkan ke metode *baseline* sebagaimana dibahas pada Subbab 4.4. Grafik kurva kerugian pelatihan model ditunjukkan pada Gambar 4.17.



Gambar 4.17 Kurva Kerugian Pelatihan (*Training Loss Curve*) Model LSTM

4.3.3 Implementasi Augmentasi Data Sintetis TimeGAN

Sebagai upaya mitigasi terhadap keterbatasan ukuran dataset yang menjadi akar penyebab ketidakstabilan model LSTM, dilakukan eksperimen augmentasi data sintetis menggunakan **TimeGAN** (Yoon et al., 2019). Model TimeGAN yang diimplementasikan dengan PyTorch dilatih pada jendela deret waktu sepanjang 24 hari aktif dengan tiga fitur numerik (omzet, jumlah transaksi, dan kuantitas terjual) melalui tiga fase pelatihan (*autoencoder*, *supervised*, dan *joint adversarial training*) masing-masing sebanyak 600 *epoch*, kemudian menghasilkan 298 sekuens data harian sintetis dari 199 sekuens data riil (rasio augmentasi 1,5). Pada skenario ini, model LSTM multi-langkah (*look-back* 14 hari aktif, horizon 7 hari) selanjutnya dilatih dalam dua kondisi: hanya menggunakan 199 sekuens data riil (*baseline real-only*), dan menggunakan gabungan data riil dengan 298 sekuens sintetis (rasio augmentasi 1,5). Hasil evaluasi perbandingan kedua kondisi tersebut disajikan pada Subbab 4.4.3.

Cuplikan implementasi dua dari lima jaringan penyusun TimeGAN pada berkas *TimeGAN.py*, yaitu jaringan *embedder* yang memetakan data riil ke ruang laten dan *generator* yang memetakan derau acak menjadi representasi laten sintetis, disajikan pada Kode Program IV.4.

```

1  class EmbedderNet(nn.Module):
2
3      """Memetakan data nyata X -> representasi laten H."""
4
5      def __init__(self):
6
7          super().__init__()
8
9          self.rnn = nn.GRU(NUM_FEAT, HIDDEN_DIM,
```

```

6             NUM_LAYERS, batch_first=True)
7
8         self.proj = nn.Sequential(
9
10             nn.Linear(HIDDEN_DIM, HIDDEN_DIM), nn.Sigmoid())
11
12     def forward(self, x):
13
14         h, _ = self.rnn(x)
15
16         return self.proj(h)
17
18 class GeneratorNet(nn.Module):
19
20     """Memetakan noise Z -> laten sintetis E."""
21
22     def __init__(self):
23
24         super().__init__()
25
26         self.rnn = nn.GRU(NUM_FEAT, HIDDEN_DIM,
27
28                             NUM_LAYERS, batch_first=True)
29
30         self.proj = nn.Sequential(
31
32             nn.Linear(HIDDEN_DIM, HIDDEN_DIM), nn.Sigmoid())
33
34     def forward(self, z):
35
36         h, _ = self.rnn(z)
37
38         return self.proj(h)
39
40 # Tiga fase pelatihan (masing-masing 600 epoch):
41 # 1) train_embedder : autoencoder embedder-recovery
42 # 2) train_supervisor: dinamika temporal ruang laten
43 # 3) train_joint    : pelatihan gabungan adversarial

```

Kode Program IV.4: Implementasi Jaringan *Embedder* dan *Generator* TimeGAN

(PyTorch)

4.3.4 Implementasi Prediksi Rekursif ke Depan

Untuk kebutuhan penyajian visualisasi ramalan penjualan harian selama 7 hari aktif ke depan secara rekursif (*recursive forecasting*), model LSTM residual yang melakukan iterasi mandiri mengalami kerentanan berupa penumpukan kesalahan (*error propagation*) di mana nilai prediksi meluruh secara eksponensial hingga bernilai sangat kecil dan tidak realistis. Setelah ditambahkan mekanisme pengaman berupa *clipping* pada *log-return* (dijepit antara persentil 10–90 data latih), *damping factor* ($\phi = 0.7$) untuk meredam perubahan pada horizon yang jauh, serta *safety rail* penahan nominal omzet harian aktif, grafik prediksi yang dihasilkan menjadi lebih stabil dan realistis. Implementasi inti dari fungsi prediksi rekursif beserta ketiga mekanisme pengaman tersebut disajikan pada Kode Program IV.5.

```

1  # Batas clip dari persentil 10-90 log-return data latih
2  LR_LO, LR_HI = np.percentile(_train_logret, [10, 90])
3  REV_FLOOR = np.percentile(_act_rev, 2)    # rail bawah
4  REV_CEIL  = _act_rev.max()                # rail atas
5
6  def forecast_hybrid_stable(model, dfa, scaler, features,
7                               look_back, n_steps, phi=0.7):
8      window = scaler.transform(

```

```

9         dfa.tail(look_back)[features].values)
10     prev_rev = float(dfa['revenue'].iloc[-1])
11     d, step, rows = pd.Timestamp(dfa['date'].max()), 0, []
12     while len(rows) < n_steps:
13         d += pd.Timedelta(days=1)
14         # Lewati hari tutup (akhir pekan, Januari & Juli)
15         if d.weekday() >= 5 or d.month in [1, 7]:
16             continue
17         step += 1
18         raw_lr = float(inv_logret(
19             model.predict(window[np.newaxis])[0, 0]))
20         lr_clip = float(np.clip(raw_lr, LR_LO, LR_HI)) # 1)
21         clip
22         lr_damp = lr_clip * (phi ** (step - 1)) # 2)
23         damp
24         rev = float(np.expml(np.loglp(prev_rev) + lr_damp))
25         rev = float(np.clip(rev, REV_FLOOR, REV_CEIL)) # 3)
26         rail
27         # Susun baris fitur baru utk tanggal d (dipersingkat)
28         raw = build_feature_row(d, raw_lr)
29         # Geser jendela masukan dengan baris fitur baru
30         window = np.vstack([window[1:],
31                               scaler.transform([raw])[0]])
32         prev_rev = rev
33         rows.append({'tanggal': d.date(),
34                     'prediksi_revenue': int(round(rev))})
35     return pd.DataFrame(rows)

```

Kode Program IV.5: Prediksi Rekursif dengan *Clipping*, *Damping*, dan *Safety Rail*

Secara statistik, prediksi rekursif jangka panjang berbasis LSTM pada data terbatas ini akan dengan cepat meredam menuju nilai rata-rata historis (sifat *mean-reverting*) dan menyerupai prediksi naif. Untuk diintegrasikan ke dalam fitur aplikasi riil kasir, metode prediksi statistik *SARIMA* dan *Seasonal-Naive* (rata-rata hari sejenis pada minggu sebelumnya) lebih direkomendasikan karena memberikan hasil prediksi multi-langkah yang jauh lebih stabil dan konsisten. Rekomendasi ini diwujudkan pada layanan API produksi, di mana *endpoint* prediksi menyajikan metode *Seasonal-Naive* ($k = 4$) sebagai metode penyaji utama prediksi harian kepada aplikasi POS. Dalam penelitian skripsi ini, hasil evaluasi akurasi ilmiah tetap didasarkan pada pengujian satu langkah ke depan (*one-step-ahead*), sedangkan visualisasi prediksi multi-langkah diposisikan murni sebagai ilustrasi fungsionalitas sistem.

4.4 Evaluasi Model

Pengukuran akurasi model dilakukan menggunakan empat metrik evaluasi secara simultan, yaitu MAE, RMSE, sMAPE, dan MASE. Penggunaan MAPE konvensional dihindari karena karakteristik dataset transaksi memiliki beberapa hari aktif dengan nilai nominal pendapatan yang sangat rendah (mendekati nol), sehingga pembagian terhadap nilai aktual tersebut akan menyebabkan pembagian dengan nol atau nilai persentase galat yang sangat besar (*exploding MAPE*) yang

tidak representatif. Metrik MASE dipilih sebagai acuan performa utama karena ternormalisasi terhadap galat prediksi naif secara *in-sample*, di mana nilai MASE kurang dari satu menandakan bahwa kinerja model prediksi lebih baik dibandingkan model naif sederhana. Sebagai model pembanding (*baselines*) pada skema satu langkah ke depan, digunakan tiga metode pembanding statistik sederhana: (1) *Naive* (kemarin) di mana prediksi hari ini disamakan dengan realisasi hari kemarin, (2) *Mean-7* berupa rata-rata bergerak dari 7 hari aktif terakhir, dan (3) model statistik *SARIMA* dengan skema prediksi *rolling one-step*. Implementasi fungsi metrik evaluasi sMAPE dan MASE yang digunakan pada seluruh pengujian disajikan pada Kode Program IV.6.

```

1  def smape(y, yhat):
2      # sMAPE: galat dibagi rata-rata |aktual| dan |prediksi|
3      d = (np.abs(y) + np.abs(yhat)) / 2
4      return np.mean(np.abs(y - yhat)
5                      / np.where(d == 0, 1, d)) * 100
6
7  def mase(y, yhat, naive_mae):
8      # MASE: galat dinormalisasi thd galat naif in-sample
9      return mean_absolute_error(y, yhat) / naive_mae
10
11 def report(y, yhat, naive_mae, label):
12     rmse = np.sqrt(mean_squared_error(y, yhat))
13     mae = mean_absolute_error(y, yhat)
14     return {'rmse': rmse, 'mae': mae,
15            'smape': smape(y, yhat),

```

```
16          'mase': mase(y, yhat, naive_mae)}
```

Kode Program IV.6: Implementasi Fungsi Metrik Evaluasi sMAPE dan MASE

4.4.1 Evaluasi Model dan Perbandingan dengan Baseline

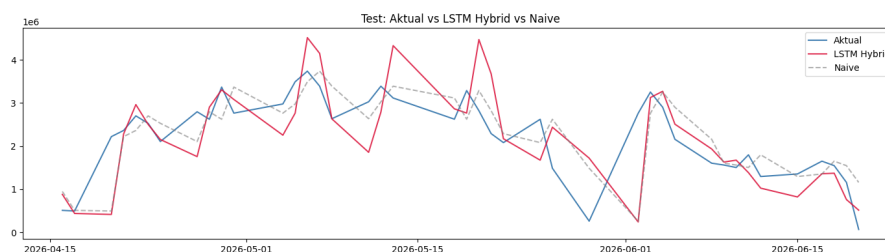
Setelah pelatihan diselesaikan, dilakukan evaluasi performa model prediksi pada dataset uji (*testing data*). Hasil evaluasi performa model LSTM Hybrid dibandingkan dengan ketiga metode pembanding (*baselines*) disajikan pada Tabel 4.3. Nilai performa LSTM Hybrid yang disajikan pada tabel merupakan nilai rata-rata dari sepuluh kali percobaan dengan inisialisasi bobot acak yang berbeda (*multi-seed*).

Tabel 4.3 Perbandingan Performa pada Data Uji (*One-Step-Ahead*)

Model	MAE (Rp)	RMSE (Rp)	sMAPE	MASE
Naive (kemarin)	519.841	697.253	30,77%	1,432
SARIMA	586.083	751.408	33,80%	1,615
Mean-7	647.263	791.746	36,25%	1,784
LSTM Hybrid (Rata-rata)	667.718	897.563	38,06%	1,840

Berdasarkan hasil perbandingan pada Tabel 4.3, secara rata-rata metode *Naive* sederhana memberikan kesalahan prediksi terkecil dengan MAE sebesar Rp 519.841,00, disusul oleh model statistik *SARIMA* dan *Mean-7*. Model LSTM Hybrid berada di posisi terakhir dengan nilai MAE rata-rata sebesar Rp 667.718,00 dan MASE sebesar 1,840. Temuan ini mengindikasikan karakteristik data pendapatan kantin EatsTEDi UGM yang didominasi oleh persistensi jangka pendek yang kuat (omzet hari ini sangat dekat dengan omzet kemarin).

Karakteristik persistensi ini dapat ditangkap secara sangat efisien oleh model naif, sementara model LSTM Hybrid yang dilatih menggunakan ukuran data yang relatif kecil mengalami *over-smoothing* (penghalusan berlebih) sehingga kurang mampu menangkap lonjakan-lonjakan pendapatan harian secara presisi. Perlu dicatat bahwa seluruh metode pada Tabel 4.3 – termasuk *Naive* itu sendiri – memiliki nilai MASE lebih besar dari satu (1,432 hingga 1,840). Hal ini terjadi karena MASE dinormalisasi terhadap galat naif satu langkah *in-sample* yang dihitung pada data latih, sehingga nilai $MASE > 1$ pada data uji semata menandakan bahwa periode uji secara umum lebih bergejolak (memiliki variansi/perubahan harian yang lebih besar) dibandingkan periode latih yang menjadi acuan normalisasi, bukan berarti seluruh metode gagal secara mutlak. Oleh karena itu, perbandingan performa yang bermakna secara metodologis pada penelitian ini adalah perbandingan MASE antar metode pada data uji yang identik (lihat kriteria kelayakan relatif pada Subbab 3.2.9.1), bukan ambang mutlak $MASE < 1$. Visualisasi perbandingan nominal pendapatan aktual terhadap nilai prediksi dari model LSTM Hybrid disajikan pada Gambar 4.18.



Gambar 4.18 Grafik Pendapatan Aktual vs. Prediksi Model LSTM Hybrid

4.4.2 Evaluasi Multi-Seed dan Analisis Kestabilan

Meningkatnya sensitivitas performa model LSTM terhadap inisialisasi bobot acak awal pada data kecil mengharuskan pengujian dilakukan menggunakan pendekatan *multi-seed evaluation* dengan melatih model sebanyak sepuluh kali menggunakan nilai *seed* acak yang berbeda. Prosedur pengujian diimplementasikan sebagai perulangan pelatihan dengan pengaturan *seed* eksplisit pada seluruh sumber keacakan (Python, NumPy, dan TensorFlow) sebagaimana disajikan pada Kode Program IV.7. Ringkasan statistik hasil pengujian sepuluh *seeds* tersebut disajikan pada Tabel 4.4.

```

1  def set_seed(s):
2      os.environ['PYTHONHASHSEED'] = str(s)
3      random.seed(s); np.random.seed(s); tf.random.set_seed(s)
4
5  rows = []
6  for s in range(N_SEEDS):    # N_SEEDS = 10
7      set_seed(s)
8      cb = [EarlyStopping('val_loss', patience=20,
9                          restore_best_weights=True, verbose=0),
10           ReduceLROnPlateau('val_loss', factor=0.5,
11                              patience=8, min_lr=1e-6,
12                              verbose=0)]
13
14     mdl = build()
15     mdl.fit(X_tr, y_tr, validation_data=(X_va, y_va),
16            epochs=200, batch_size=16, callbacks=cb, verbose=0)
17
18     # Rekonstruksi log-return prediksi ke nominal Rupiah

```

```

16     logret_hat = inv_logret (
17         mdl.predict(X_te, verbose=0).flatten())
18     y_lstm = np.expml(np.loglp(rev_prev) + logret_hat)
19     r = metrics(rev_true, y_lstm); r['seed'] = s
20     rows.append(r)
21
22 res = pd.DataFrame(rows)    # rata-rata +- deviasi per metrik

```

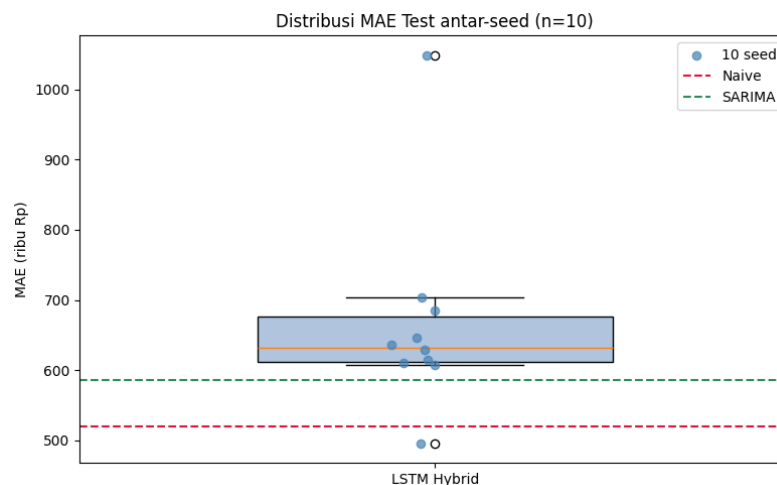
Kode Program IV.7: Prosedur Evaluasi *Multi-Seed* (10 Kali Pelatihan Ulang)

Tabel 4.4 Statistik Performa LSTM Hybrid pada 10 Seed

Metrik	Rata-rata $\pm$ Simpangan baku	Rentang (min – maks)
MAE (Rp)	667.718 $\pm$ 145.024	495.527 – 1.048.967
RMSE (Rp)	897.563 $\pm$ 165.804	703.721 – 1.320.587
sMAPE	38,06% $\pm$ 3,96%	29,16% – 44,63%
MASE	1,840 $\pm$ 0,400	1,365 – 2,891

Hasil pada Tabel 4.4 menunjukkan nilai simpangan baku (*standard deviation*) yang cukup signifikan (MAE $\pm$ Rp 145.024,00 dan MASE $\pm$ 0,400). Hal ini membuktikan adanya ketidakstabilan performa model LSTM yang disebabkan oleh ukuran data yang terbatas, performa model sangat bergantung pada keberuntungan inisialisasi parameter awal. Selain itu, sebaran data menunjukkan bahwa hanya terdapat 1 dari 10 percobaan (10%) yang berhasil mengungguli performa model pembandingan *Naive* dan *SARIMA*, yaitu pada *seed* ke-7 dengan MAE sebesar Rp 495.527,00. Oleh karena itu, pelaporan kinerja model LSTM berdasarkan uji coba tunggal (*single run*) berisiko memberikan kesimpulan yang kurang valid atau menyesatkan. Distribusi nilai MAE dari sepuluh *seeds* tersebut

divisualisasikan dalam bentuk *boxplot* pada Gambar 4.19.



Gambar 4.19 Boxplot Distribusi MAE Hasil Pengujian *Multi-Seed*

4.4.3 Evaluasi Augmentasi Data Sintetis TimeGAN

Sebagai upaya mitigasi terhadap keterbatasan ukuran dataset yang menjadi akar penyebab ketidakstabilan model LSTM, hasil implementasi augmentasi data sintetis TimeGAN (lihat Subbab 4.3.3) dievaluasi dengan membandingkan performa model LSTM multi-langkah (*look-back* 14 hari aktif, horizon 7 hari) yang dilatih dalam dua kondisi: hanya menggunakan 199 sekuens data riil (*baseline real-only*), dan menggunakan gabungan data riil dengan 298 sekuens sintetis (rasio augmentasi 1,5). Hasil perbandingan pada data uji disajikan pada Tabel 4.5.

Tabel 4.5 Perbandingan Performa Eksperimen Augmentasi TimeGAN (*Multi-Step*, Horizon 7)

Model	MAE (Rp)	RMSE (Rp)	sMAPE	MASE
Seasonal-Naive ($k = 5$)	755.403	975.130	37,58%	2,082
Seasonal-Naive ($k = 7$)	783.179	985.180	38,24%	2,158
LSTM + Augmentasi TimeGAN	860.695	1.047.842	41,49%	2,372
LSTM Tanpa Augmentasi	1.750.355	1.935.119	115,40%	4,823

Perlu dicatat bahwa nilai galat pada Tabel 4.5 tidak dapat dibandingkan secara langsung dengan Tabel 4.3, karena eksperimen augmentasi ini menggunakan skema prediksi multi-langkah langsung (*direct multi-step*, horizon 7 hari sekaligus) yang secara inheren lebih sulit daripada skema satu langkah ke depan (*one-step-ahead*).

Hasil eksperimen menunjukkan dua temuan penting. Pertama, augmentasi TimeGAN terbukti efektif memperbaiki kualitas pelatihan LSTM pada data terbatas: nilai MAE model turun drastis dari Rp 1.750.355,00 menjadi Rp 860.695,00 (perbaikan sekitar 50,8%), dan nilai MASE membaik dari 4,823 menjadi 2,372. Kedua, meskipun mengalami perbaikan signifikan, model LSTM teraugmentasi tetap belum mampu mengungguli *baseline Seasonal-Naive* ($k = 5$, MAE Rp 755.403,00) yang memanfaatkan pola musiman mingguan secara sederhana. Perlu dicatat bahwa evaluasi multi-langkah ini menguji *Seasonal-Naive* pada konfigurasi $k = 5$ dan $k = 7$, sedangkan layanan produksi memakai $k = 4$ sebagai kompromi responsivitas–stabilitas. Ketiganya menunjukkan karakteristik performa yang serupa karena sama-sama merata-ratakan beberapa kemunculan terakhir hari sejenis. Temuan ini memperkuat kesimpulan sebelumnya bahwa pada volume data saat ini, metode statistik musiman sederhana masih menjadi pilihan

paling andal untuk disajikan kepada pengguna, sekaligus menunjukkan bahwa augmentasi data sintetis merupakan arah yang menjanjikan apabila dikombinasikan dengan data riil yang lebih panjang di masa mendatang.

4.5 Hasil Pengujian Sistem

Pengujian sistem dilakukan untuk memvalidasi fungsionalitas aplikasi dan memeriksa kebenaran logika internal komponen kritis pada layanan API prediksi yang telah diintegrasikan. Pengujian mencakup uji coba fungsional *Black Box*, uji telusur logika kode *White Box*, serta rekapitulasi evaluasi kinerja model prediksi yang telah dibahas pada Subbab 4.4.

4.5.1 Skenario Pengujian

Rancangan skenario pengujian fungsional *black box* disusun untuk 14 kasus penggunaan utama yang melibatkan peran Owner dan Kasir, sebagaimana telah dijabarkan pada Tabel 3.18 (Bab III), meliputi antara lain registrasi toko, autentikasi pengguna, pengelolaan produk dan kategori, proses *checkout* transaksi, pencetakan struk, sinkronisasi *offline-first*, serta dasbor laporan analitik. Skenario pengujian *white box* disusun secara terpisah dengan menelusuri jalur logika kode pada tiga fungsi kritis di layanan API prediksi, yaitu fungsi *Seasonal-Naive* (*by_dow*), fungsi metrik evaluasi sMAPE dan MASE, serta fungsi prediksi rekursif (*forecast_hybrid_stable*), sebagaimana telah dirancang pada Subbab 3.2.9.3. Kedua rancangan pengujian ini bersifat saling melengkapi: pengujian *black box* memvalidasi kesesuaian antara masukan dan keluaran pada level antarmuka

pengguna, sedangkan pengujian *white box* memvalidasi kebenaran perhitungan pada level kode program.

4.5.2 Hasil Black Box Testing

Pengujian fungsionalitas sistem menggunakan metode Black Box diterapkan pada 14 skenario kasus penggunaan utama yang melibatkan peran Owner dan Kasir.

Hasil pengujian diringkas pada Tabel 4.6.

Tabel 4.6 Hasil Pengujian Black Box

No.	Fitur / Skenario	Hasil yang Diharapkan	Hasil Riil	Status
1	Registrasi Toko	Akun owner baru dan data toko berhasil didaftarkan di cloud database.	Sesuai Harapan	Berhasil
2	Login Pengguna	Masuk ke dasbor utama sesuai hak akses peran (Owner/Kasir).	Sesuai Harapan	Berhasil
3	Checkout Belanja	Transaksi tersimpan, stok barang berkurang otomatis di lokal.	Sesuai Harapan	Berhasil
4	Terapkan Diskon	Total belanja terpotong sesuai nilai nominal diskon/voucher.	Sesuai Harapan	Berhasil
5	Cetak Struk Fisik	Printer thermal mencetak detail nota transaksi via Bluetooth.	Sesuai Harapan	Berhasil
6	Kelola Produk	Menambah, mengedit, dan menghapus katalog produk di basis data.	Sesuai Harapan	Berhasil
7	Kelola Kategori	Mengatur kategori produk untuk pengelompokan menu kasir.	Sesuai Harapan	Berhasil
8	Audit Mutasi Stok	Riwayat keluar masuk barang tercatat di log stock_history.	Sesuai Harapan	Berhasil

No.	Fitur / Skenario	Hasil yang Diharapkan	Hasil Riil	Status
9	Dasbor Ringkasan	Menampilkan ringkasan omzet dan grafik penjualan aktual.	Sesuai Harapan	Berhasil
10	Smart Analytics	Grafik prediksi LSTM untuk 7 hari ke depan dapat ditampilkan.	Sesuai Harapan	Berhasil
11	Broadcast Notif	Pengiriman notifikasi broadcast dari owner muncul di layar kasir.	Sesuai Harapan	Berhasil
12	Sinkronisasi Data	Data transaksi offline terunggah otomatis ke cloud saat online.	Sesuai Harapan	Berhasil
13	Kelola Staf Karyawan	Owner dapat menambah dan mengatur peran staf kasir.	Sesuai Harapan	Berhasil
14	Atur Profil Toko	Informasi nama, alamat, dan logo toko berhasil diubah.	Sesuai Harapan	Berhasil

Berdasarkan hasil uji coba pada Tabel 4.6, seluruh fungsi utama aplikasi POS mobile dan server-side API berjalan dengan status sukses ("Berhasil"), membuktikan kelayakan operasional sistem untuk penggunaan harian.

Sebagai bagian dari skenario #12 (Sinkronisasi Data), pengujian fitur *offline-first* dilakukan lebih rinci untuk memastikan keandalan penyimpanan lokal menggunakan Isar DB ketika koneksi internet terputus dan proses sinkronisasi otomatis ketika koneksi terhubung kembali. Rincian skenario pengujian sinkronisasi tersebut dijabarkan pada Tabel 4.7.

Tabel 4.7 Hasil Pengujian Mekanisme Sinkronisasi

No	Skenario Pengujian	Hasil Pengujian	Status
1	Melakukan transaksi checkout saat jaringan terputus (Luring).	Transaksi tersimpan di Isar DB lokal dengan status <i>is_synced = false</i> (penanda yang sama dinamai <i>isSynced</i> pada lapisan kode Dart, mengikuti konvensi penamaan masing-masing platform). Stok produk berkurang di lokal.	Berhasil
2	Membuka riwayat transaksi kasir saat luring.	Menampilkan daftar transaksi lokal secara instan tanpa memicu error atau halaman kosong.	Berhasil
3	Menghubungkan perangkat kembali ke internet (Daring).	Mesin <i>SyncNotifier</i> mendeteksi koneksi dan mengunggah data transaksi ke Supabase.	Berhasil
4	Memeriksa status database cloud setelah sinkronisasi.	Data transaksi masuk ke Supabase, dan status di lokal berubah menjadi <i>is_synced = true</i> .	Berhasil

Hasil uji coba membuktikan bahwa arsitektur offline-first menjamin operasional POS kasir tetap berjalan tanpa kendala saat konektivitas internet tidak stabil, serta menjaga konsistensi data transaksi saat online kembali.

4.5.3 Hasil White Box Testing

Pengujian *white box* dilakukan dengan menelusuri jalur logika (*code path*) dari tiga fungsi kritis pada layanan API prediksi menggunakan data masukan yang telah diketahui nilai keluarannya secara manual, kemudian membandingkan hasil

eksekusi aktual terhadap hasil kalkulasi *ground truth*. Ringkasan hasil penelusuran jalur logika disajikan pada Tabel 4.8.

Tabel 4.8 Hasil Pengujian White Box

No	Fungsi / Jalur Logika	Kondisi Uji	Hasil yang Diharapkan	Status
1	<i>by_dow</i> (Seasonal-Naive)	Data historis hari kerja tersedia $\geq k$ (jalur utama)	Nilai prediksi merupakan rata-rata k kemunculan terakhir hari kerja yang sama	Berhasil
2	<i>by_dow</i> (Seasonal-Naive)	Data historis hari kerja tersedia $< k$ (jalur cabang <i>partial</i>)	Nilai prediksi merupakan rata-rata dari seluruh data hari kerja yang tersedia	Berhasil
3	<i>by_dow</i> (Seasonal-Naive)	Data historis hari kerja tidak tersedia sama sekali (jalur <i>fallback</i>)	Nilai prediksi menggunakan rata-rata 10 hari aktif terakhir keseluruhan	Berhasil
4	<i>smape</i>	Nilai aktual dan prediksi sama-sama bernilai 0 (jalur penyebut nol)	Fungsi tidak menghasilkan galat pembagian dengan nol (<i>division by zero</i>)	Berhasil
5	<i>mase</i>	Galat naif <i>in-sample</i> bernilai positif normal (jalur utama)	Nilai MASE terhitung sesuai rasio MAE model terhadap MAE naif	Berhasil
6	<i>forecast_hybrid_stable</i>	Nilai <i>log-return</i> mentah melebihi persentil 90 data latih (jalur <i>clipping</i>)	Nilai <i>log-return</i> dipotong tepat pada batas atas persentil 90	Berhasil
7	<i>forecast_hybrid_stable</i>	Iterasi horizon ke- n dengan $n > 1$ (jalur <i>damping</i>)	Besaran perubahan <i>log-return</i> teredam sesuai faktor $\phi^{(n-1)}$	Berhasil
8	<i>forecast_hybrid_stable</i>	Nilai omzet hasil rekonstruksi melampaui nilai maksimum historis (jalur <i>safety rail</i>)	Nilai omzet dipotong tepat pada batas <i>REV_CEIL</i>	Berhasil

Berdasarkan hasil pada Tabel 4.8, seluruh jalur logika kritis pada ketiga fungsi yang diuji, yaitu perhitungan *Seasonal-Naive*, metrik evaluasi sMAPE/MASE, dan prediksi rekursif, telah tervalidasi menghasilkan keluaran yang sesuai dengan perhitungan manual, sehingga tidak ditemukan kesalahan logika (*logic error*) pada percabangan kode yang diuji.

4.5.4 Evaluasi Kinerja Model

Evaluasi kinerja model telah dipaparkan secara rinci pada Subbab 4.4.

4.6 Pembahasan

Bagian ini membahas temuan hasil implementasi, kontribusi fitur offline-first terhadap kelancaran operasional kasir, serta kemampuan analitik prediksi LSTM dalam meminimalkan masalah persediaan barang.

4.6.1 Pembahasan Pengujian Fungsional dan Arsitektur Offline-First

Hasil pengujian Black Box pada 14 skenario menunjukkan tingkat kesuksesan 100% ("Berhasil"). Integrasi antara basis data lokal Isar DB dan basis data cloud Supabase terbukti sangat andal. Melalui skema sinkronisasi yang dirancang, kasir tidak perlu bergantung pada koneksi internet saat menginput transaksi. Hal ini menghilangkan risiko terhambatnya antrean kasir akibat koneksi internet yang lambat atau terputus secara tiba-tiba.

Penggunaan local database Isar DB juga memberikan performa loading data yang sangat cepat. Secara observasi, operasi baca-tulis produk dan transaksi lokal

terasa hampir seketika (*near-instant*) dan jauh lebih responsif dibandingkan operasi yang bergantung pada pemanggilan REST API database cloud secara langsung, yang turut dipengaruhi oleh kondisi dan latensi jaringan. Hal ini meningkatkan efisiensi kerja kasir secara signifikan. Perlu dicatat bahwa perbandingan ini bersifat observasi kualitatif, bukan hasil pengukuran latensi terstruktur (mis. jumlah sampel, alat ukur, dan kondisi jaringan yang terkendali), sehingga angka waktu tempuh kuantitatif belum dapat dilaporkan pada penelitian ini.

4.6.2 Pembahasan Hasil Prediksi Penjualan LSTM

Secara keseluruhan, hasil pengujian menunjukkan bahwa pada dataset transaksi penjualan kantin EatsTEDI UGM dengan 313 hari aktif, model LSTM, termasuk varian hybrid residual yang merupakan konfigurasi terbaik, tidak mengungguli metode pembandingan sederhana (*Naive* dan *Mean-7*) maupun model statistik *SARIMA* secara rata-rata, dan hanya mampu kompetitif pada kasus terbaiknya (1 dari 10 percobaan *multi-seed*). Hasil evaluasi ini bukan merupakan suatu kegagalan atau anomali sistem, melainkan sejalan dengan temuan luas dalam berbagai literatur prediksi deret waktu komersial ritel (seperti pada kompetisi prediksi global M3 dan M4). Literatur tersebut membuktikan bahwa metode statistik klasik dan naif kerap mengungguli pendekatan pembelajaran mendalam (*deep learning*) ketika data historis yang tersedia relatif terbatas dan deret waktu didominasi secara kuat oleh komponen persistensi jangka pendek.

Dalam konteks operasional kantin kampus EatsTEDI UGM, dominasi

komponen persistensi ini sangat nyata karena nilai pendapatan harian sangat berkorelasi kuat dengan nilai pendapatan hari aktif sebelumnya. Pola ketergantungan jangka pendek ini ditangkap dengan sangat presisi oleh model *Naive* (kemarin) yang memiliki kesalahan MAE terendah sebesar Rp 519.841,00. Di sisi lain, model LSTM Hybrid yang memiliki kapasitas parameter lebih besar cenderung melakukan penghalusan nilai prediksi secara berlebihan (*over-smoothing*) pada data latih yang pendek, sehingga tidak mampu menangkap lonjakan-lonjakan ekstrem omzet harian yang dipicu oleh dinamika aktivitas mahasiswa di lapangan.

Sebagai upaya mengatasi akar permasalahan keterbatasan data tersebut, penelitian ini turut menguji peran augmentasi data sintetis menggunakan TimeGAN. Hasil eksperimen menunjukkan bahwa penambahan sekuens sintetis mampu memperbaiki kualitas pembelajaran model LSTM secara substansial (penurunan MAE sekitar 50,8% dibandingkan model tanpa augmentasi pada skema multi-langkah), yang menandakan bahwa kegagalan LSTM pada penelitian ini memang bersumber dari kelangkaan data, bukan dari kesalahan arsitektur semata. Namun demikian, model LSTM teraugmentasi tetap belum melampaui *baseline Seasonal-Naive*, sehingga augmentasi sintetis diposisikan sebagai pelengkap yang menjanjikan, bukan pengganti, dari akumulasi data transaksi riil jangka panjang.

Kontribusi ilmiah dari penelitian ini terletak pada ketelitian metodologis evaluasi model yang diterapkan, yaitu:

1. **Penggunaan Model Pembanding (*Baselines*) yang Tepat:** Menyandingkan model LSTM dengan metode naif, rata-rata bergerak, dan SARIMA untuk memvalidasi apakah penambahan kompleksitas komputasi model AI memberikan nilai tambah akurasi prediksi yang sepadan.
2. **Pemilihan Metrik Evaluasi yang Valid:** Menggunakan sMAPE dan MASE yang tahan terhadap pembagian bernilai nol atau sangat kecil pada hari-hari libur, menghindari kesalahan *exploding MAPE* yang menyesatkan.
3. **Pengujian Kestabilan melalui Multi-Seed:** Melaporkan sebaran nilai performa model dari sepuluh kali pelatihan secara acak, sehingga mengungkap ketidakstabilan model LSTM pada ukuran data yang kecil secara transparan.
4. **Eksplorasi Augmentasi Data Sintetis:** Menguji efektivitas TimeGAN sebagai strategi mitigasi keterbatasan data pada prediksi penjualan UMKM, lengkap dengan kuantifikasi perbaikan galat terhadap model tanpa augmentasi dan posisinya relatif terhadap *baseline* musiman.

Kerangka evaluasi ini menghasilkan kesimpulan analisis yang jujur, objektif, dan dapat dipertanggungjawabkan secara akademik, sekaligus memberikan rekomendasi praktis bagi pengelola bisnis: untuk skala volume data transaksi harian UMKM saat ini, metode *Naive*, *Seasonal-Naive*, atau *SARIMA* lebih direkomendasikan untuk diintegrasikan sebagai fitur visualisasi dasbor prediksi penjualan harian pada aplikasi POS, sedangkan penerapan algoritma LSTM baru relevan untuk ditinjau kembali di masa depan apabila volume data transaksi

historis telah bertambah secara signifikan (mencakup minimal 3–5 tahun operasional). Rekomendasi tersebut telah diimplementasikan secara nyata pada penelitian ini: layanan API prediksi produksi yang terhubung ke aplikasi POS menyajikan metode *Seasonal-Naive* ($k = 4$) untuk prediksi harian, *Mean-4* untuk mingguan, dan *Mean-3* untuk bulanan, sementara model LSTM beserta hasil eksperimen augmentasi TimeGAN didokumentasikan sebagai artefak riset yang siap diaktifkan kembali ketika data telah memadai.

4.7 Keterbatasan Sistem dan Penelitian

Meskipun sistem POS mobile dengan fitur prediksi penjualan harian ini telah berhasil diimplementasikan dan diuji, terdapat beberapa keterbatasan operasional maupun metodologi penelitian yang harus diperhatikan.

4.7.1 Keterbatasan Sistem POS

Beberapa keterbatasan operasional dari sistem POS mobile yang dikembangkan meliputi:

1. **Keterbatasan Cold Start pada Toko Baru:** Model prediksi LSTM memerlukan data transaksi historis runtun waktu yang kontinu. Sebagaimana dijelaskan pada Subbab 3.1.6, model secara teknis membutuhkan minimal 7 hari aktif transaksi (panjang *sequence LOOK_BACK*) agar dapat dijalankan sama sekali, namun hasil proyeksi baru dianggap cukup andal untuk ditampilkan tanpa peringatan *cold start* setelah data transaksi terkumpul minimal selama 30 hari sebagai periode pemanasan awal (*warm-up period*).

2. **Ketergantungan Data Terkini:** Meskipun operasional transaksi kasir dapat berjalan tanpa internet (offline), kualitas proyeksi modul prediksi LSTM bergantung pada kelengkapan data transaksi aktual yang menjadi *payload* permintaan analisis. Jika data transaksi terbaru belum tercatat atau tersinkronkan, akurasi hasil proyeksi prediksi untuk hari berikutnya dapat menurun karena tidak menangkap tren transaksi paling akhir.
3. **Ketergantungan Akses Server untuk Hasil Prediksi:** Karena keterbatasan sumber daya komputasi dan konsumsi baterai perangkat mobile, proses kalkulasi prediksi model LSTM yang berat dialihkan sepenuhnya ke layanan inferensi Python yang di-*hosting* pada platform Hugging Face. Akibatnya, meskipun kasir dapat bertransaksi secara offline, pengguna tetap memerlukan koneksi internet aktif untuk memperbarui visualisasi prediksi analitik terbaru dari API inferensi ke dasbor aplikasi. Selain itu, layanan inferensi gratis pada Hugging Face dapat mengalami kondisi *cold start* (jeda pemuatan saat *server* baru aktif kembali dari *idle*) yang memperlambat respons analisis pertama.

4.7.2 Keterbatasan Penelitian Model Prediksi

Keterbatasan utama dalam pemodelan prediksi pada penelitian ini meliputi:

1. **Keterbatasan Volume Dataset:** Jumlah data transaksi aktif riil dari platform EatsTEDI UGM relatif sedikit untuk standar pemodelan pembelajaran mendalam (*deep learning*), yakni hanya 313 hari aktif yang menyusut menjadi sekitar 212 sampel latih setelah pembentukan *sequence*.

Keterbatasan ini berdampak langsung pada tingginya sensitivitas model terhadap inisialisasi bobot acak awal (*seed variance*) yang memicu ketidakstabilan akurasi model.

2. **Keterbatasan Pengamatan Pola Musiman Jangka Panjang:** Rentang waktu transaksi yang kurang dari dua tahun belum memungkinkan model LSTM untuk mempelajari pola musiman tahunan (*annual seasonality*) secara utuh dan komprehensif.
3. **Deret Waktu yang Terputus-putus:** Banyaknya hari non-aktif transaksi akibat libur kalender akademik kampus (seperti sepanjang bulan Januari dan Juli) serta akhir pekan menyebabkan deret waktu menjadi patah dan terputus-putus, yang menyulitkan jaringan saraf dalam mempelajari dependensi keterkaitan temporal jangka panjang secara kontinu.
4. **Bias Penyaringan Hari Aktif ($\text{revenue} > 0$):** Penyaringan data pada tahap *preprocessing* hanya mempertahankan hari kerja yang memiliki pendapatan positif ($\text{revenue} > 0$), sebagaimana dijelaskan pada Subbab 3.1.7. Konsekuensinya, hari kerja aktif dengan penjualan nol yang sebenarnya nyata turut terbuang dari dataset, bukan hanya akhir pekan atau libur kalender akademik. Akibat lanjutannya, deret *log-return* r_t yang menjadi target model dihitung antar hari aktif berurutan yang jarak kalender riilnya tidak seragam – dapat berupa 1 hari, namun juga dapat mencapai lebih dari 10 hari setelah masa libur panjang. Ketidakteraturan interval waktu ini melanggar sebagian asumsi keteraturan deret waktu (*regular*

time-series) yang mendasari arsitektur LSTM, dan diduga turut menjadi salah satu faktor yang memperberat kesulitan model dalam mempelajari pola dependensi temporal secara konsisten.

BAB V

PENUTUP

Bab ini menyajikan kesimpulan dari hasil perancangan, implementasi, dan pengujian sistem Point of Sale (POS) mobile dengan fitur prediksi penjualan harian menggunakan metode Long Short-Term Memory (LSTM) dan arsitektur offline-first, serta saran-saran untuk pengembangan sistem lebih lanjut di masa mendatang.

5.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, implementasi, dan pengujian yang telah dilakukan pada penelitian ini, maka dapat ditarik beberapa kesimpulan sebagai berikut:

1. Penelitian ini telah berhasil membangun aplikasi *Point of Sale (POS) mobile* berbasis Flutter yang terintegrasi dengan basis data cloud Supabase PostgreSQL dan layanan API prediksi penjualan harian berbasis Python dan Flask. Modul riset LSTM berbasis TensorFlow/Keras dikembangkan dan didokumentasikan secara lengkap sebagai artefak penelitian, sedangkan layanan API produksi yang benar-benar dipanggil aplikasi menyajikan metode *Seasonal-Naive* berdasarkan hasil evaluasi komparatif, melalui arsitektur *prediction service* yang bersifat *model-agnostic* dan siap menyajikan model `.keras` begitu data mencukupi.
2. Penerapan arsitektur *offline-first* menggunakan basis data lokal Isar DB terbukti handal dalam menjaga keberlangsungan transaksi kasir tanpa

ketergantungan penuh pada koneksi internet. Secara observasi, operasi baca-tulis lokal terasa jauh lebih responsif dibandingkan operasi yang bergantung pada jaringan (belum diukur secara kuantitatif dan terstruktur), dan sinkronisasi otomatis menggunakan modul *SyncNotifier* berhasil mengunggah data transaksi lokal ke database cloud saat perangkat terhubung kembali ke internet.

3. Implementasi metode prediksi *Long Short-Term Memory* (LSTM) dengan pendekatan *hybrid residual* (memprediksi *log-return*) pada dataset harian aktif menghasilkan rata-rata kesalahan prediksi (*multi-seed evaluation*) berupa MAE sebesar Rp 667.718,00 $\pm$ Rp 145.024,00, RMSE sebesar Rp 897.563,00 $\pm$ Rp 165.804,00, sMAPE sebesar 38,06% $\pm$ 3,96%, dan MASE sebesar 1,840 $\pm$ 0,400. Hasil ini menunjukkan bahwa model LSTM Hybrid yang dikembangkan belum mampu mengungguli metode pembandingan *Naive* (MAE Rp 519.841,00, MASE 1,432) dan *SARIMA* secara rata-rata, dikarenakan deret pendapatan yang pendek serta tingginya persistensi jangka pendek pada data penjualan kantin EatsTEDI UGM. Upaya augmentasi data sintetis menggunakan TimeGAN terbukti menurunkan galat model LSTM multi-langkah secara signifikan (MAE membaik dari Rp 1.750.355,00 menjadi Rp 860.695,00), namun tetap belum mengungguli *baseline Seasonal-Naive*, sehingga layanan API prediksi produksi menyajikan metode *Seasonal-Naive* sebagai metode penyaji utama.
4. Hasil pengujian fungsionalitas menggunakan metode *Black Box Testing*

terhadap 14 skenario kasus penggunaan utama yang melibatkan peran Owner dan Kasir berjalan dengan sukses tanpa kegagalan (tingkat keberhasilan 100%), menunjukkan sistem layak dioperasikan secara penuh pada unit usaha UMKM.

5.2 Saran

Berdasarkan keterbatasan sistem yang diidentifikasi selama proses pengembangan dan pengujian, berikut adalah beberapa saran konstruktif untuk pengembangan sistem lebih lanjut:

1. **Penerapan Metode Prediksi Hybrid:** Untuk meningkatkan akurasi proyeksi penjualan pada data yang memiliki volatilitas tinggi atau tren musiman yang sangat dinamis, disarankan untuk menggabungkan model LSTM dengan model *Gated Recurrent Unit* (GRU) atau model statistik klasik ARIMA menjadi model *hybrid* (seperti ARIMA-LSTM atau LSTM-GRU).
2. **Mengatasi Masalah Cold Start:** Untuk memfasilitasi toko baru yang belum memiliki riwayat transaksi selama 30 hari, disarankan mengimplementasikan teknik *transfer learning* atau melatih model *global* awal menggunakan data transaksi historis dari toko sejenis sebelum melatih model secara spesifik menggunakan data toko tersebut.
3. **Penguatan Keamanan Data Lokal:** Guna melindungi data transaksi sensitif pelaku UMKM yang disimpan secara lokal pada perangkat fisik,

disarankan untuk mengaktifkan fitur enkripsi basis data lokal pada Isar DB dengan menggunakan kunci enkripsi dinamis yang dikelola secara aman oleh perangkat.

4. **Penyediaan Skalabilitas Tinggi (High Availability):** Untuk menghadapi potensi ekspansi multi-outlet dengan ribuan kasir aktif, arsitektur modul AI berbasis Flask disarankan dimigrasikan ke infrastruktur cloud dengan konfigurasi *load balancer*, *Docker container clusters*, dan replikasi basis data Supabase PostgreSQL untuk meminimalisasi *single point of failure*.
5. **Notifikasi Multi-Kanal Otomatis:** Mengembangkan fitur integrasi notifikasi dengan layanan pihak ketiga seperti WhatsApp API atau SMS Gateway untuk mengirimkan laporan ringkasan omzet harian secara langsung ke nomor WhatsApp Owner, serta memberikan peringatan dini jika terdapat stok produk tertentu yang berada di bawah ambang batas minimum (*safety stock*).
6. **Integrasi Sistem Pembayaran Digital (E-Wallet & QRIS Dynamic):** Menambahkan integrasi langsung dengan penyedia gerbang pembayaran (*payment gateway*) lokal untuk menghasilkan kode QRIS dinamis secara real-time pada layar transaksi kasir, sehingga mempermudah pembukuan dan meminimalkan kesalahan input nominal transaksi nontunai oleh kasir.
7. **Penambahan Fitur `gap_days` pada Variabel Masukan Model:** Untuk mengurangi dampak ketidakseragaman interval waktu antar hari aktif akibat penyaringan `revenue > 0` (lihat Subbab 4.7.2), disarankan menyertakan

fitur `gap_days` (jarak hari kalender terhadap hari aktif sebelumnya) sebagai variabel masukan tambahan pada model prediksi, sehingga model dapat mempelajari secara eksplisit pengaruh jeda waktu antar observasi terhadap besaran perubahan pendapatan.

DAFTAR PUSTAKA

- [1] S. Mansur, K. Sattar, S. E. Hosseini, S. Pervez, I. Ahmad, K. Saleem, and A. Z. Elhendi, "Sales Forecasting for Retail Stores Using Hybrid Neural Networks and Sales-Affecting Variables," *PeerJ Computer Science*, vol. 11, e3058, 2025, doi: 10.7717/peerj-cs.3058.
- [2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] M. Mitra, S. Roy, S. De, S. Bhattacharyya, J. Platos, and V. Snasel, "Harnessing LSTM Neural Networks and Hyperparameter Optimization for Precise Sales Forecasting in Retail," in *Proc. 10th International Conference on Advanced Intelligent Systems and Informatics (AISIS 2024)*, Lecture Notes on Data Engineering and Communications Technologies, vol. 220. Cham, Switzerland: Springer, 2024, doi: 10.1007/978-3-031-71619-5_11.
- [4] T. de Castro Moraes, X.-M. Yuan, and E. P. Chew, "Hybrid Convolutional Long Short-Term Memory Models for Sales Forecasting in Retail," *Journal of Forecasting*, vol. 43, no. 5, pp. 1278–1293, 2024, doi: 10.1002/for.3073.
- [5] "Financial Time Series Augmentation Using Transformer Based GAN Architecture," *arXiv preprint arXiv:2602.17865*, 2026. [*Preprint*, belum melalui *peer-review*].
- [6] J. Yoon, D. Jarrett, and M. van der Schaar, "Time-series Generative Adversarial Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 5508–5518, 2019.
- [7] P. Tang, Z. Li, X. Wang, X. Liu, and P. Mou, "Time Series Data Augmentation for Energy Consumption Data Based on Improved TimeGAN," *Sensors*, vol. 25, no. 2, 2025, doi: 10.3390/s25020493.
- [8] A.-A. Semenoglou, E. Spiliotis, and V. Assimakopoulos, "Data Augmentation for Univariate Time Series Forecasting with Neural Networks," *Pattern Recognition*, vol. 134, art. no. 109132, 2023, doi: 10.1016/j.patcog.2022.109132.
- [9] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 Competition: Background, Organization, and Implementation," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1325–1336, 2022, doi: 10.1016/j.ijforecast.2021.07.007.
- [10] P. Yadav and C. R. Barde, "Retail Real-Time Sales Prediction System Using LSTM and XGBoost," *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)*, vol. 14, no. 4, 2025.

- [11] R. A. Sembiring and Y. Sary, “Analisa dan Implementasi *Long Short-Term Memory* (LSTM) dalam Kebutuhan Persediaan Barang di PT. Gunung Sari Indonesia,” *Jurnal Ilmiah Teknik Mesin, Elektro dan Komputer*, 2025.
- [12] U. M. Jannah dkk., “Analisis Prediksi Penjualan Isi Ulang Air Galon Menggunakan Metode LSTM dan SARIMA,” *Jurnal Teknologi Informasi dan Multimedia*, 2025.
- [13] R. Verdiyanto, D. Hartanti, and E. Purwanto, “Pengembangan Aplikasi *Point of Sales* untuk Prediksi Penjualan Harian Usaha Minuman Menggunakan Algoritma *Random Forest Regression*,” *Infotek: Jurnal Informatika dan Teknologi*, vol. 8, no. 2, 2025.
- [14] E. M. Sipayung, C. Fiarni, and Wawan, “Evaluasi Penggunaan Aplikasi *Point of Sale* Menggunakan *Technology Acceptance Model* pada UMKM,” *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, vol. 9, no. 1, pp. 18–24, 2020, doi: 10.22146/jnteti.v9i1.116.
- [15] M. Siddik and S. Samsir, “Rancang Bangun Sistem Informasi POS (*Point of Sale*) untuk Kasir Menggunakan Konsep Bahasa Pemrograman Orientasi Objek,” *JOISIE (Journal of Information Systems and Informatics Engineering)*, vol. 4, no. 1, p. 43, 2020, doi: 10.35145/joisie.v4i1.607.
- [16] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [17] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2015.
- [18] R. J. Hyndman and A. B. Koehler, “Another Look at Measures of Forecast Accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.
- [19] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative Adversarial Nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, pp. 2672–2680, 2014.
- [20] Google LLC, “Flutter Documentation,” 2026. [Daring]. Tersedia: <https://docs.flutter.dev/> [Diakses: 29 Juli 2026].
- [21] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [22] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2011.
- [23] ISO/IEC/IEEE 29119-4:2021, *Software and Systems Engineering — Software*

Testing — Part 4: Test Techniques, International Organization for Standardization, Geneva, Switzerland, 2021.

LAMPIRAN

LEMBAR KONSULTASI PEMBIMBING I
LEMBAR KONSULTASI SKRIPSI
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
PROGRAM STUDI TEKNIK INFORMATIKA

Nama : Muhammad Kholis
NIM : 2022573010098
Judul TA : Rancang Bangun Aplikasi *Point of Sale* dengan
Fitur Prediksi Penjualan Harian Menggunakan
Metode *Long Short-Term Memory* (LSTM)
Pembimbing I : Zulfan Khairil Simbolon, S.T., M.Eng.
NIP : 196909021993031004

No.	Tanggal	Hasil Konsultasi	Paraf Pembimbing

LEMBAR KONSULTASI PEMBIMBING II
LEMBAR KONSULTASI SKRIPSI
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
PROGRAM STUDI TEKNIK INFORMATIKA

Nama : Muhammad Kholis
NIM : 2022573010098
Judul TA : Rancang Bangun Aplikasi *Point of Sale* dengan
Fitur Prediksi Penjualan Harian Menggunakan
Metode *Long Short-Term Memory* (LSTM)
Pembimbing II : Azhar, S.T., M.T.
NIP : 196408301990031005

No.	Tanggal	Hasil Konsultasi	Paraf Pembimbing